Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Ria Chaudhari** of **D20A** semester **VIII,** have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026.**

Subject Teacher
Name:Mrs. Kajal Joseph
Signature:

Head of Department
Name: Dr. Mrs. Shalu Chopra
Signature:

Lab Teacher
Name: Kajal Joseph
Signature:

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	**Computing & Emerging Technology:** Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	**Professionalism & Innovation :** Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	**PO**											**PSO**	
	PO 1	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PSO 1**	**PSO 2**
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Ria Chaudhari D20A14

Blockchain Lab Exp 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Theory:

1.Cryptographic Hash functions in Blockchain

A **cryptographic hash function** is a mathematical algorithm that converts input data of any size into a fixed-length output (called a *hash* or *digest*).In blockchain, the most commonly used hash function is **SHA-256**.

Properties of Cryptographic Hash Functions

Deterministic – Same input always produces the same hash.
Fixed Output Size – No matter the input size, output length is fixed.
Pre-image Resistance – Cannot reverse the hash to find original input.
Collision Resistance – Hard to find two different inputs with the same hash.
Avalanche Effect – Small input change → completely different hash.

2.What is a Merkle Tree?

A Merkle Tree (Hash Tree) is a tree structure where:

- Each leaf node is a hash of a transaction.
- Each non-leaf node is a hash of its two child hashes.
- The top hash is called the Merkle Root.

It is used to efficiently verify data integrity in blockchain.

3.What is a Cryptographic Puzzle and explain the Golden Nonce

A cryptographic puzzle is a mathematical problem that:

- Is hard to solve
- Is easy to verify

In blockchain, this is used in Proof-of-Work (PoW).

Miners must find a number called a nonce such that:

```
Hash(Block Data + Nonce) → starts with required number of zeros
```

Example:

```
0000ab34f567...
```

The difficulty increases as required leading zeros increase.

What is the Golden Nonce?

The Golden Nonce is the special nonce value that solves the cryptographic puzzle.

It is the number that makes the block hash satisfy the required difficulty condition.

Example:

`Block data + 45231 → hash starts with 00000`

Here, `45231` is the Golden Nonce.

Miners try millions of nonces until they find this one.

4.How does a Merkle Tree work?

Let's say we have 4 transactions:`T1, T2, T3, T4`

Step 1: Hash all transactions

```
H1 = hash(T1)
H2 = hash(T2)
H3 = hash(T3)
H4 = hash(T4)
```

Step 2: Combine pairwise and hash again

```
H12 = hash(H1 + H2)
H34 = hash(H3 + H4)
```

Step 3: Final combination

```
Merkle Root = hash(H12 + H34)
```

If the number of transactions is odd → duplicate the last transaction.

This final hash (Merkle Root) is stored inside the block header.

5.Benefits of Merkle Tree

1. Efficient Verification:You don't need all transactions to verify one transaction.

2. Saves Memory:Lightweight nodes (SPV nodes) only store block headers.

3. Fast Detection of Tampering:If one transaction changes → Merkle Root changes.

4. Scalability:Efficient even with thousands of transactions.

6.Use cases of Merkle Tree

1. Blockchain (Bitcoin, Ethereum):Used to store transaction hashes.

2. Distributed Systems:Used in IPFS and distributed storage verification.

3. Git Version Control:Git uses Merkle Trees to track file changes.

4. Data Integrity Verification:Used to verify large datasets efficiently.

5. Peer-to-Peer Networks:Ensures file chunks are not modified.

Codes:

1.Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

```
Enter a string: ria
Hash: 543a37820bd6021085abf00b6ca801c9ec9bb025acdf7afb34de50d9988c12a4
```

2.Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Output:

```
Enter a string: chaudhari
Enter the nonce: 2
Hash Code: c6bc2db45850dbeec58d3efe6f221415b5609ff9750aa54e95ab9677ae0f60c7
```

3.Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
```

```
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

4.Generating Merkle Tree for given set of Transactions

```
import hashlib

def build_merkle_tree(transactions):
```

```
    if len(transactions) == 0:
        return None

    # Step 1: Hash all transactions first
    print("Initial Transaction Hashes:")
    transactions = [
        hashlib.sha256(tx.encode('utf-8')).hexdigest()
        for tx in transactions
    ]
    for h in transactions:
        print(h)
    print()
  # Step 2: Build the Merkle Tree
    level = 1
    while len(transactions) > 1:
        print(f"Level {level} Hashes:")

        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined =
hashlib.sha256(combined.encode('utf-8')).hexdigest()
            print(hash_combined)
            new_transactions.append(hash_combined)

        print()
        transactions = new_transactions
```

```
        level += 1

    return transactions[0]
# Example usage with valid transactions
transactions = [
    "Ria -> Sneha : $200",
    "Bob -> Dave : $500",
    "Dave -> Ria : $100",
    "Eve -> Alice : $300",
    "Ria -> Bob : $50"
]
merkle_root = build_merkle_tree(transactions)
print("Final Merkle Root:", merkle_root)
```

Output:

```
Initial Transaction Hashes:
01f9026c654756e9af874e75ceb8138e6520a61f6def7098feb361de346b4db7
768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
212de6601ed008a56290ec1bef60bfe5f64ee6695e81cf8e239bf4b93865d92d
cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
47a5e8cd41dea1884c88c9b426ad6182afe1b10ddd822498f4372ff8d13a26c7

Level 1 Hashes:
72a4dab01d66f62b8ea35f2b502b3ba36704915d45a9e5a808085ba741e4ce0b
0f3b9f14696b169cb363b1513fc188b5150ca44c6a5696b1c5eb86517457bd52
8b469f9dc1eb4fd441458ed7a8415c33c6dd15a7345eda6399c58a968f49b55f

Level 2 Hashes:
975ce11db2641d7a674d671349d7628496b0f408b8479f9bdbe03baa3e199d03
99c108e700e59123346a90ba73339efd46cf5a83a312c914f1b230f5b51a7d5a

Level 3 Hashes:
71893fb777e72ee25d103fdd49c518832d820f593c50e83a6a9311954c198f66

Final Merkle Root: 71893fb777e72ee25d103fdd49c518832d820f593c50e83a6a9311954c198f66
```

Conclusion:

Cryptographic hash functions secure blockchain data by ensuring integrity and immutability. Merkle Trees efficiently organize and verify large numbers of transactions using a single Merkle Root.Cryptographic puzzles in Proof-of-Work maintain network security by requiring computational effort.Together, these mechanisms make blockchain systems secure, transparent, and tamper-resistant.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain & DLT Lab

Experiment 02

Aim:Create a Blockchain using Python

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Ria Chaudhari D20A14

BLOCKCHAIN LAB EXP 2

AIM: Create a Blockchain using Python

THEORY:

1.What is Blockchain?

Blockchain is a decentralized and distributed digital ledger used to record transactions across many computers in a secure and tamper-proof manner. The data is stored in units called blocks, and each block is connected to the previous one using cryptographic hashes, forming a chain. Every block contains transaction data, its own hash, and the hash of the previous block, which ensures integrity and transparency. Once information is added to the blockchain, it becomes extremely difficult to modify, making the system reliable and trustworthy. Blockchain technology is widely used in cryptocurrencies, supply chain management, healthcare, and many other applications where secure record keeping is required.

2.Process of Mining

Mining is the process of adding a new block to the blockchain.Miners compete to solve a mathematical puzzle (Proof of Work).

Steps:

1. Transactions are broadcast to the network.
2. Miners collect them into a block.
3. They try to find a **nonce** value.
4. When nonce is combined with block data → it must produce a hash that satisfies a rule (example: starts with `0000`).
5. First miner who finds it → broadcasts the solution.
6. Other nodes verify it.
7. If valid → block is added → miner gets reward.

Mining provides: Security, Consensus,New coin generation

3.How to Check the Validity of Blocks in a Blockchain

To verify blockchain integrity, nodes check mainly three things:

1. Hash correctness

Recalculate the block hash and compare it with the stored hash.

If different → block is invalid.

2. Previous hash link

The block's `previous_hash` must match the hash of the previous block.

If not → chain is broken.

3. Proof of Work validity

Check whether the hash satisfies the difficulty rule
(example: starts with required number of zeros).

CODE:

```
# Importing libraries

import datetime # To generate timestamps for blocks

import hashlib # To generate SHA-256 hashes

import json # To convert block data into JSON format

from flask import Flask, jsonify # To create REST API endpoints

# Part 1 - Building the Blockchain

class Blockchain:

def __init__(self):

# List to store the blockchain

self.chain = []

# Create the genesis block

self.create_block(proof=1, previous_hash='0')

def create_block(self, proof, previous_hash):

"""

Create a new block and add it to the blockchain

"""

block = {

'index': len(self.chain) + 1,

'timestamp': str(datetime.datetime.now()),

'proof': proof,

'previous_hash': previous_hash

}

self.chain.append(block)
```

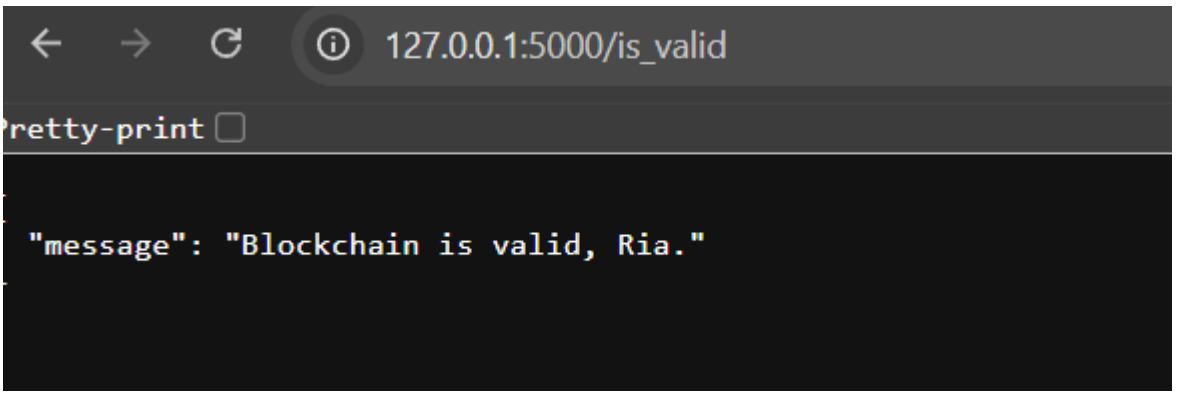
```
return block
def get_previous_block(self):
"""
Return the last block in the chain
"""
return self.chain[-1]
def proof_of_work(self, previous_proof):
"""
Proof of Work Algorithm:
Find a number such that the hash of
(new_proof^2 - previous_proof^2)
starts with '0000'
"""
new_proof = 1
check_proof = False
while not check_proof:
hash_operation = hashlib.sha256(
str(new_proof**2 - previous_proof**2).encode()
).hexdigest()
if hash_operation[:4] == '0000':
check_proof = True
else:
new_proof += 1
return new_proof
def hash(self, block):
"""
```

```
Create a SHA-256 hash of a block
"""
encoded_block = json.dumps(block, sort_keys=True).encode()
return hashlib.sha256(encoded_block).hexdigest()
def is_chain_valid(self, chain):
"""
Check whether the blockchain is valid
"""
previous_block = chain[0]
block_index = 1
while block_index < len(chain):
block = chain[block_index]
# Check previous hash
if block['previous_hash'] != self.hash(previous_block):
return False
# Check proof of work
previous_proof = previous_block['proof']
proof = block['proof']
hash_operation = hashlib.sha256(
str(proof**2 - previous_proof**2).encode()
).hexdigest()
if hash_operation[:4] != '0000':
return False
previous_block = block
block_index += 1
return True
```

```
# Part 2 - Mining the Blockchain (Flask API)

# Create Flask app

app = Flask(__name__)

# Create Blockchain object

blockchain = Blockchain()

# API Endpoint - Mine a new block

@app.route('/mine_block', methods=['GET'])

def mine_block():

previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

proof = blockchain.proof_of_work(previous_proof)

previous_hash = blockchain.hash(previous_block)

block = blockchain.create_block(proof, previous_hash)

response = {

'message': 'Congratulations Ria, you just mined a block!',

'index': block['index'],

'timestamp': block['timestamp'],

'proof': block['proof'],

'previous_hash': block['previous_hash']

}

return jsonify(response), 200

# API Endpoint - Get full blockchain

@app.route('/get_chain', methods=['GET'])

def get_chain():

response = {

'chain': blockchain.chain,
```

```
'length': len(blockchain.chain)

}

return jsonify(response), 200

# API Endpoint - Check blockchain validity

@app.route('/is_valid', methods=['GET'])

def is_valid():

valid = blockchain.is_chain_valid(blockchain.chain)

if valid:

response = {'message': 'Blockchain is valid, Ria.'}

else:

response = {'message': 'Blockchain is NOT valid, Ria.'}

return jsonify(response), 200

# Run the Flask Application

if __name__ == '__main__':

app.run(host='0.0.0.0', port=5000)
```

OUTPUT:



CONCLUSION:

In this experiment, a simple blockchain was successfully implemented using Python and Flask. The system allows block mining, viewing the complete blockchain, and checking its validity. Proof-of-Work was used to make block creation secure, while cryptographic hashing ensured that the data stored in blocks remains unchanged. This implementation helps in understanding the basic concepts of blockchain such as immutability, consensus, and decentralization, and provides a good foundation for learning advanced blockchain applications.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Ria Chaudhari D20A14

Blockchain Lab Experiment 3

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

THEORY:

Q1.Challenges in P2P Networks

- No central authority to control the network.
- All nodes must agree on one blockchain (**consensus problem**).
- Some nodes may send fake or malicious data.
- Network delays can cause different chain versions.
- Difficult to maintain synchronization as network grows.
- Nodes may go offline → system must still work.

Q2.How Transactions are Performed on the Network

- The user sends a transaction request.
- Transaction is broadcast to all nodes.
- It is stored temporarily in the **mempool**.
- Miner selects transactions from mempool.
- Miner solves the Proof of Work.
- New block is created.
- Block is added to the blockchain.
- Updated chain is shared with other nodes

Q3.Role of Mempool

A mempool (memory pool) is a temporary storage area for unconfirmed transactions.
Functions of Mempools:

● Stores pending transactions before they are added to a block.

● Helps miners select transactions for block creation.

● Prevents network congestion by organizing transaction queues.

Q4.Libraries & Tools Used

Tools

- **Postman** → send GET & POST requests.
- **VS Code / Terminal** → run servers.

Python Libraries

- **Flask** → create APIs/endpoints.
- **datetime** → timestamp for each block.
- **hashlib** → SHA-256 hashing.
- **jsonify** → send response in JSON format.
- **request** → get data from POST request.
- **uuid4** → generate unique miner ID.
- **urlparse** → extract node address.
- **requests** → communicate with other nodes.

CODE:

```
# Module 2 - Create a Cryptocurrency

# To be installed:

# Flask==0.12.2: pip install Flask==0.12.2

# Postman HTTP Client: https://www.getpostman.com/

# requests==2.18.4: pip install requests==2.18.4

# Importing the libraries

import datetime

import hashlib

import json
```

```
from flask import Flask, jsonify, request

import requests

from uuid import uuid4 # Generate a unique id that is in hex

from urllib.parse import urlparse # To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

def __init__(self):

self.chain = []

self.transactions = [] # Adding transactions before they are added to a block

self.create_block(proof = 1, previous_hash = '0')

self.nodes = set() # Set is used as there is no order to be maintained as the nodes can be from all around the globe

def create_block(self, proof, previous_hash):

block = {'index': len(self.chain) + 1,

'timestamp': str(datetime.datetime.now()),

'proof': proof,

'previous_hash': previous_hash,

'transactions': self.transactions} # Adding transactions to make the blockchain a cryptocurrency

self.transactions = [] # The list of transacction should become empty after they are added to a block

self.chain.append(block)

return block

def get_previous_block(self):

return self.chain[-1]

def proof_of_work(self, previous_proof):

new_proof = 1

check_proof = False
```

```
while check_proof is False:

hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()

if hash_operation[:4] == '0000':

check_proof = True

else:

new_proof += 1

return new_proof

def hash(self, block):

encoded_block = json.dumps(block, sort_keys = True).encode()

return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):

previous_block = chain[0]

block_index = 1

while block_index < len(chain):

block = chain[block_index]

if block['previous_hash'] != self.hash(previous_block):

return False

previous_proof = previous_block['proof']

proof = block['proof']

hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()

if hash_operation[:4] != '0000':

return False

previous_block = block

block_index += 1

return True

# This method will add the trqnsaction to the list of trnsactions
```

```
def add_transaction(self, sender, receiver, amount):

self.transactions.append({'sender': sender,

'receiver': receiver,

'amount': amount})

previous_block = self.get_previous_block()

return previous_block['index'] + 1 # It will return the block index to which the transaction should be added

# This function will add the node containing an address to the set of nodes created in init function

def add_node(self, address):

parsed_url = urlparse(address) # urlparse will parse the url from the address

self.nodes.add(parsed_url.netloc) # Add is used and not append as it's a set. Netloc will only return '127.0.0.1:5000'

# Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the nodes on the network

def replace_chain(self):

network = self.nodes # network variable is the set of nodes all around the globe

longest_chain = None # It will hold the longest chain when we scan the network

max_length = len(self.chain) # This will hold the length of the chain held by the node that runs this function

for node in network:

response = requests.get(f'http://{node}/get_chain') # Use get chain method already created to get the length of the chain

if response.status_code == 200:

length = response.json()['length'] # Extract the length of the chain from get_chain fiunction

chain = response.json()['chain']

if length > max_length and self.is_chain_valid(chain): # We check if the length is bigger and if the chain is valid then

max_length = length # We update the max length
```

```
longest_chain = chain # We update the longest chain

if longest_chain: # If longest_chain is not none that means it was replaced

self.chain = longest_chain # Replace the chain of the current node with the longest chain

return True

return False # Return false if current chain is the longest one

# Part 2 - Mining our Blockchain

# Creating a Web App

app = Flask(__name__)

# Creating an address for the node on Port 5000. We will create some other nodes as well on different ports

node_address = str(uuid4()).replace('-', '') #

# Creating a Blockchain

blockchain = Blockchain()

# Mining a new block

@app.route('/mine_block', methods = ['GET'])

def mine_block():

previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

proof = blockchain.proof_of_work(previous_proof)

previous_hash = blockchain.hash(previous_block)

blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) # Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block

block = blockchain.create_block(proof, previous_hash)

response = {'message': 'Congratulations, you just mined a block!',

'index': block['index'],

'timestamp': block['timestamp'],

'proof': block['proof'],
```

```
'previous_hash': block['previous_hash'],

'transactions': block['transactions']}

return jsonify(response), 200

# Getting the full Blockchain

@app.route('/get_chain', methods = ['GET'])

def get_chain():

response = {'chain': blockchain.chain,

'length': len(blockchain.chain)}

return jsonify(response), 200

# Checking if the Blockchain is valid

@app.route('/is_valid', methods = ['GET'])

def is_valid():

is_valid = blockchain.is_chain_valid(blockchain.chain)

if is_valid:

response = {'message': 'All good. The Blockchain is valid.'}

else:

response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}

return jsonify(response), 200

# Adding a new transaction to the Blockchain

@app.route('/add_transaction', methods = ['POST']) # Post method as we have to pass
something to get something in return

def add_transaction():

json = request.get_json() # This will get the json file from postman. In Postman we will create
a json file in which we will pass the values for the keys in the json file

transaction_keys = ['sender', 'receiver', 'amount']

if not all(key in json for key in transaction_keys): # Checking if all keys are available in json

return 'Some elements of the transaction are missing', 400
```

```
index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])

response = {'message': f'This transaction will be added to Block {index}'}

return jsonify(response), 201 # Code 201 for creation

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes

@app.route('/connect_node', methods = ['POST']) # POST request to register the new nodes from the json file

def connect_node():

json = request.get_json()

nodes = json.get('nodes') # Get the nodes from json file

if nodes is None:

return "No node", 400

for node in nodes:

blockchain.add_node(node)

response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:',

'total_nodes': list(blockchain.nodes)}

return jsonify(response), 201

# Replacing the chain by the longest chain if needed

@app.route('/replace_chain', methods = ['GET'])

def replace_chain():

is_chain_replaced = blockchain.replace_chain()

if is_chain_replaced:

response = {'message': 'The nodes had different chains so the chain was replaced by the longest one.',

'new_chain': blockchain.chain}

else:

response = {'message': 'All good. The chain is the largest one.',
```

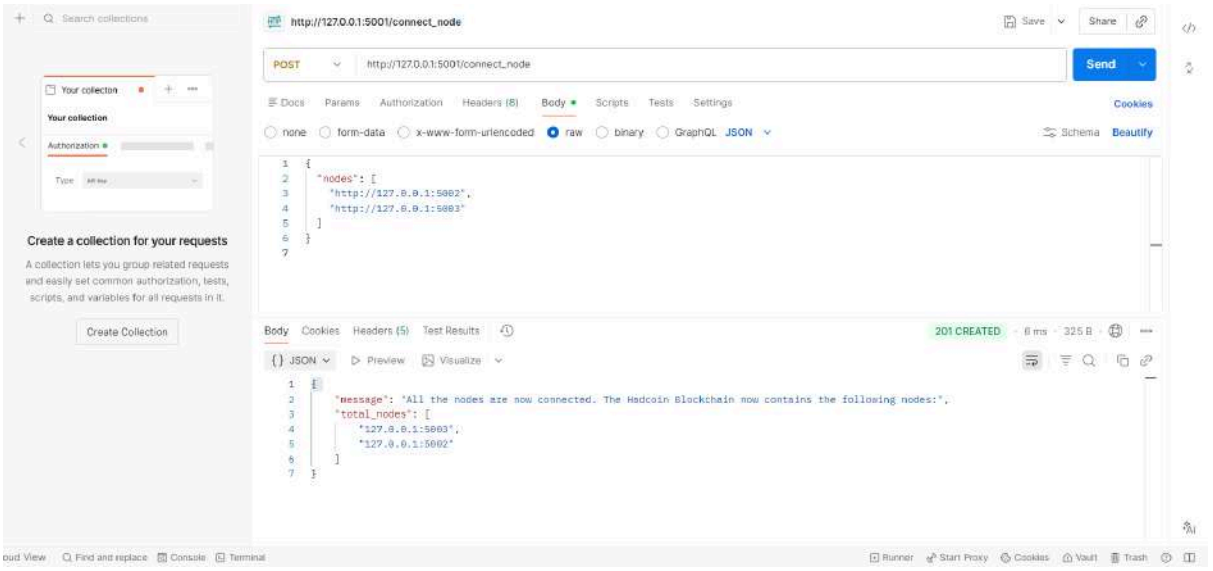
'actual_chain': blockchain.chain}

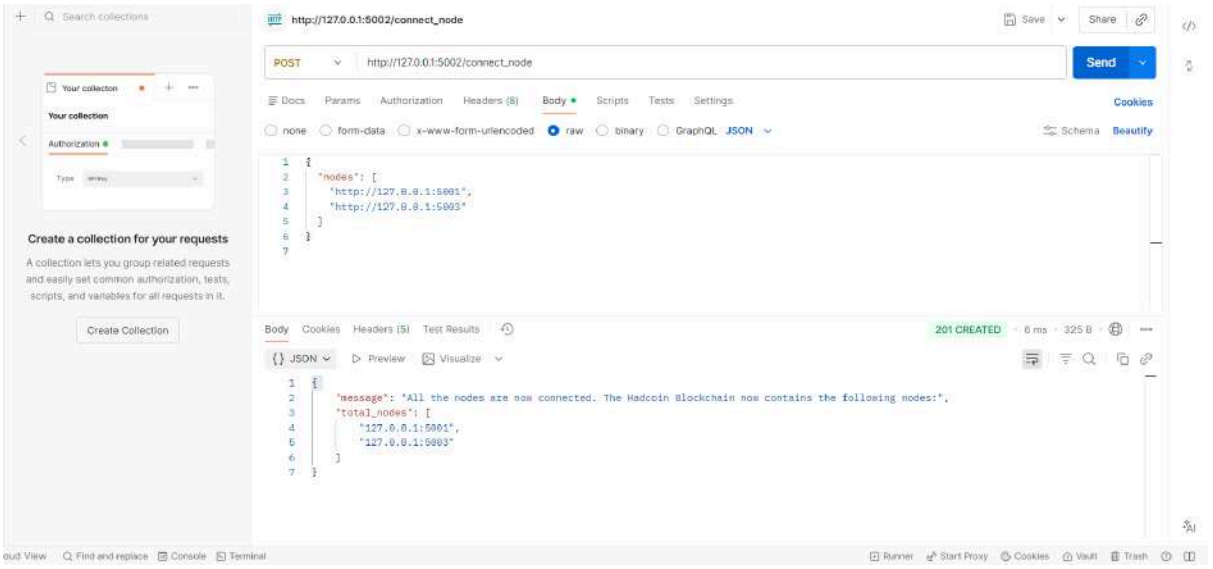
return jsonify(response), 200

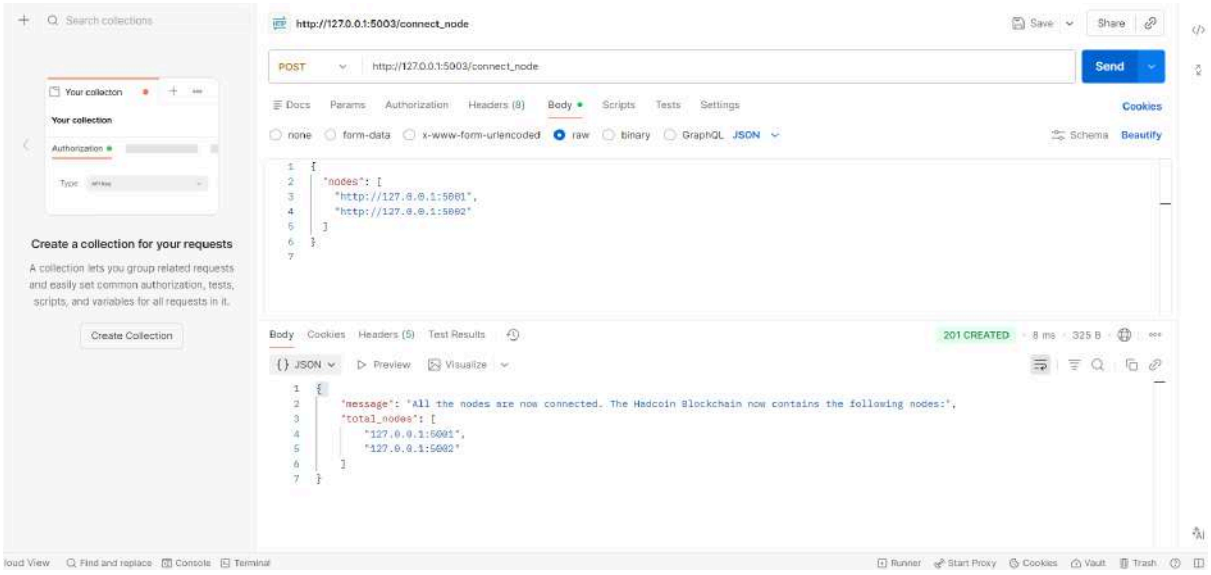
Running the app

app.run(host = '0.0.0.0', port = 5001)

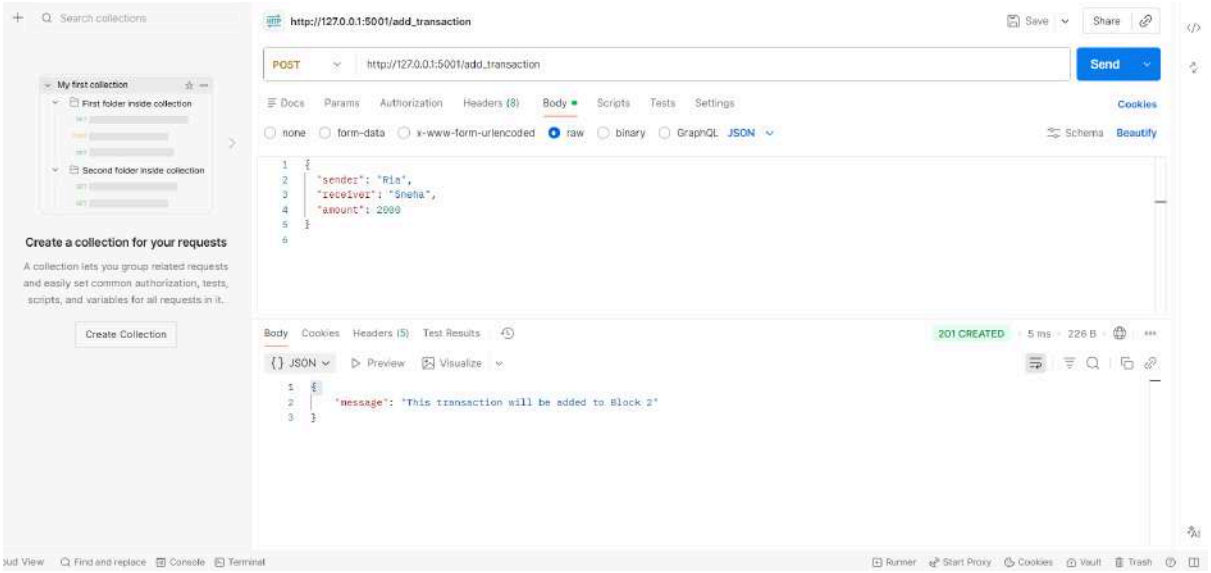
Step 1: Connect nodes

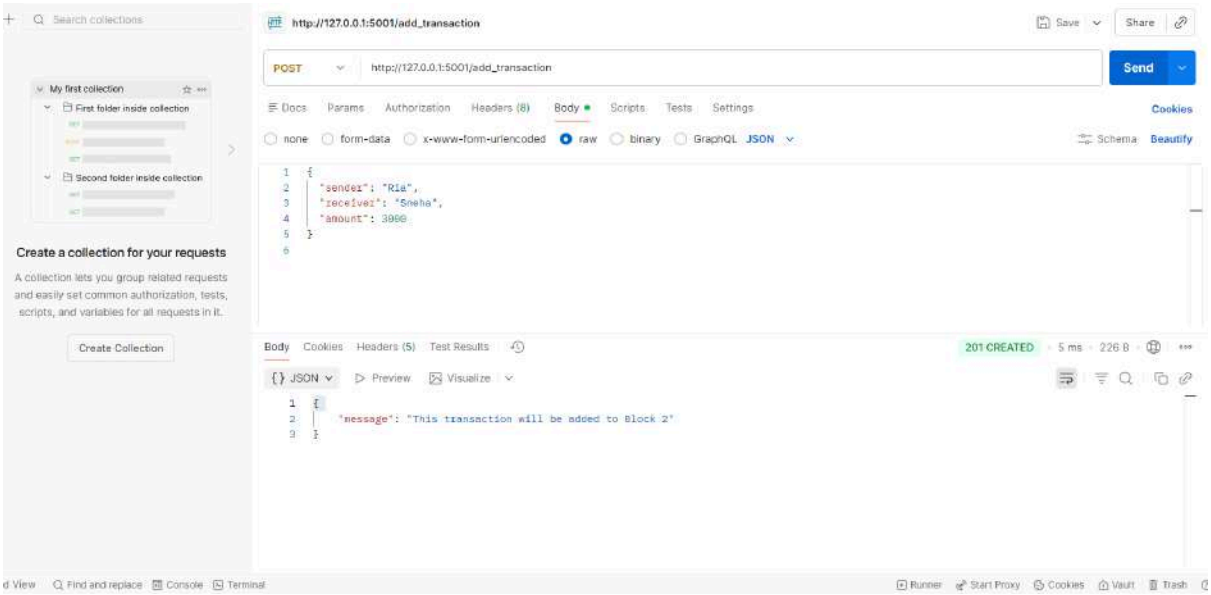




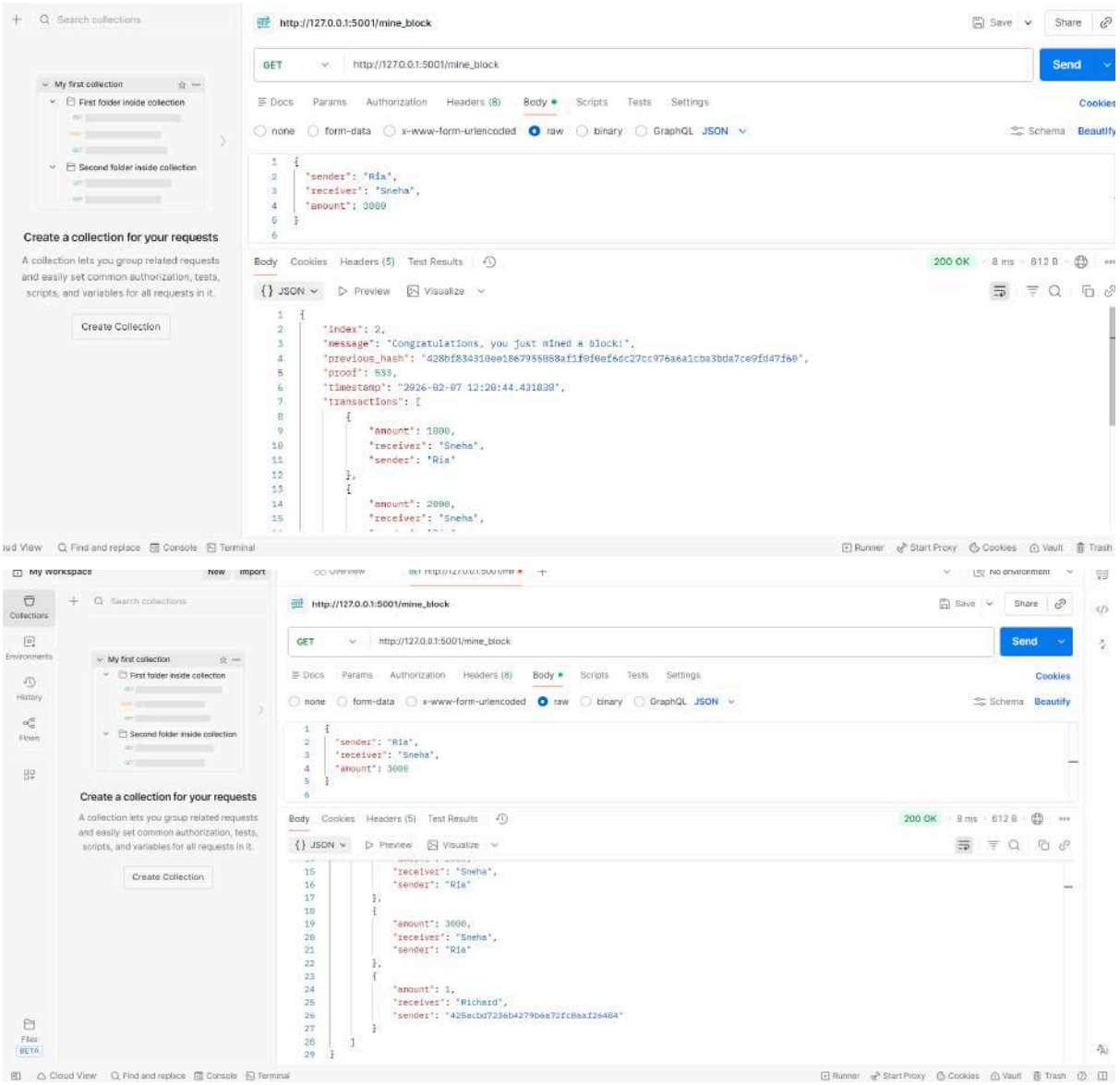


Step 2: Add transaction

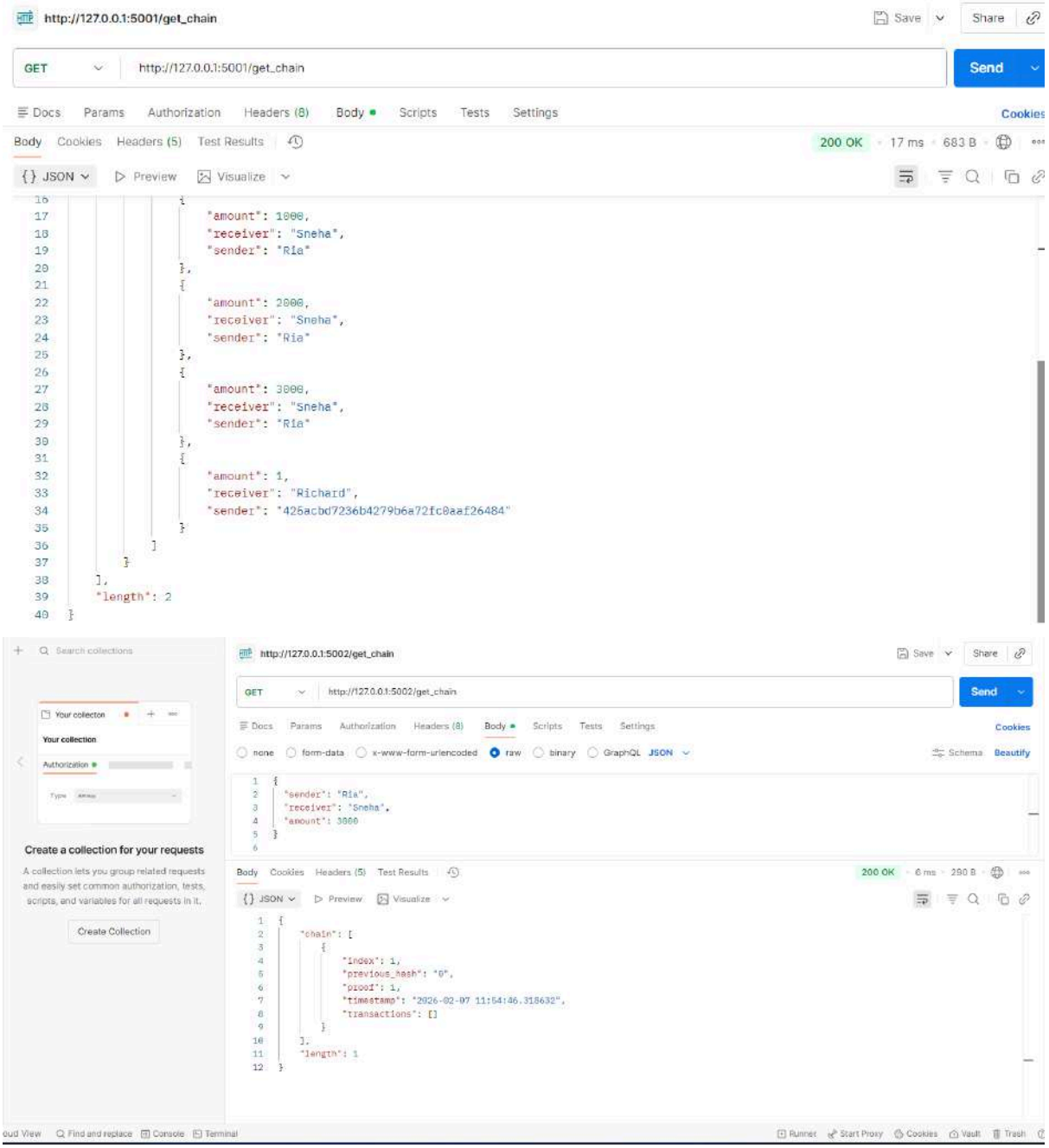


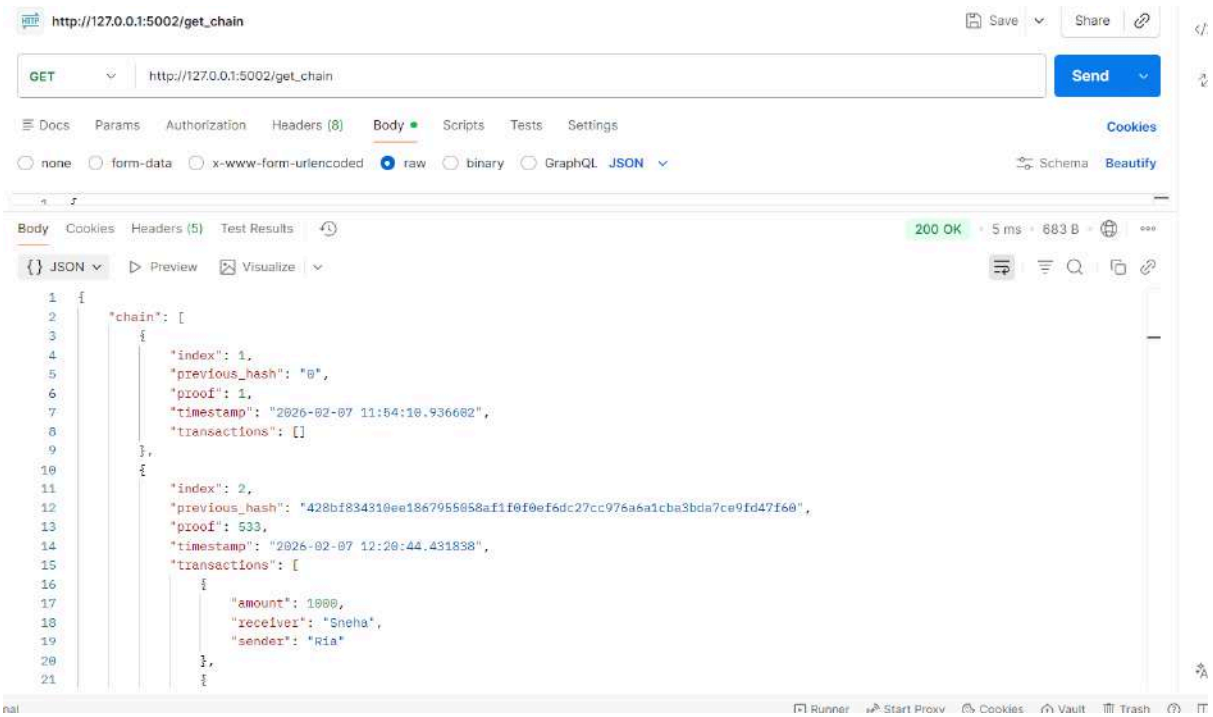


Step 3: Mine blocks

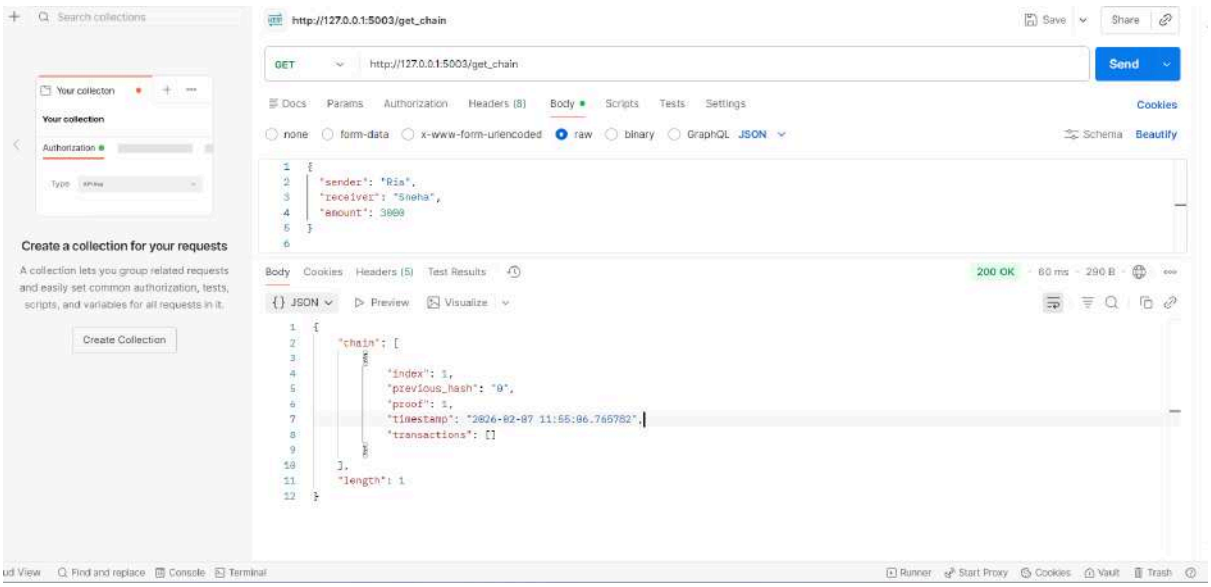


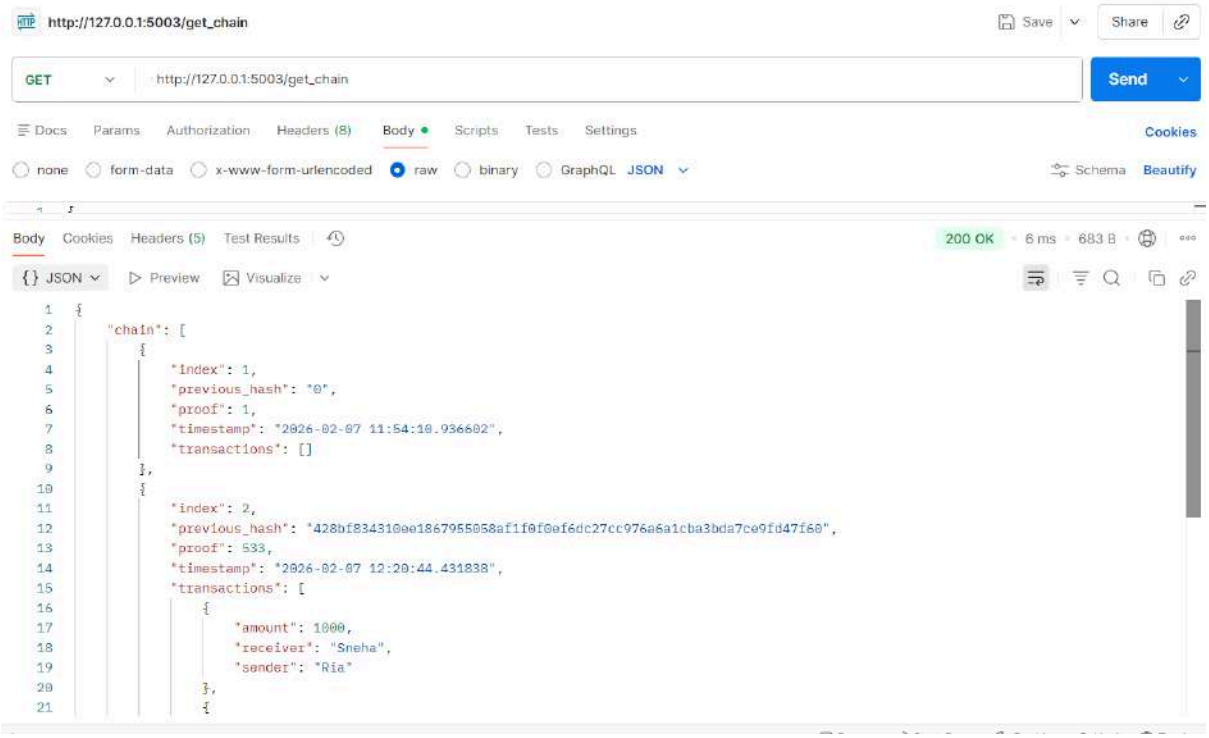
Step 4: Check chain before consensus





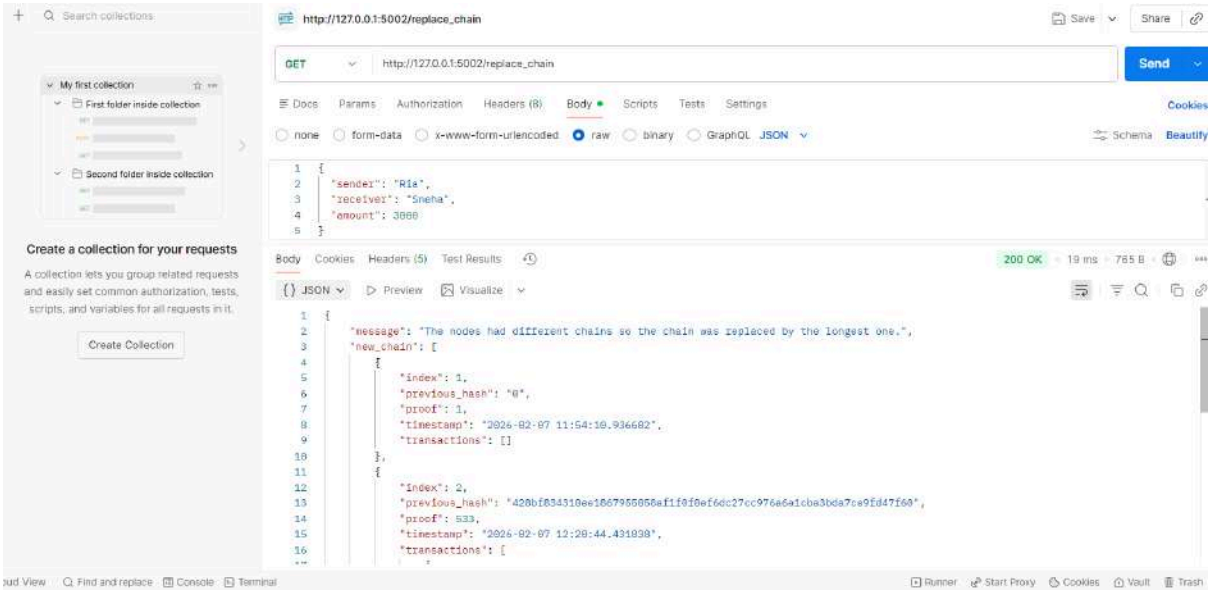

```
{
    "amount": 2000,
    "receiver": "Sneha",
    "sender": "Ria"
},
{
    "amount": 3000,
    "receiver": "Sneha",
    "sender": "Ria"
},
{
    "amount": 1,
    "receiver": "Richard",
    "sender": "425acbd7236b4279b6a72fc0aaf26484"
}
]
}
],
"length": 2
}
```


Search collections
Your collection
Authorization
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://127.0.0.1:5003/get_chain
Save
Share
GET
http://127.0.0.1:5003/get_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"sender": "Ria",
"receiver": "Sneha",
"amount": 3000
}
Body
Cookies
Headers (5)
Test Results
200 OK
60 ms
290 B
JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-07 11:55:06.765782",
"transactions": []
}
],
"length": 1
}
Find and replace
Console
Terminal
Runner
Start Proxy
Cookies
Vault
Trash

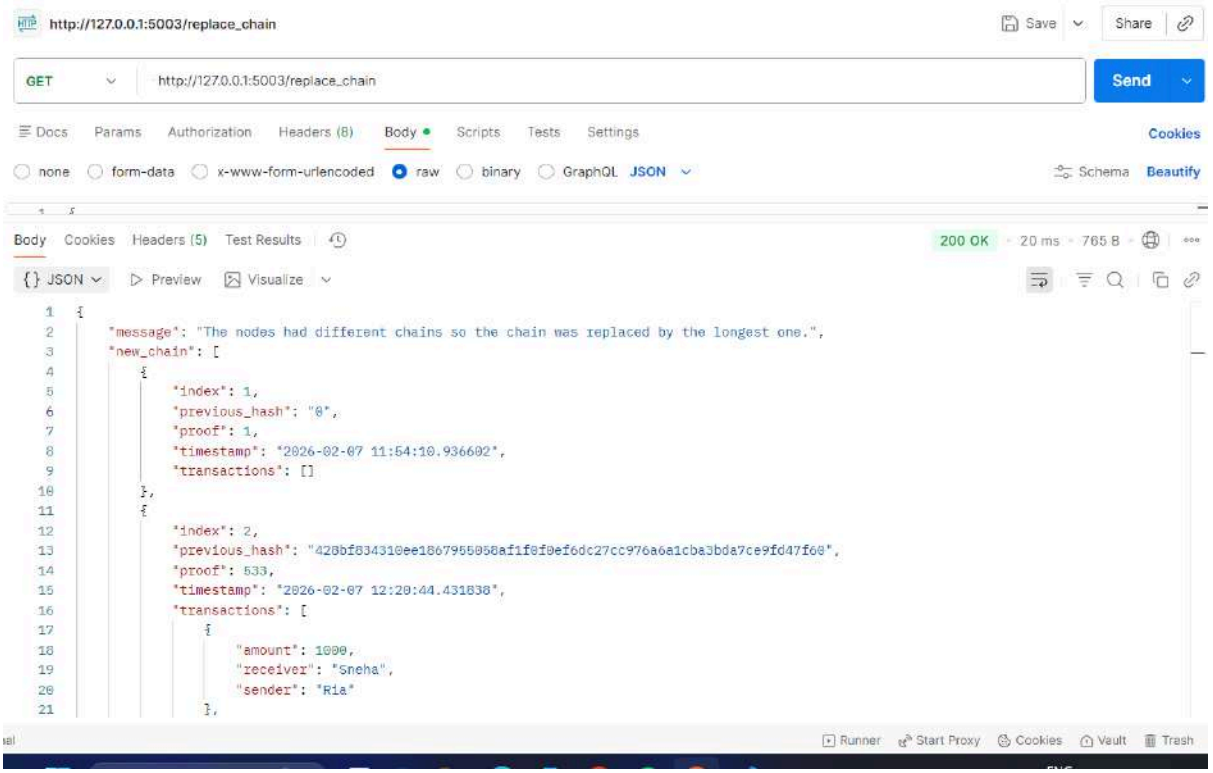

http://127.0.0.1:5003/get_chain
Save
Share
GET
http://127.0.0.1:5003/get_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
6 ms
683 B
JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-07 11:54:10.936602",
"transactions": []
},
{
"index": 2,
"previous_hash": "428bf834310ee1867955058af1f0f0ef6dc27cc976a6a1cba3bda7ce9fd47f60",
"proof": 533,
"timestamp": "2026-02-07 12:20:44.431838",
"transactions": [
{
"amount": 1000,
"receiver": "Sneha",
"sender": "Ria"
},
{

```
            {
                "amount": 2000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 3000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 1,
                "receiver": "Richard",
                "sender": "425acbd7236b4279b6a72fc0aaf26484"
            }
        ]
    }
],
"length": 2
}
```


Step 5:Apply consensus



```
        "transactions": [
            {
                "amount": 1000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 2000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 3000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 1,
                "receiver": "Richard",
                "sender": "425acbd7236b4279b6a72fc0aaf26484"
            }
        ]
    }
]
}
```

```
            {
                "amount": 2000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 3000,
                "receiver": "Sneha",
                "sender": "Ria"
            },
            {
                "amount": 1,
                "receiver": "Richard",
                "sender": "425acbd7236b4279b6a72fc0aaf26484"
            }
        ]
    }
]
}
```


CONCLUSION:

In this experiment, a cryptocurrency blockchain was successfully created using Python and Flask, where multiple nodes performed transactions and mining independently. Different blockchain lengths were generated across nodes and the consensus mechanism based on the longest chain rule was applied using the replace_chain function. After consensus, all nodes synchronized to the longest valid chain, demonstrating how decentralized networks maintain consistency, security, and agreement without a central authority

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Ria Chaudhari D20A14

Blockchain Experiment 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts

THEORY:

Q1: Primitive Data Types, Variables, Functions - pure, view

In Solidity, **primitive data types** form the foundation of smart contract development. Commonly used types include:

● uint / int: unsigned and signed integers of different sizes (e.g., uint256, int128).

● bool: represents logical values (true or false).

● address: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.

● bytes / string: store binary data or textual data.

Variables in Solidity can be

● state variables: stored on the blockchain permanently

● local variables: temporary, created during function execution

● global variables: special predefined variables such as msg.sender, msg.value, and block.timestamp

Functions allow execution of contract logic. Special types of functions include:

● pure: cannot read or modify blockchain state; they work only with inputs and internal computations.

● view: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

Q2: Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to

return results after computation.

For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

Q3: Visibility, Modifiers and Constructors

Function Visibility defines who can access a function:

● public: available both inside and outside the contract.

● private: only accessible within the same contract.

● internal: accessible within the contract and its child contracts.

● external: can be called only by external accounts or other contracts.

Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

Q4: Control Flow : if-else, loops

Control flow in Solidity is similar to traditional programming languages:

● if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.

● Loops (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

Q5: Data Structures : Arrays, Mappings, structs, enums

● Arrays: Can be fixed or dynamic and are used to store ordered lists of elements.

Example: an array of addresses for registered users.

● Mappings: Key-value pairs that allow quick lookups.

Example: mapping(address => uint) for storing balances.

Unlike arrays, mappings do not support iteration.

● Structs: Allow grouping of related properties into a single data type.

Example: struct Player {string name; uint score;}.

● Enums: Used to define a set of predefined constants, making code more readable.

Example: enum Status { Pending, Active, Closed }.

Q6: Data Locations

Solidity uses three primary data locations for storing variables:

● storage: Data stored permanently on the blockchain. Examples: state variables.

● memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.

● calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory.

Understanding data locations is essential, as they directly impact gas costs and performance.

Q7: Transactions : Ether and wei, Gas and Gas Price, Sending Transactions

● Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei,the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.

● Gas and Gas Price: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.

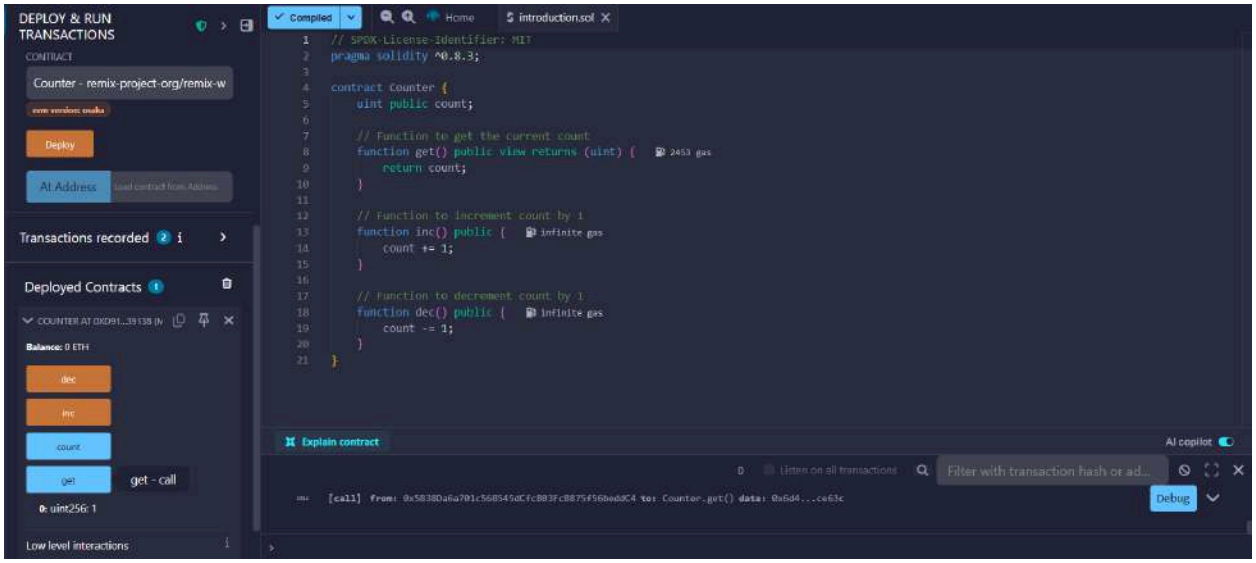
● Sending Transactions: Transactions are used for transferring Ether or interacting with

contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

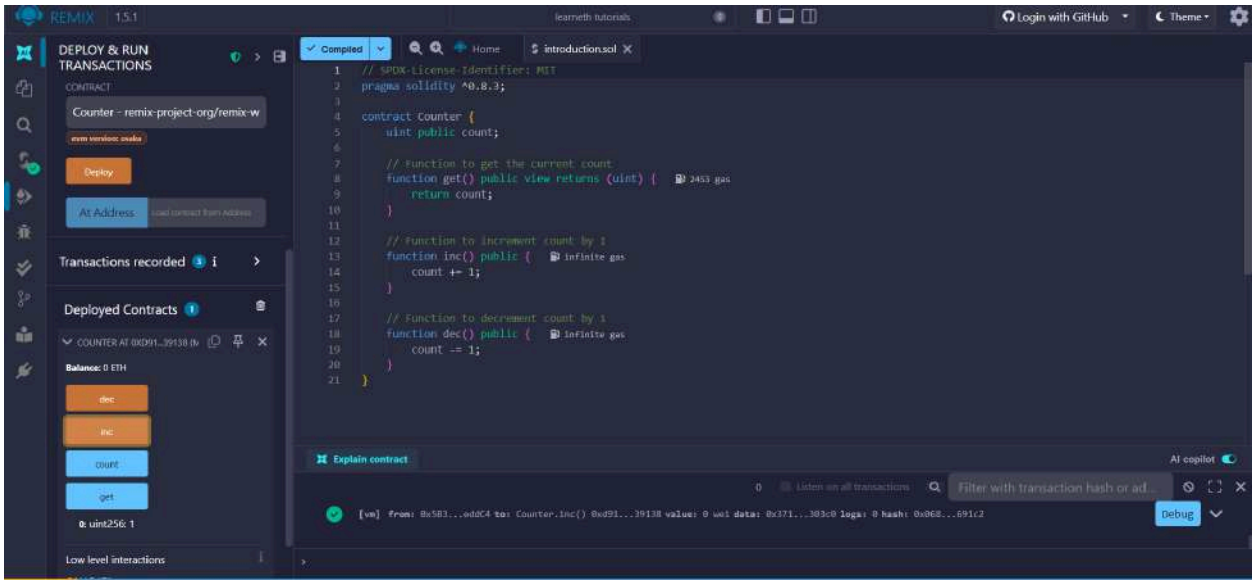
TASKS PERFORMED:

Tutorial 1: Introduction

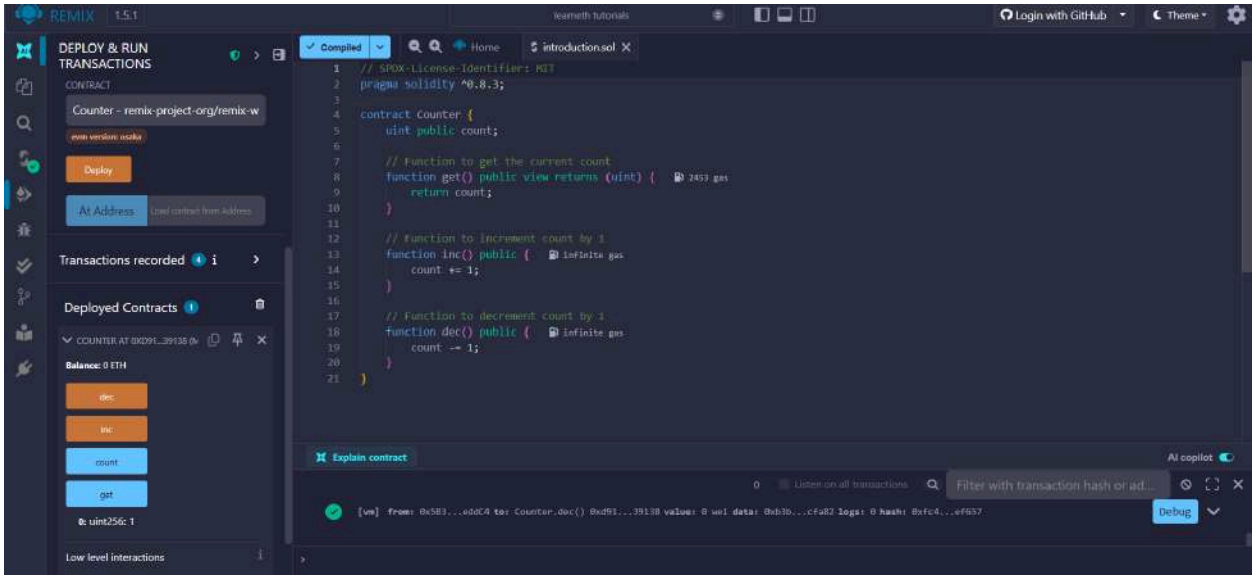
a)get



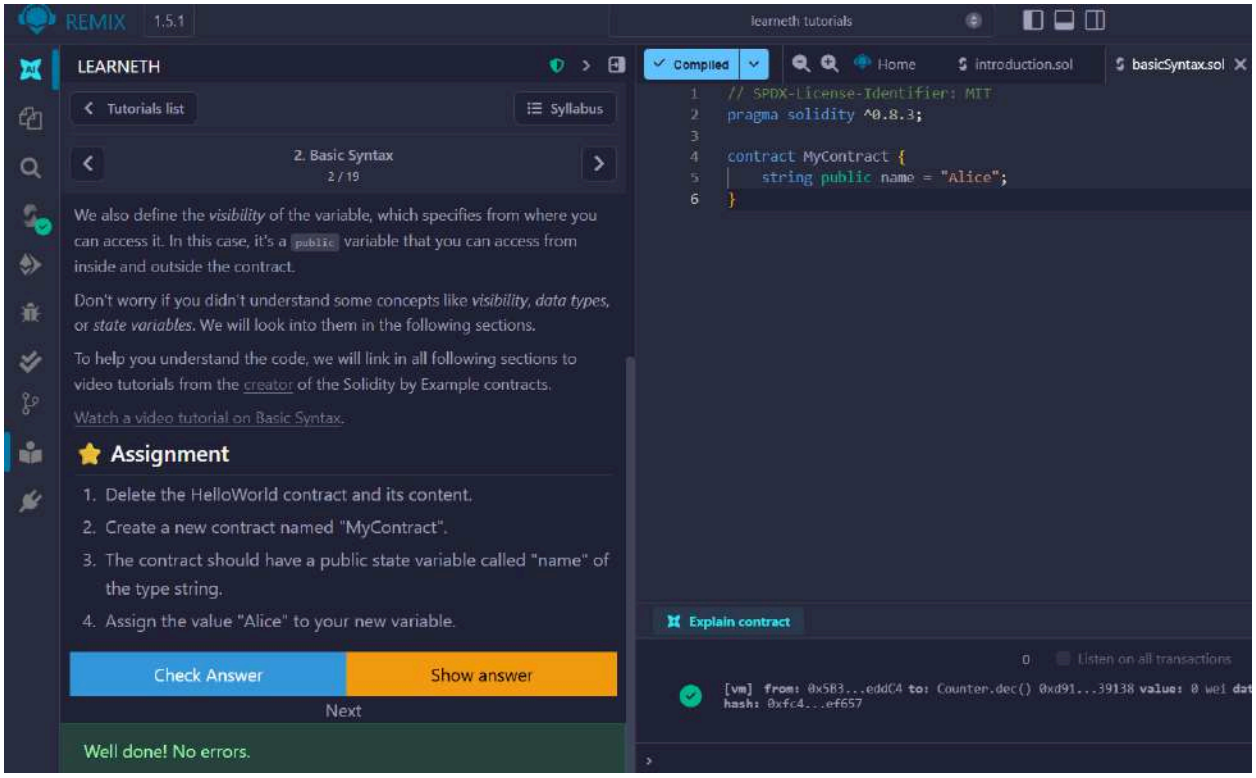
b)inc



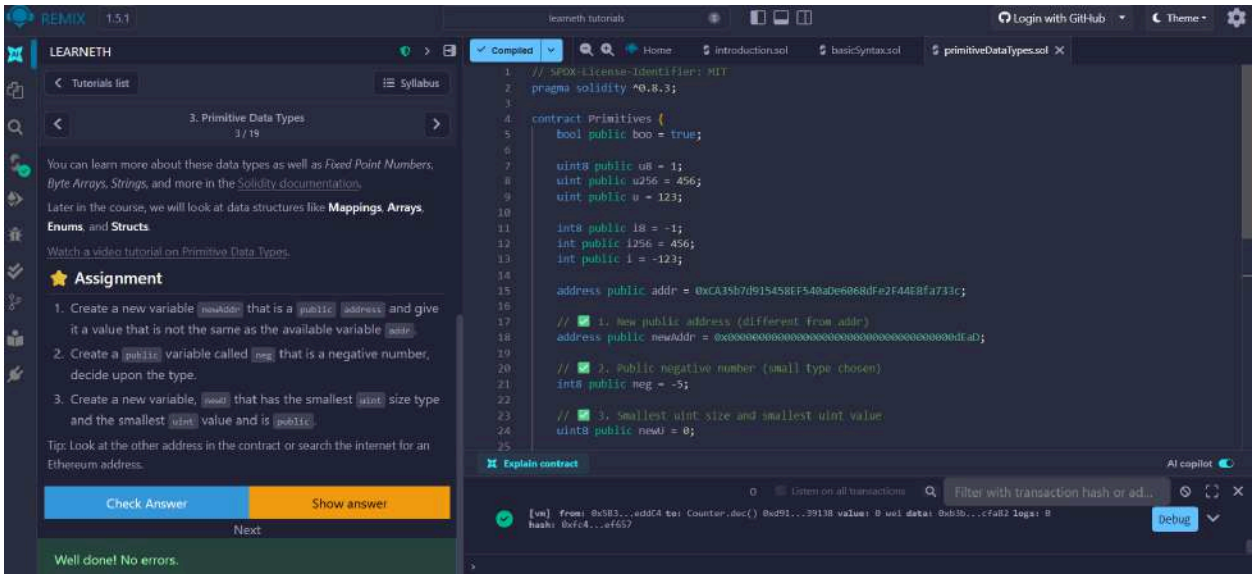
c)dec



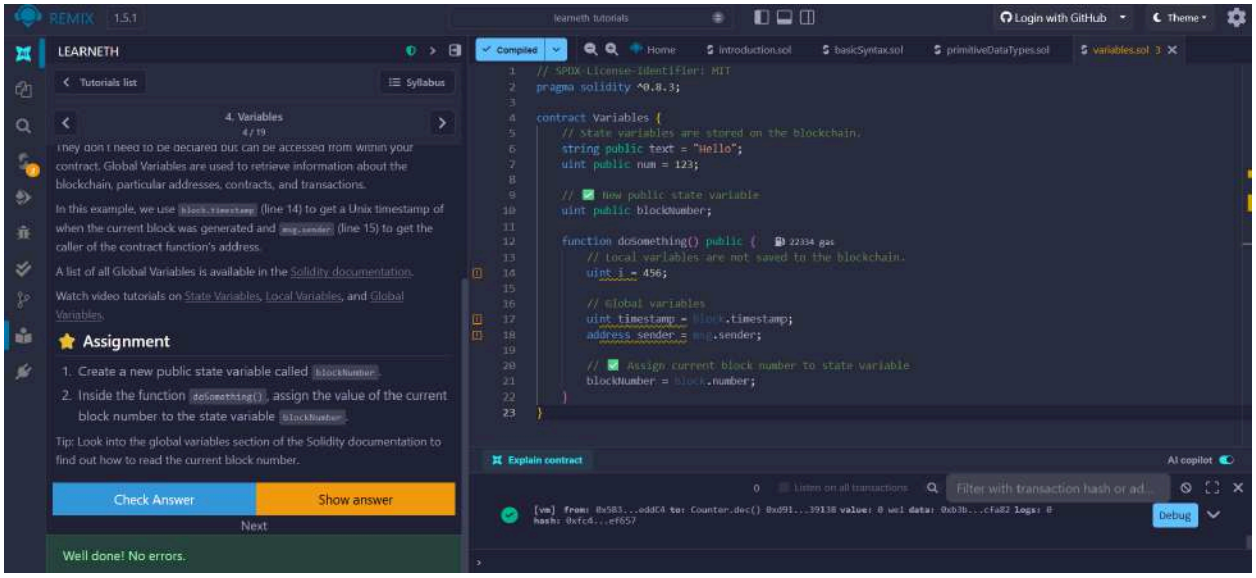
Tutorial 2:Basic Syntax



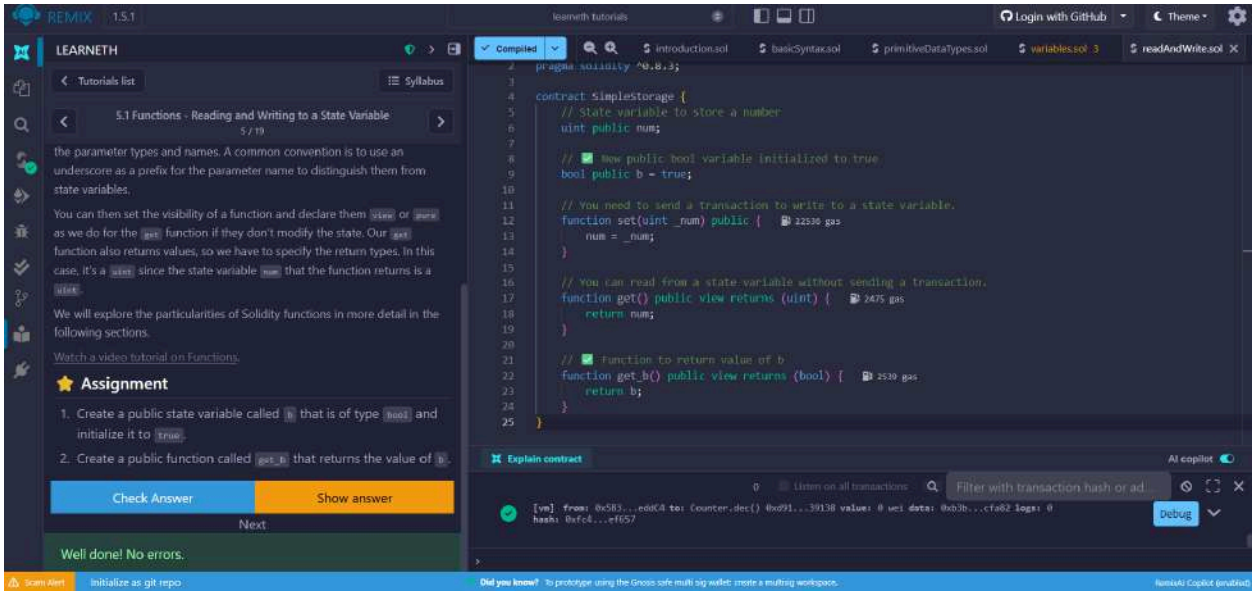
Tutorial 3:Primitive Data Types



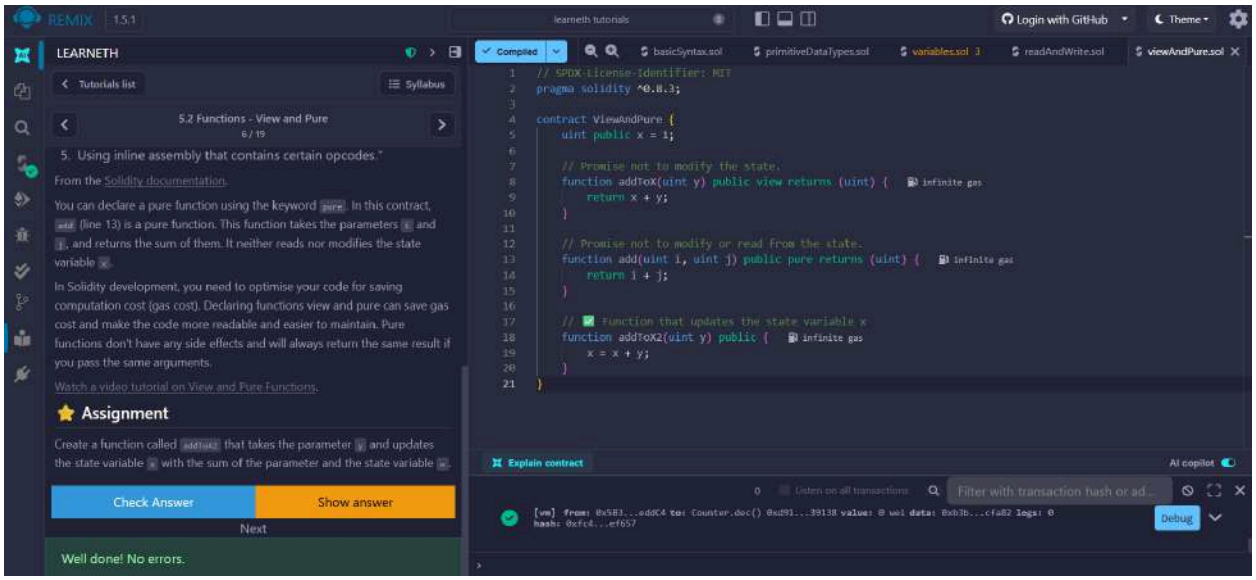
Tutorial 4:Variables



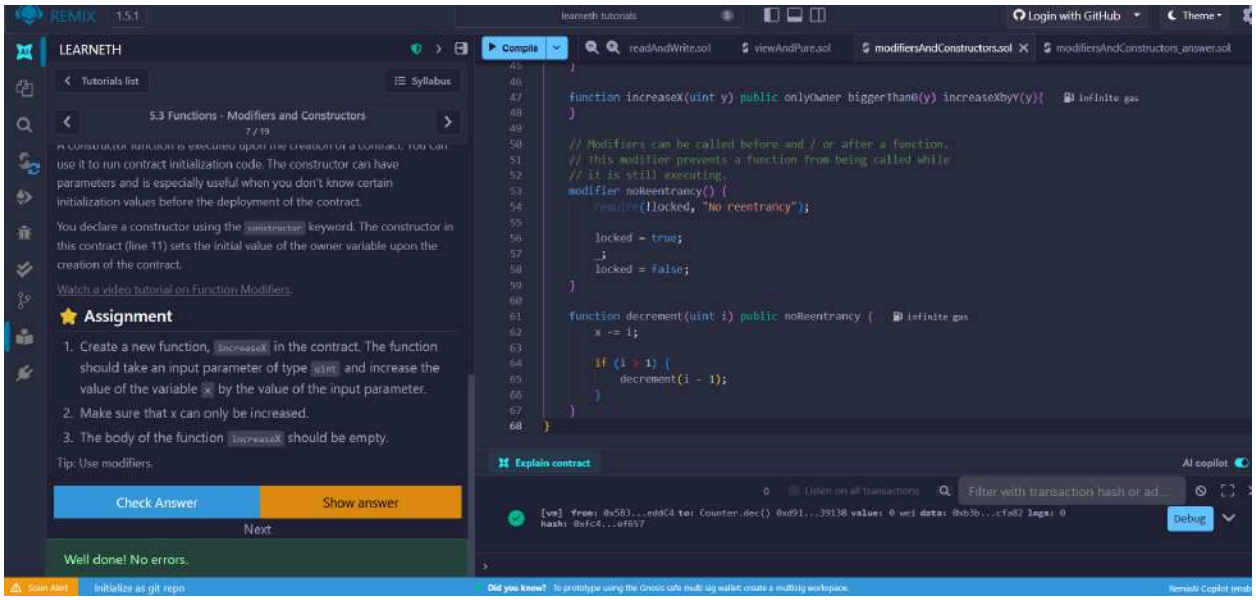
Tutorial 5: Functions - Reading and Writing to a State Variable



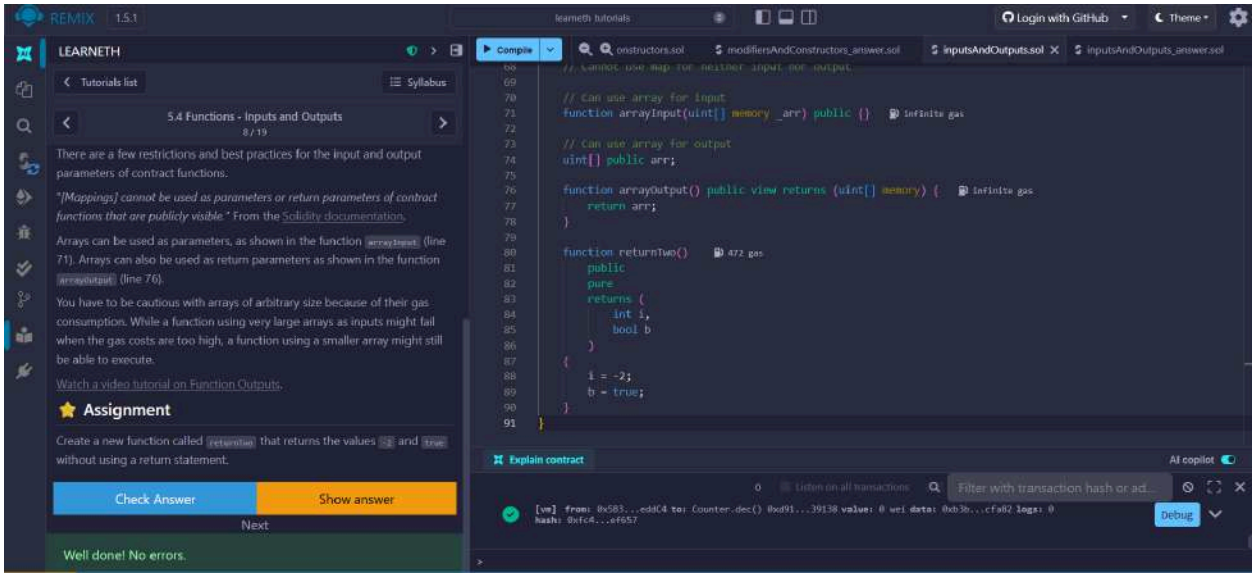
Tutorial 6: Functions - View and Pure



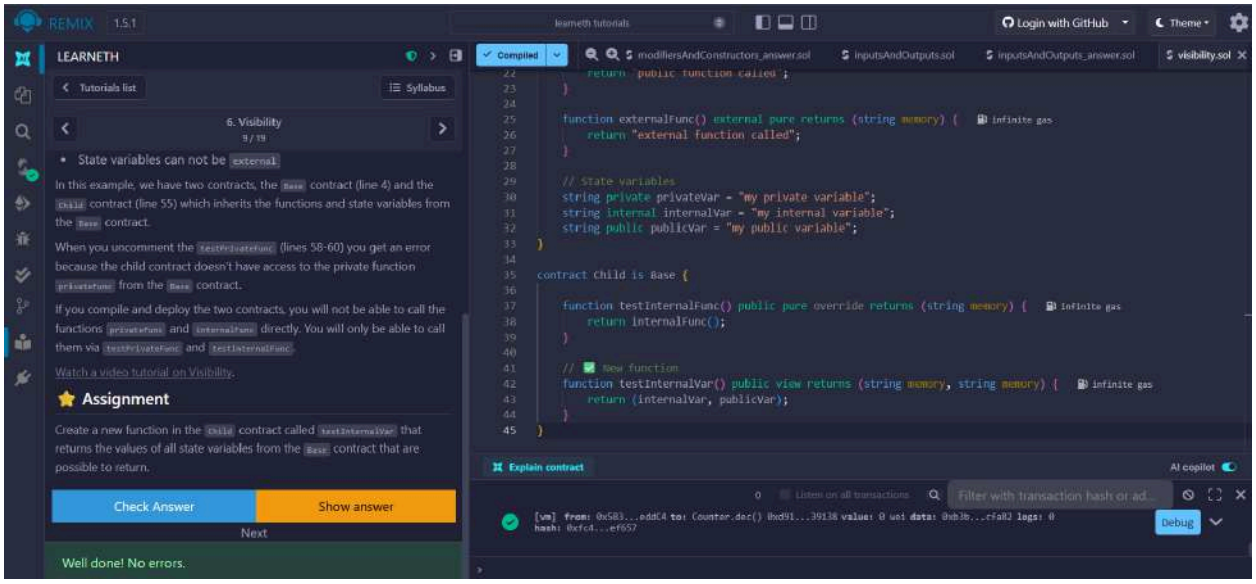
Tutorial 7:Modifiers and Constructors



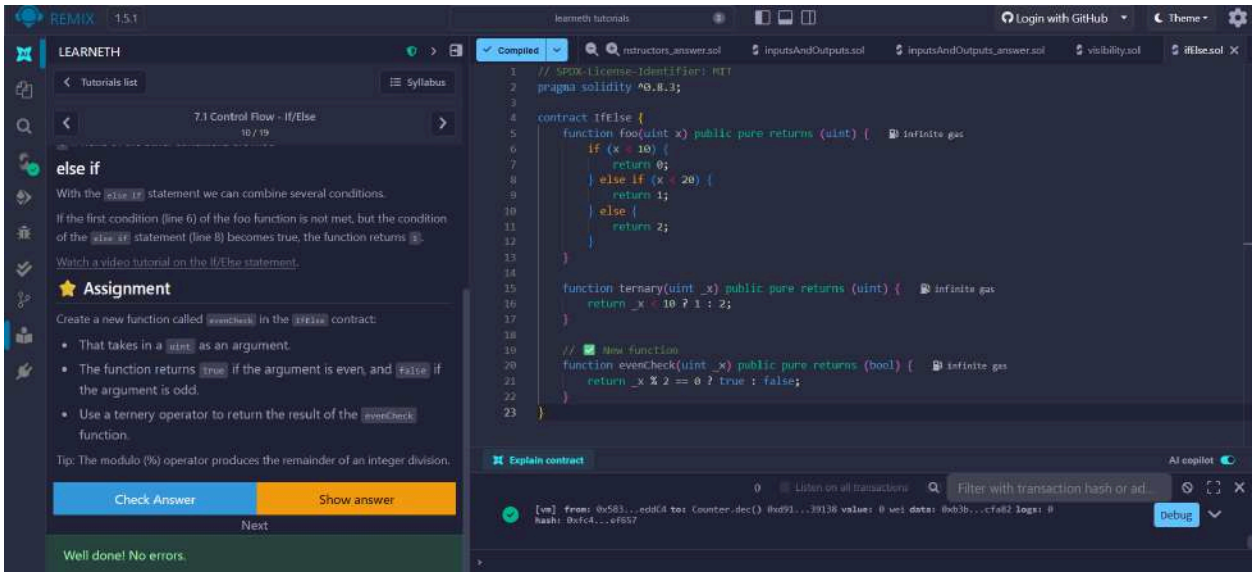
Tutorial 8:Inputs and Outputs



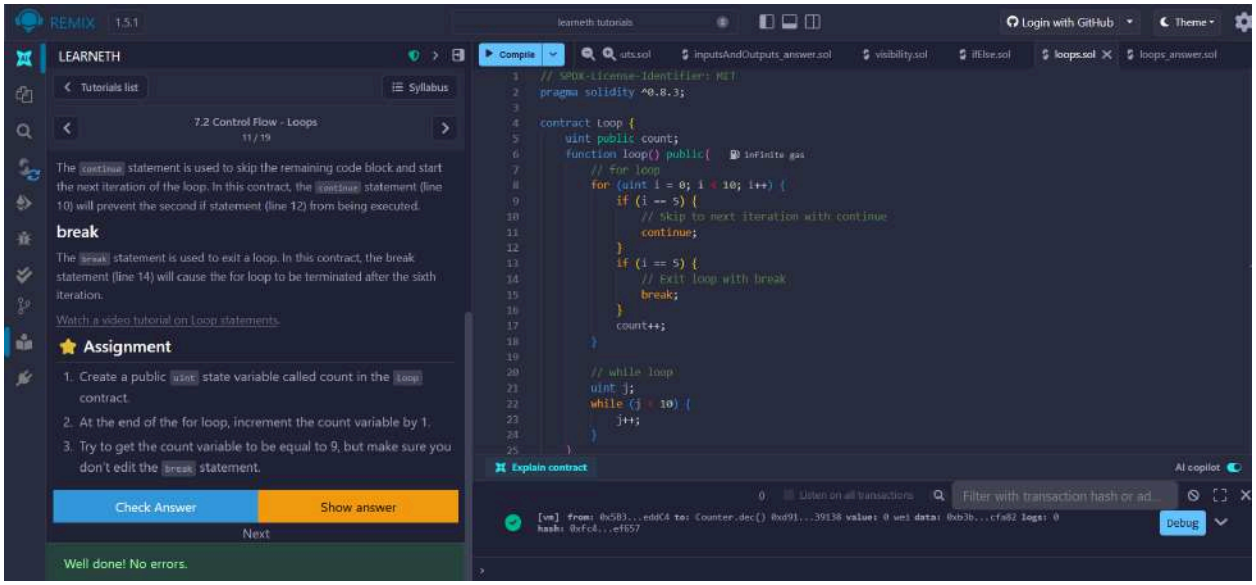
Tutorial 9:Visibility



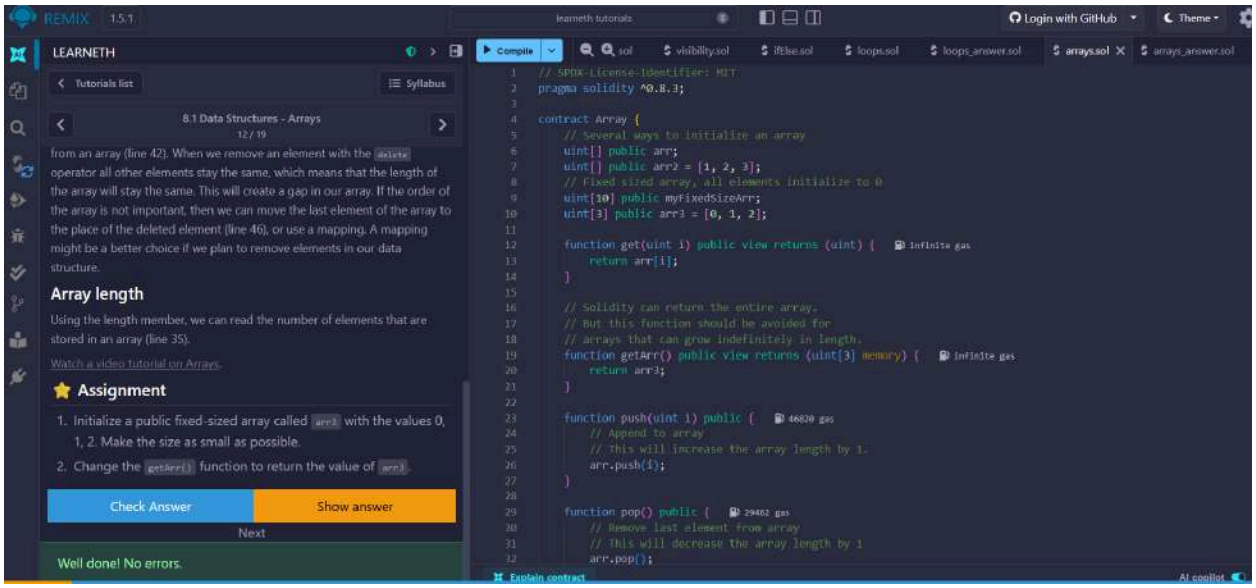
Tutorial 10: Control Flow-If/Else



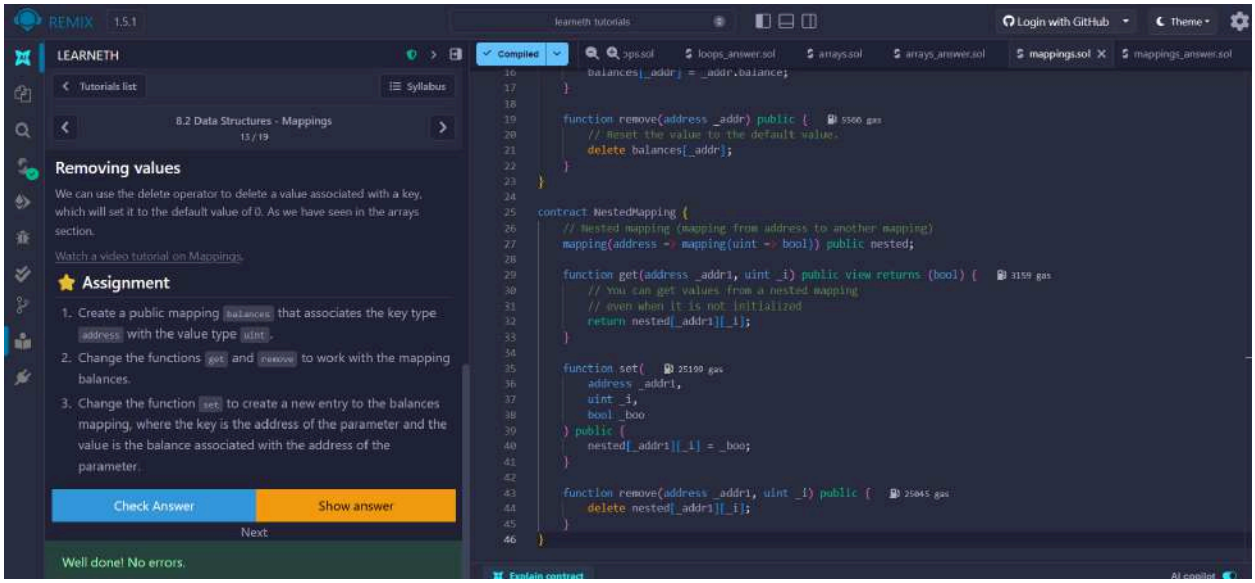
Tutorial 11:Loops



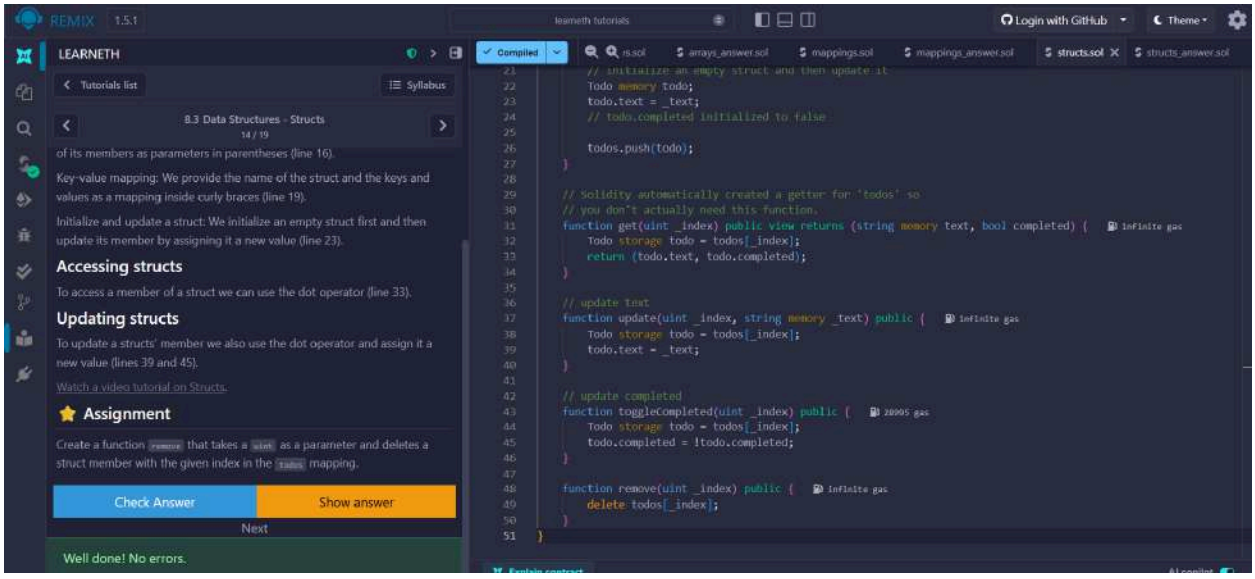
Tutorial 12:Data Structures:Arrays



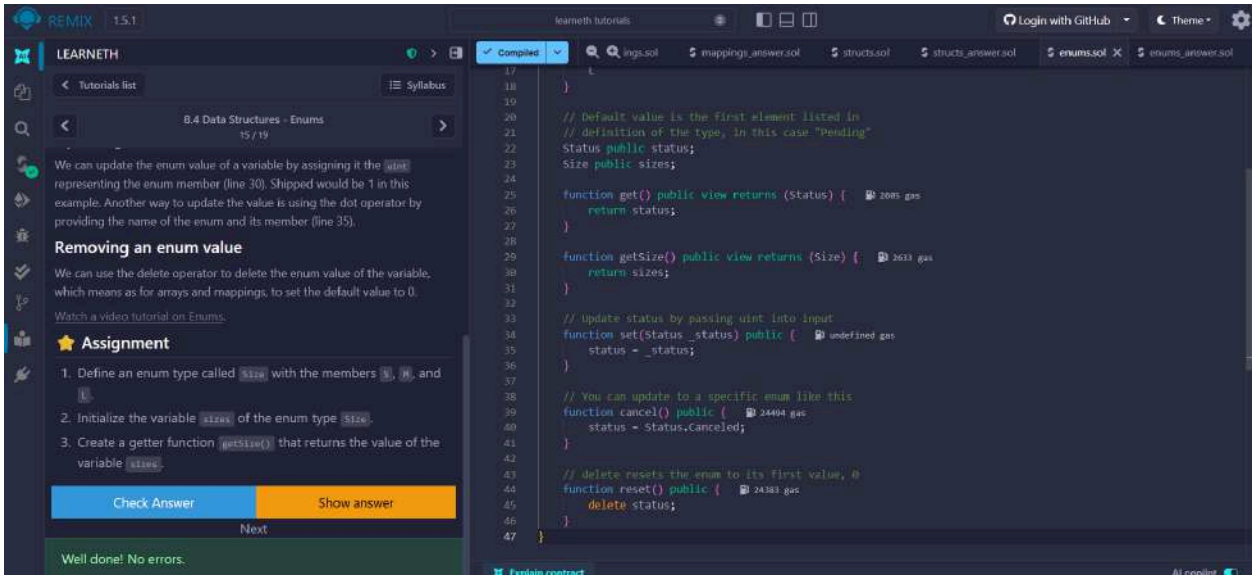
Tutorial 13:Data Structures:Mappings



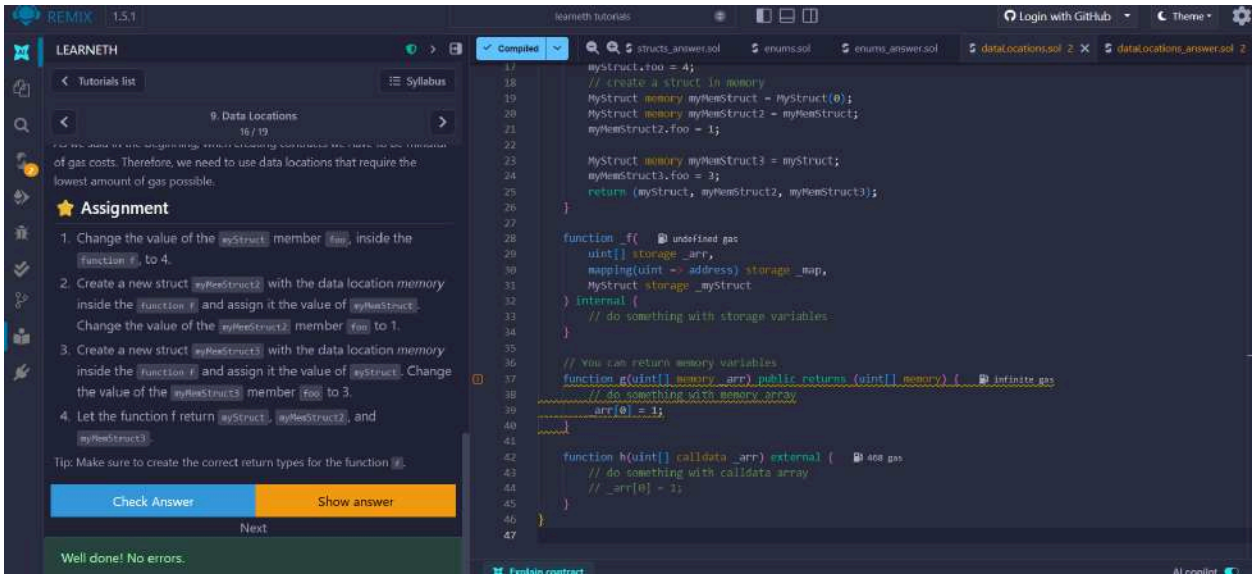
Tutorial 14:Data Structures:Structs



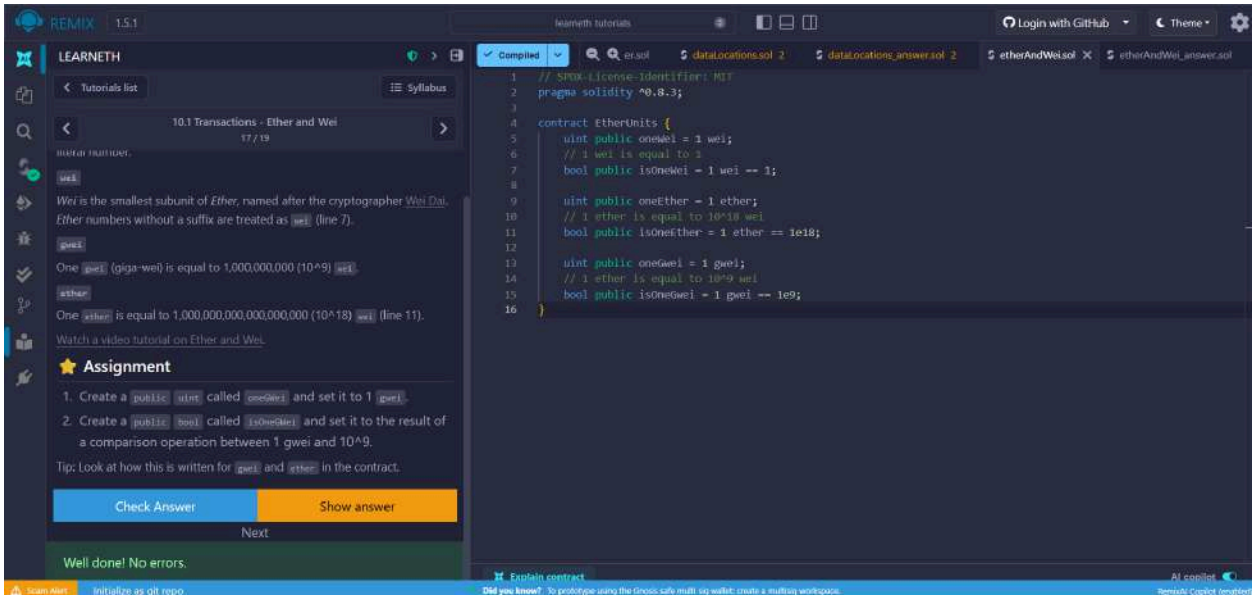
Tutorial 15: Data Structures:Enums



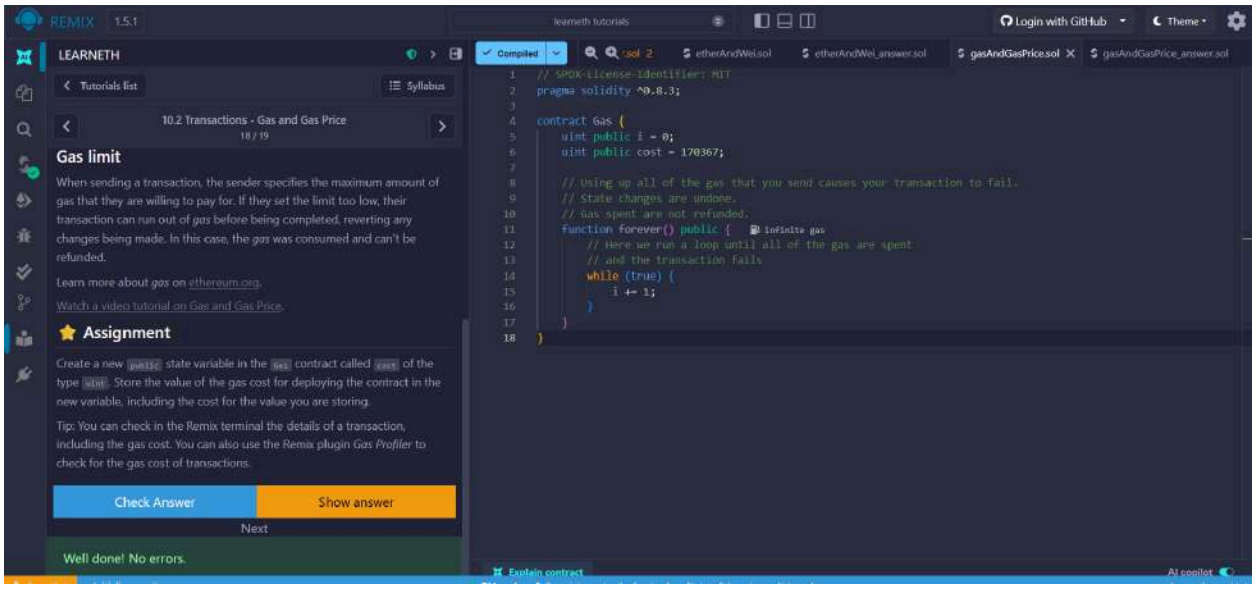
Tutorial 16:Data Locations



Tutorial 17:Transactions-Ether and Wei



Tutorial 18:Transactions-Gas and Gas Price



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Ria Chaudhari D20A14

Blockchain Exp 5

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

Q1: What is the relevance of required statements in the functions of Solidity Programs?

require is used to validate conditions before executing a function.

If the condition is false:

- The transaction reverts
- All state changes are undone
- Remaining gas is refunded
- An error message is displayed

It is mainly used for:

- Input validation
- Access control
- Security checks

Example:

require(!voters[msg.sender].voted, "Already voted.");

This prevents a user from voting twice.

Q2: Understand the keywords mapping, storage and memory

mapping:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType).

For example: mapping(address => Voter) public voters;

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

storage:

In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains

available throughout the contract's lifecycle.

memory:

In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

Q3: Why bytes32 instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in efficiency and gas optimization.

bytes32 is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.

string is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
```

```
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted;  // if true, that person already voted
        address delegate; // person delegated to
        uint vote;   // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal {
        // If you can limit the length to a certain number of bytes,
        // always use one of bytes1 to bytes32 because they are much
cheaper
        string name;
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        // For each of the provided proposal names,
        // create a new proposal object and add it
        // to the end of the array.
        for (uint i = 0; i < proposalNames.length; i++) {
            // 'Proposal({...})' creates a temporary
            // Proposal object and 'proposals.push(...)'
            // appends it to the end of 'proposals'.
```

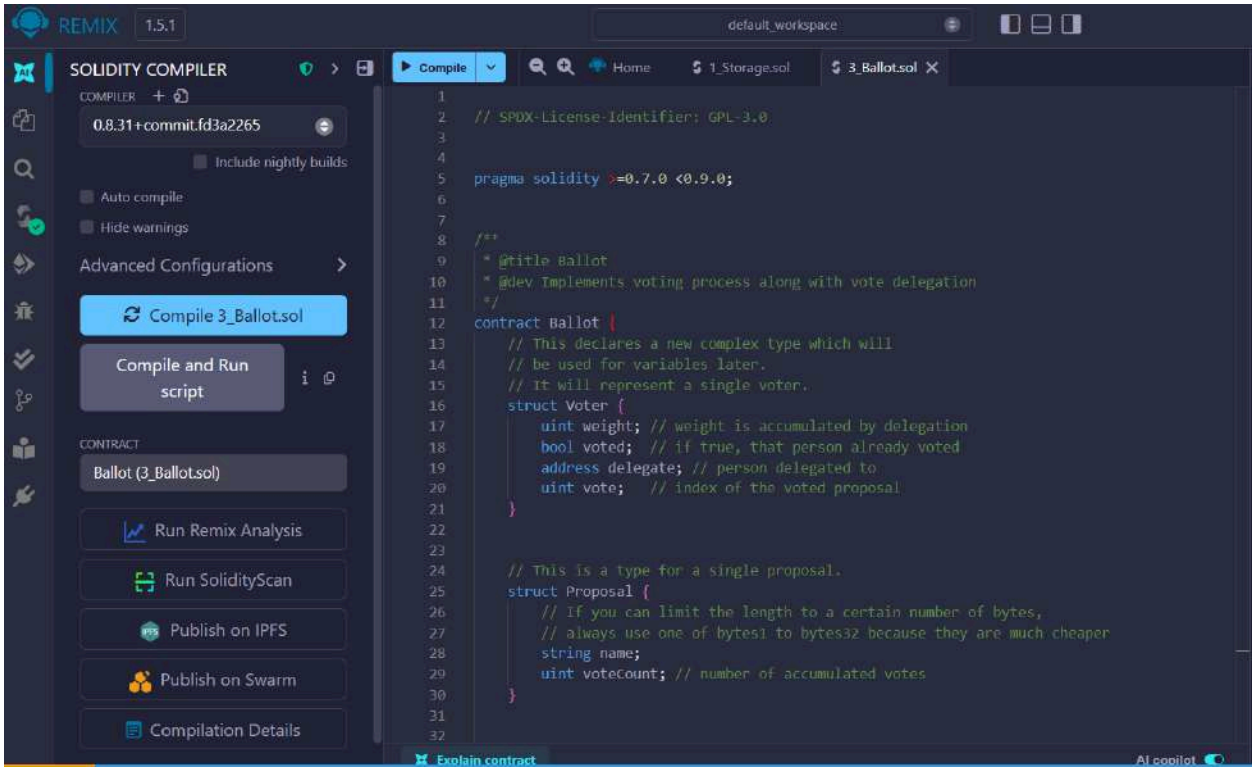
```
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }

     /**
     * @dev Give 'voter' the right to vote on this ballot. May only be
called by 'chairperson'.
     * @param voter address of voter
     */
    function giveRightToVote(address voter) external {
        // If the first argument of `require` evaluates
        // to 'false', execution terminates and all
        // changes to the state and to Ether balances
        // are reverted.
        // This used to consume all gas in old EVM versions, but
        // not anymore.
        // It is often a good idea to use 'require' to check if
        // functions are called correctly.
        // As a second argument, you can also provide an
        // explanation about what went wrong.
        require(
            msg.sender == chairperson,
            "Only chairperson can give right to vote."
        );
        require(
            !voters[voter].voted,
            "The voter already voted."
        );
        require(voters[voter].weight == 0, "Voter already has the right to
vote.");
        voters[voter].weight = 1;
    }

    /**
     * @dev Delegate your vote to the voter 'to'.
     * @param to address to which vote is delegated
     */
```

```
function delegate(address to) external {
    // assigns reference
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");

    require(to != msg.sender, "Self-delegation is disallowed.");

    // Forward the delegation as long as
    // 'to' also delegated.
    // In general, such loops are very dangerous,
    // because if they run too long, they might
    // need more gas than is available in a block.
    // In this case, the delegation will not be executed,
    // but in other situations, such loops might
    // cause a contract to get "stuck" completely.
    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;

        // We found a loop in the delegation, not allowed.
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];

    // Voters cannot delegate to accounts that cannot vote.
    require(delegate_.weight >= 1);

    // Since 'sender' is a reference, this
    // modifies 'voters[msg.sender]'.
    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        // If the delegate already voted,
        // directly add to the number of votes
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        // If the delegate did not vote yet,
        // add to her weight.
```

```
            delegate_.weight += sender.weight;
        }
    }

    /**
     * @dev Give your vote (including votes delegated to you) to proposal
'proposals[proposal].name'.
     * @param proposal index of proposal in the proposals array
     */
    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;

        // If 'proposal' is out of the range of the array,
        // this will throw automatically and revert all
        // changes.
        proposals[proposal].voteCount += sender.weight;
    }

    /**
     * @dev Computes the winning proposal taking all previous votes into
account.
     * @return winningProposal_ index of winning proposal in the proposals
array
     */
    function winningProposal() public view
            returns (uint winningProposal_)
    {
        uint winningVoteCount = 0;
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }
```

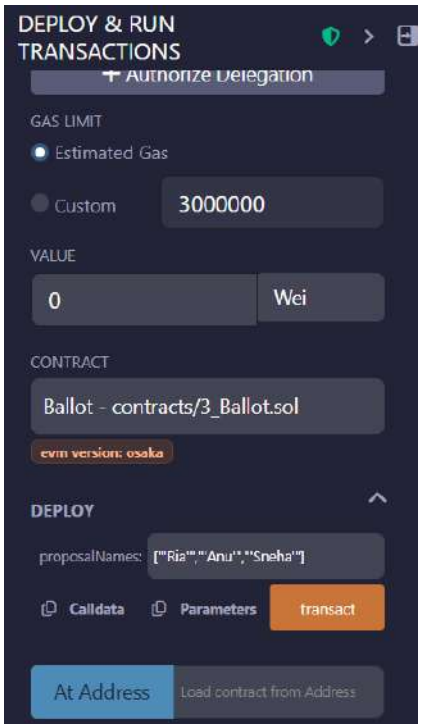
```
    /**
     * @dev Calls winningProposal() function to get the index of the
winner contained in the proposals array and then
     * @return winnerName_ the name of the winner
     */
    function winnerName() external view
            returns (string memory winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }
}
```

Compiled ballot.sql



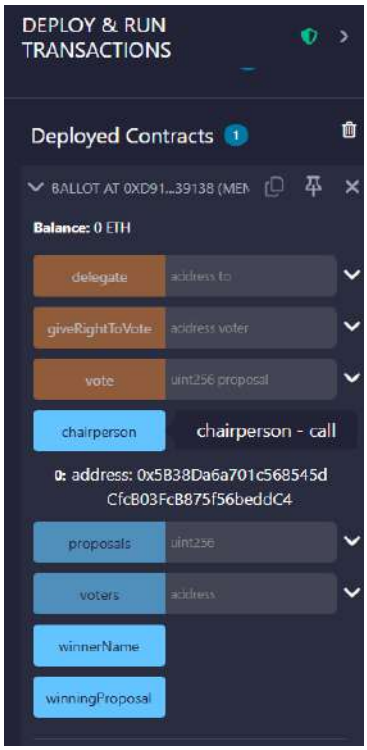
Step 1: Deploy Contract

3 proposals are created “Ria”, “Anu” and “Sneha” all with vote count 0.



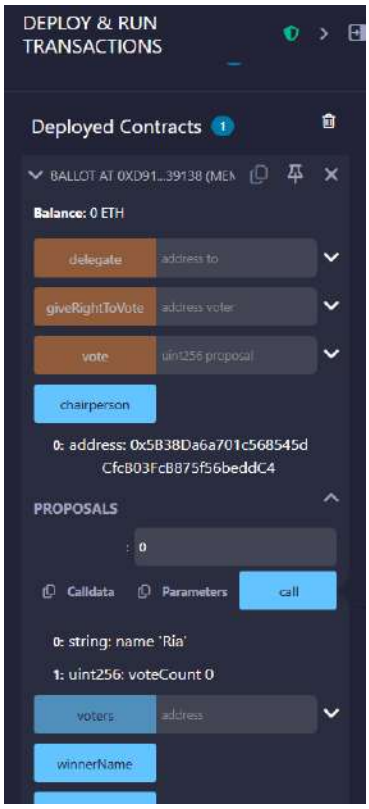
Step 2: Check Chairperson

Here, Account 1 - “Ria” is the chairperson.

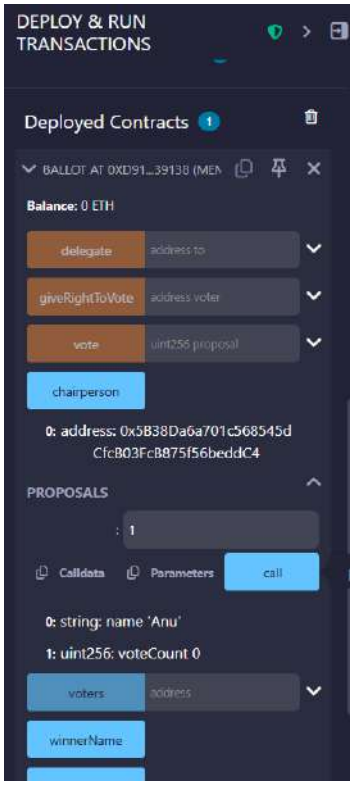


Step 3: Check proposals

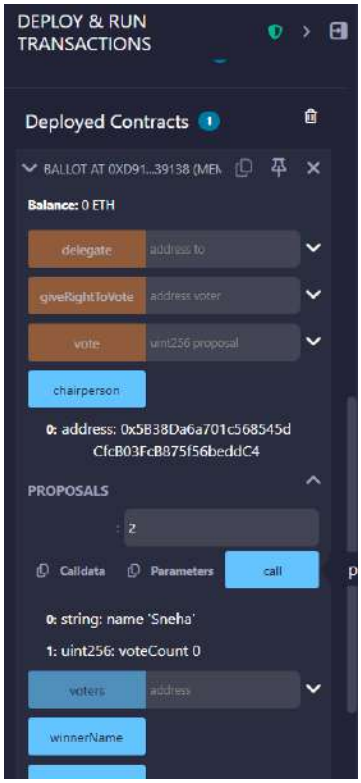
For Account 1- “Ria”



For account 2:’Anu’

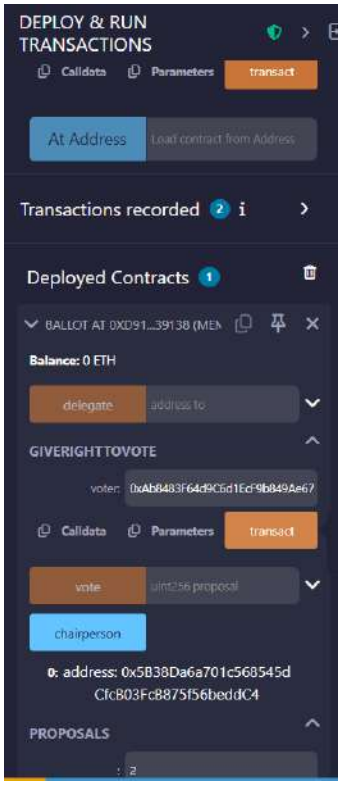


For Account 3:'Sneha'



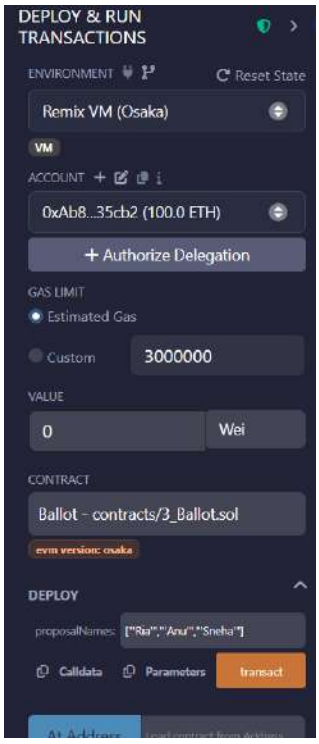
Step 4: Give voting right to another account

Keep account 1 - Ria selected and in **giveRightToVote(address voter)** paste the address of Account 2 - “Anu” and click on the transact button.

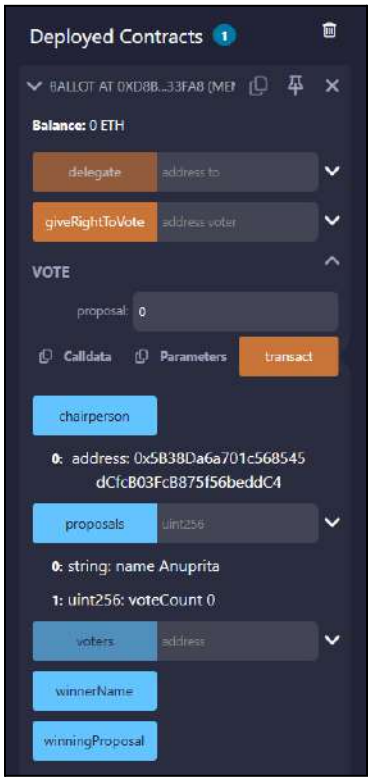


Step 5: Switch to Account 2

From Account dropdown select Account 2 - “Anu”

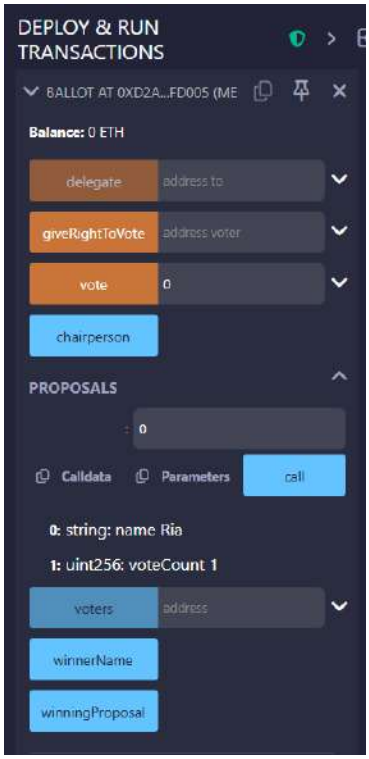


Now, vote for Account 1 - “Ria” by **vote(0)**.



Step 6: Check Proposal

Here, the vote count for Account 1 - “Ria” increases.



Step 7: Check winner

Here, Account 1 - “Ria” is the winner as it has the highest votes.

CONCLUSION:

Through this experiment, the Voting/Ballot smart contract was successfully deployed using Solidity in the Remix IDE. Important concepts such as require statements, mapping, and data locations like storage and memory were understood while performing the contract execution. The difference between using bytes32 and string for proposal names was also studied, which helped in understanding gas efficiency and readability. Overall, this experiment helped in gaining practical knowledge about designing and deploying a voting smart contract on the blockchain.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Ria Chaudhari **D20A** **14**

Blockchain Experiment 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

Q1: What is a Smart Contract?

A smart contract is a self-executing digital agreement stored on a blockchain. It automatically runs when certain conditions written in the code are met. Smart contracts do not require any middleman like a bank or lawyer because the rules and execution are controlled by the program itself. Once the contract is deployed on the blockchain, it cannot be easily changed and all transactions are recorded permanently. This makes the process transparent, secure, and reliable.

Q2: Significance of smart Contracts in Ethereum Blockchain

Smart contracts play a very important role in the Ethereum blockchain.

- **Automation:** Transactions and agreements are executed automatically when conditions are met.
- **No Intermediaries:** Removes the need for third parties like banks or agents.
- **Transparency:** All contract details and transactions are recorded on the blockchain and can be verified.
- **Security:** Data stored on the blockchain is difficult to alter, making contracts more secure.
- **Cost Efficient:** Reduces transaction and processing costs by eliminating middlemen.
- **Used in Many Applications:** Examples include voting systems, financial services (DeFi), supply chain management, and digital agreements.

CODE:

Smart Contract 1: OrderRecord.sol

Records each order placed on the platform permanently on the blockchain. Stores the buyer's wallet address, a comma-separated list of product IDs, the total amount in paisa, and a timestamp.

```
//Smart Contract 1: OrderRecord.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract OrderRecord {
    struct Order {
        address buyer;
        string productIds;
```

```
        uint256 totalPaisa;
        uint256 timestamp;
    }

    Order[] public orders;

    event OrderPlaced(address buyer, string productIds, uint256
totalPaisa, uint256 timestamp);

    function placeOrder(string memory _productIds, uint256 _totalPaisa)
public {
        orders.push(Order(msg.sender, _productIds, _totalPaisa,
block.timestamp));
        emit OrderPlaced(msg.sender, _productIds, _totalPaisa,
block.timestamp);
    }

    function getOrderCount() public view returns (uint256) {
        return orders.length;
    }
}
```

Smart Contract 2: ReviewVerification.sol

Records product reviews on the blockchain to ensure authenticity and prevent tampering. Stores the product ID, reviewer's wallet address, star rating (1–5), and a timestamp. Also prevents the same wallet from reviewing the same product twice.

```
//Smart Contract 2: ReviewVerification.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ReviewVerification {
    struct Review {
        string productId;
        address reviewer;
        uint8 rating;
        uint256 timestamp;
    }
```

```
    Review[] public reviews;
    mapping(string => mapping(address => bool)) public hasReviewed;

    event ReviewAdded(string productId, address reviewer, uint8 rating,
uint256 timestamp);

    function addReview(string memory _productId, uint8 _rating) public {
        require(_rating >= 1 && _rating <= 5, "Rating must be 1-5");
        require(!hasReviewed[_productId][msg.sender], "Already reviewed");

        reviews.push(Review(_productId, msg.sender, _rating,
block.timestamp));
        hasReviewed[_productId][msg.sender] = true;

        emit ReviewAdded(_productId, msg.sender, _rating,
block.timestamp);
    }

    function getReviewCount() public view returns (uint256) {
        return reviews.length;
    }
}
```

PROCEDURE

Step 1: Open Remix IDE

Open the Remix Integrated Development Environment in the browser. Remix IDE is used to write, compile, deploy, and test Solidity smart contracts.

https://remix.ethereum.org

Step 2: Create Smart contracts

a. **Order Record Contract**
   Records every order placed through the application permanently on the blockchain.

   Stores on blockchain: buyer wallet address, product IDs (comma-separated), total amount in paisa, and timestamp.

   - buyer wallet address
   - product IDs (comma-separated)
   - total amount in paisa
   - timestamp.

   Benefit:

- Prevents tampering of order records
- Provides transparent and immutable proof of purchase

b. Review Verification Contract

Ensures that product reviews are genuine and cannot be altered after submission.

Stores on blockchain:

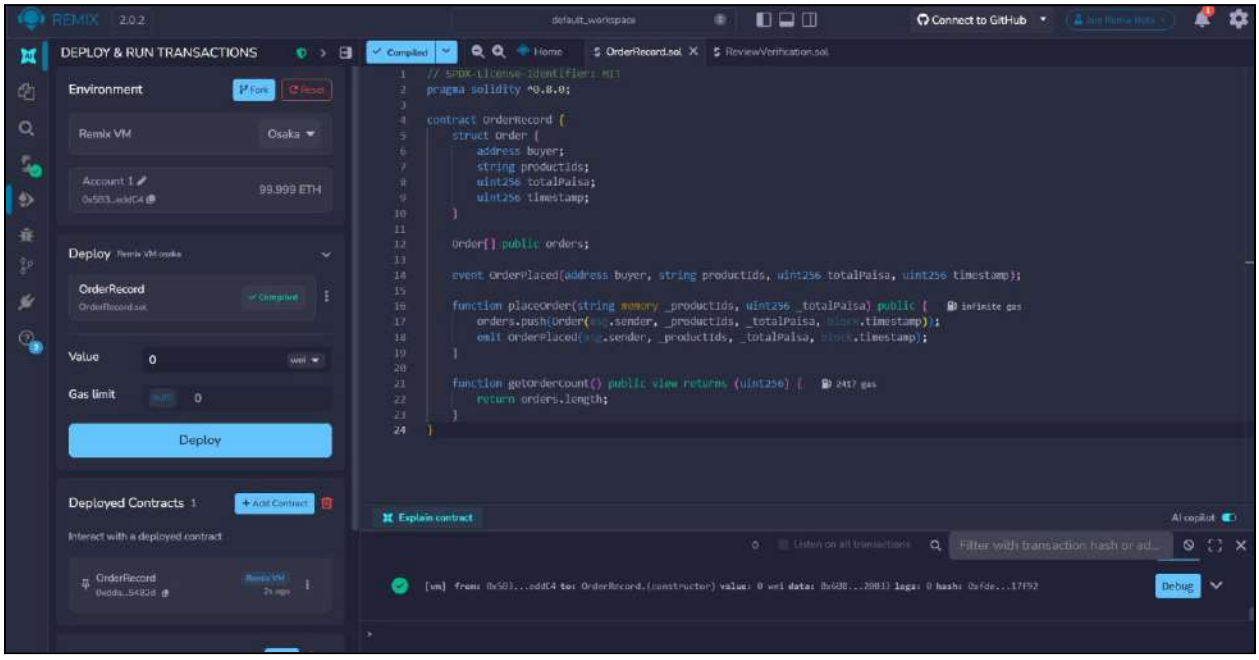
- product ID
- reviewer wallet address
- rating (1–5)
- timestamp.

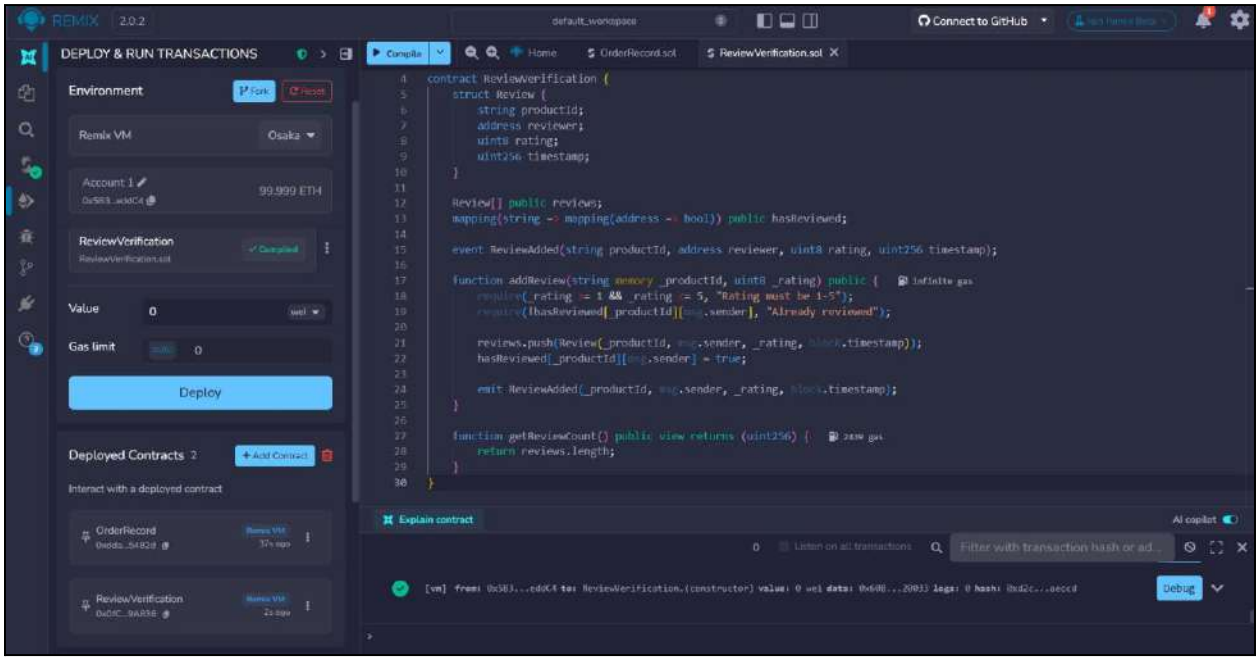
Benefit:

- Prevents fake or duplicate reviews from the same wallet
- Provides immutable, verifiable proof of each review

Step 3: Compile and deploy Smart contracts

Compile the smart contracts using the Solidity compiler in Remix IDE. After successful compilation, deploy the contracts on the Sepolia Test Network using the MetaMask wallet.

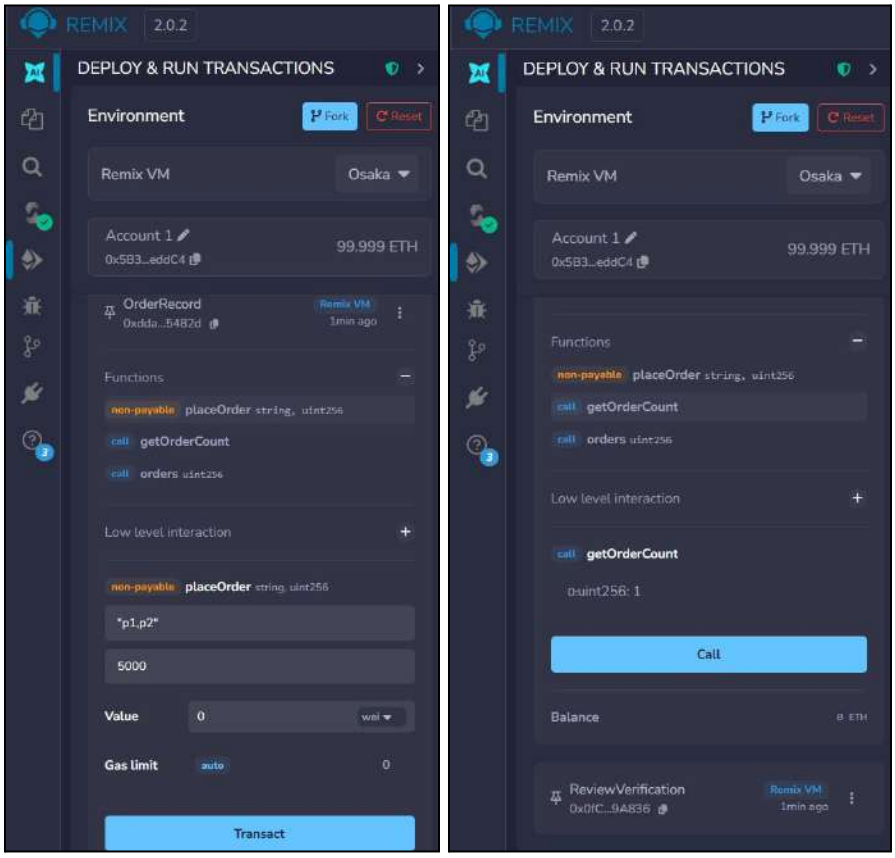




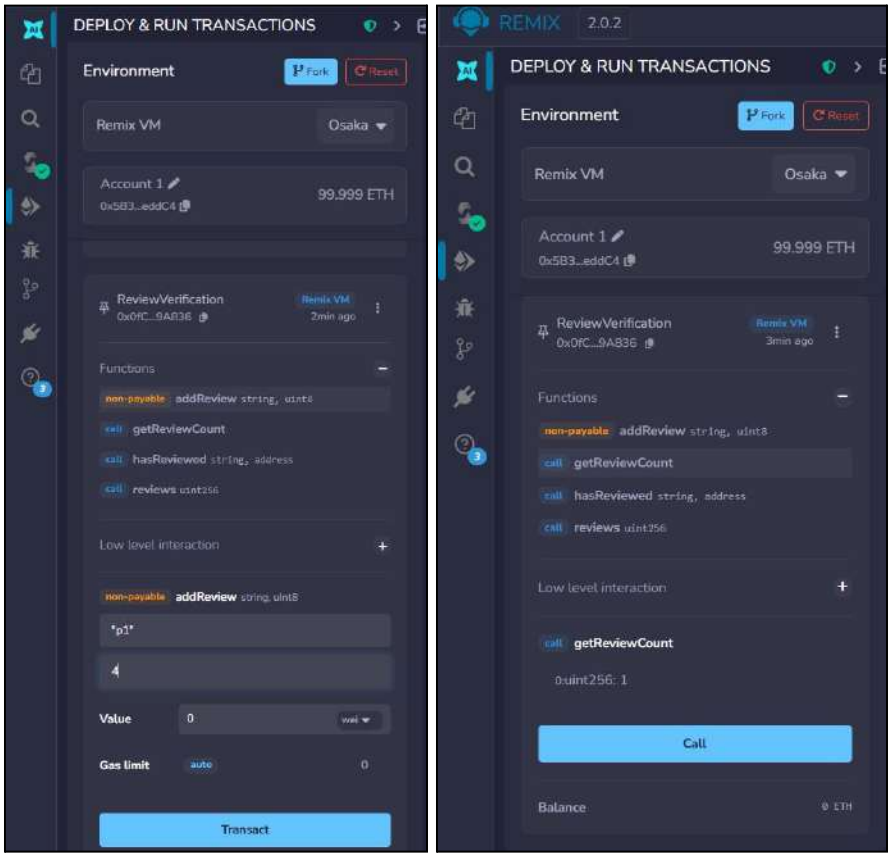
Step 4: Test the contracts

After deployment, test the contract functions such as addProduct, getProduct, and addReview. Verify that product details and reviews are stored correctly on the blockchain and can be retrieved using the contract functions.

a. Order Record Contract



b. Review Verification Contract



CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Ria Chaudhari **D20A** **14**

Blockchain Experiment 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

THEORY:

Q1: What is MetaMask?

MetaMask is a cryptocurrency wallet and browser extension that allows users to interact with blockchain networks such as Ethereum. It enables users to store, send, and receive digital assets securely. MetaMask also acts as a bridge between web browsers and decentralized applications (DApps), allowing users to sign transactions and deploy smart contracts. It provides account management, private key security, and seamless integration with platforms like Remix IDE.

Q2: What is a Test Network (Testnet)?

A test network, or testnet, is a blockchain environment used for testing purposes. It simulates the functionality of the main Ethereum network but uses virtual currency instead of real funds. Developers use testnets like Sepolia or Goerli to deploy and test smart contracts without financial risk. Testnets help identify bugs, verify functionality, and ensure proper execution before deploying applications on the mainnet.

Q3: Steps to Connect MetaMask with Remix IDE for Performing Transactions

- Install the MetaMask extension from the Chrome Web Store
- Create a new wallet or import an existing wallet using a secret recovery phrase
- Enable test networks in MetaMask settings
- Select a test network such as Sepolia
- Add test ETH using a faucet for performing transactions
- Open Remix IDE in the browser (https://remix.ethereum.org)
- Write and compile the smart contract in Remix
- Go to the Deploy & Run Transactions tab
- Select Injected Provider - MetaMask as the environment
- Connect MetaMask account when prompted
- Click Deploy and confirm the transaction in MetaMask
- Interact with deployed contract functions and approve transactions via MetaMask

CODE:

Smart Contract 1: OrderRecord.sol

Records each order placed on the platform permanently on the blockchain. Stores the buyer's wallet address, a comma-separated list of product IDs, the total amount in paisa, and a timestamp.

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
contract OrderRecord {
    struct Order {
        address buyer;
        string productIds;   // comma-separated e.g. "p1,p2,p3"
        uint256 totalPaisa;  // ₹199.99 stored as 19999
        uint256 timestamp;
    }

    Order[] public orders;

    event OrderPlaced(address buyer, string productIds, uint256
totalPaisa, uint256 timestamp);

    function placeOrder(string memory _productIds, uint256 _totalPaisa)
public {
        orders.push(Order(msg.sender, _productIds, _totalPaisa,
block.timestamp));
        emit OrderPlaced(msg.sender, _productIds, _totalPaisa,
block.timestamp);
    }

    function getOrderCount() public view returns (uint256) {
        return orders.length;
    }
}
```

Smart Contract 2: ReviewVerification.sol

Records product reviews on the blockchain to ensure authenticity and prevent tampering. Stores the product ID, reviewer's wallet address, star rating (1–5), and a timestamp. Also prevents the same wallet from reviewing the same product twice.

```
//Smart Contract 2: Review Verification.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ReviewVerification {
    struct Review {
        string productId;
```

```
        address reviewer;
        uint8 rating;
        uint256 timestamp;
    }

    Review[] public reviews;
    mapping(string => mapping(address => bool)) public hasReviewed;

    event ReviewAdded(string productId, address reviewer, uint8 rating,
uint256 timestamp);

    function addReview(string memory _productId, uint8 _rating) public {
        require(_rating >= 1 && _rating <= 5, "Rating must be 1-5");
        require(!hasReviewed[_productId][msg.sender], "Already reviewed");

        reviews.push(Review(_productId, msg.sender, _rating,
block.timestamp));
        hasReviewed[_productId][msg.sender] = true;

        emit ReviewAdded(_productId, msg.sender, _rating,
block.timestamp);
    }

    function getReviewCount() public view returns (uint256) {
        return reviews.length;
    }
}
```

PROCEDURE

Step 1: Install Metamask Chrome extension

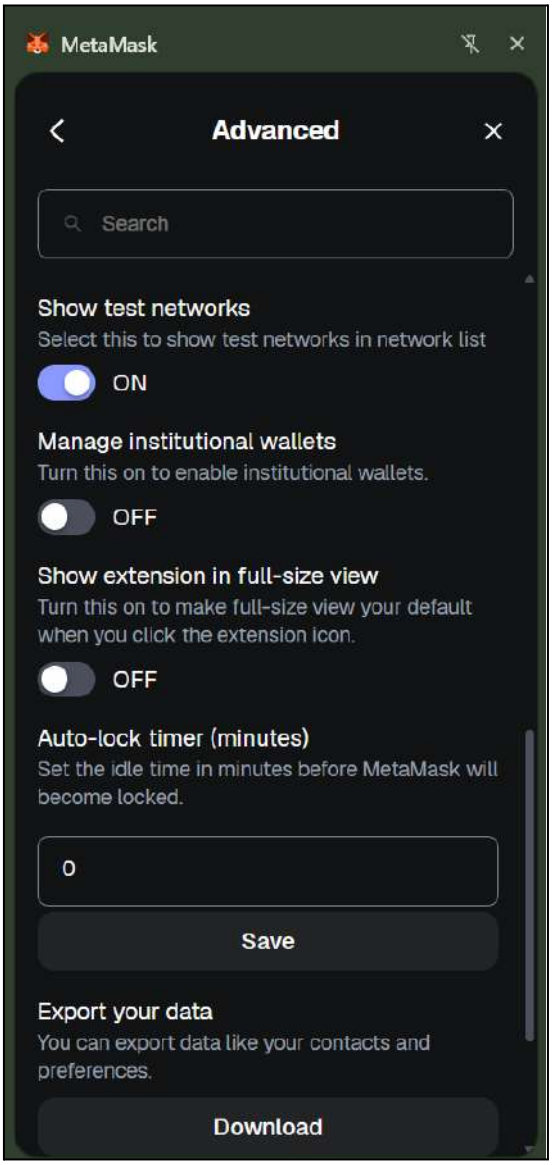
Install the MetaMask wallet extension from the Chrome Web Store. MetaMask is used to interact with the Ethereum blockchain and manage Ethereum accounts.

LINK:

https://chromewebstore.google.com/detail/metamask/nkbihfbeogaeaoehlefnkodbefgpgknn?pli=1

Step 2: Enable Test Network

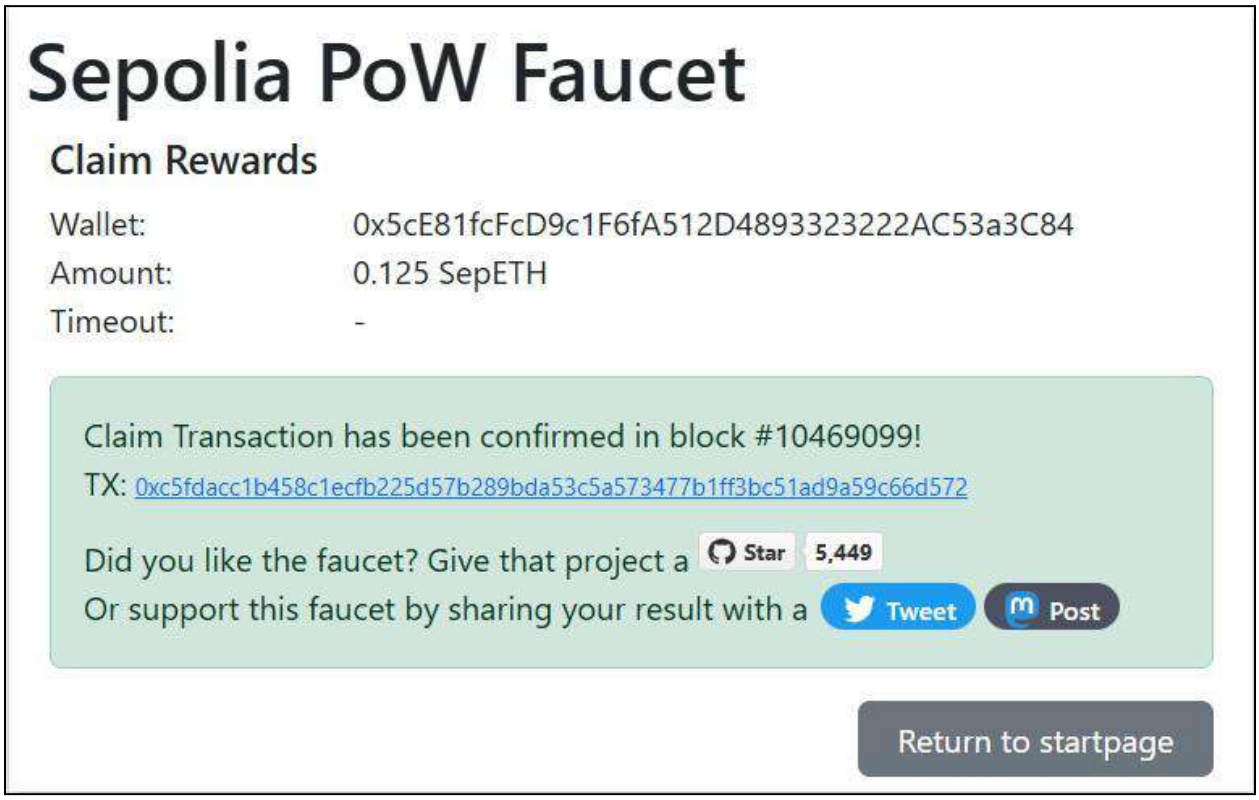
Open MetaMask and enable Test Networks from the settings. Select the Sepolia Test Network which allows users to test smart contracts without using real cryptocurrency.



Step 3: Claim free test ETH

Visit the Sepolia Faucet website and request free test Ethereum (ETH). These tokens are used to deploy and test smart contracts on the Sepolia test network.

https://sepolia-faucet.pk910.de/#/claim/c21e4c9c-4717-417f-b7cb-7fb785414d23



Step 4: Open Remix IDE

Open the Remix Integrated Development Environment in the browser. Remix IDE is used to write, compile, deploy, and test Solidity smart contracts.

https://remix.ethereum.org

Step 5: Create Smart contracts

a. **Order Record Contract**

Records every order placed through the application permanently on the blockchain.

Stores on blockchain: buyer wallet address, product IDs (comma-separated), total amount in paisa, and timestamp.

- buyer wallet address
- product IDs (comma-separated)
- total amount in paisa
- timestamp.

Benefit:

- Prevents tampering of order records
- Provides transparent and immutable proof of purchase

b. **Review Verification Contract**

Ensures that product reviews are genuine and cannot be altered after submission.

Stores on blockchain:

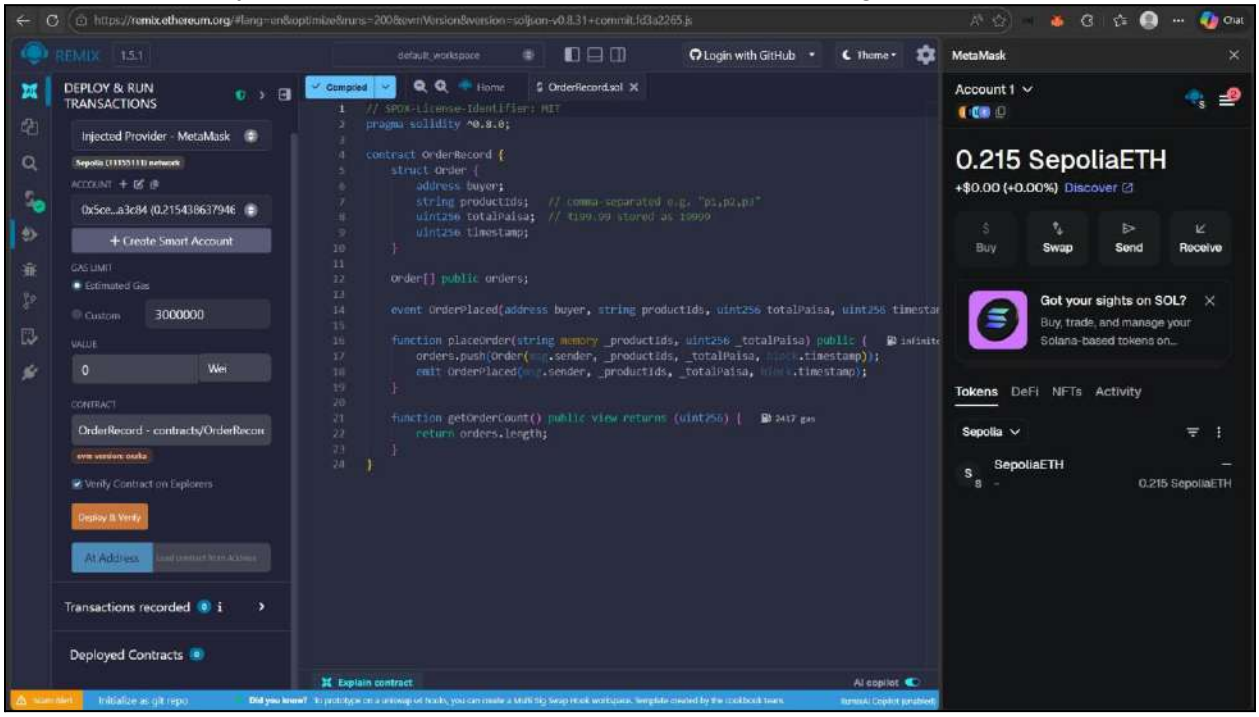
- product ID
- reviewer wallet address
- rating (1–5)
- timestamp.

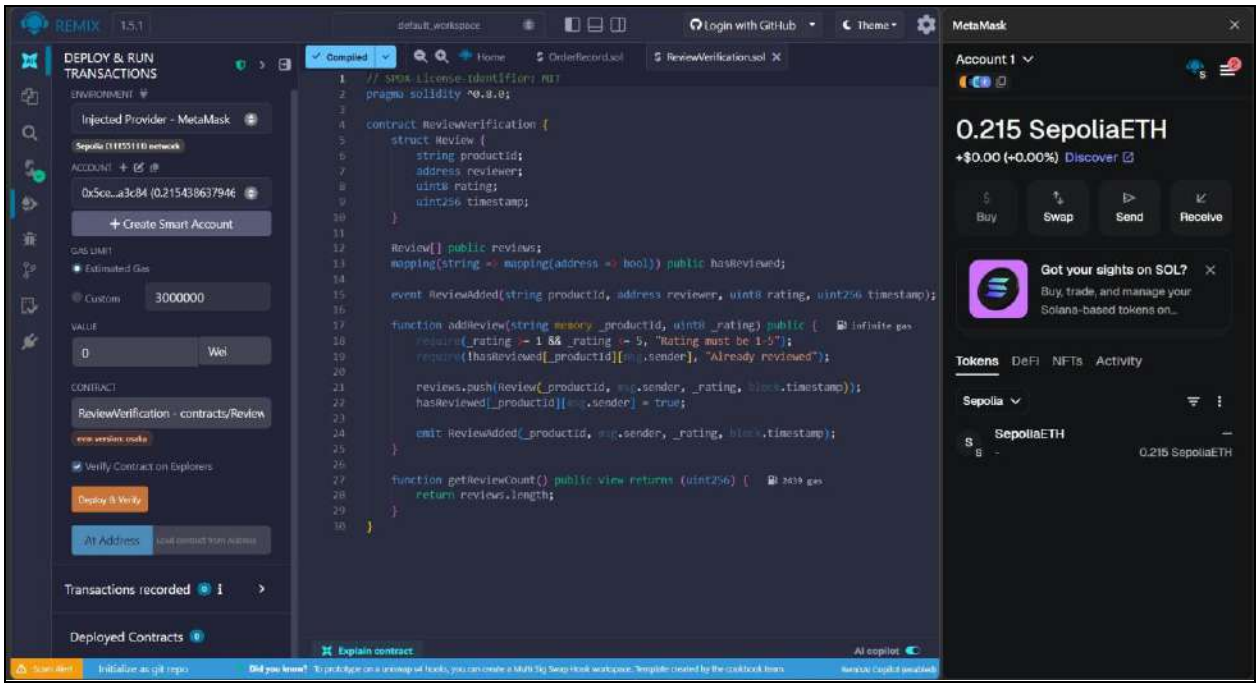
Benefit:

- Prevents fake or duplicate reviews from the same wallet
- Provides immutable, verifiable proof of each review

Step 6: Compile and deploy Smart contracts

Compile the smart contracts using the Solidity compiler in Remix IDE. After successful compilation, deploy the contracts on the Sepolia Test Network using the MetaMask wallet.

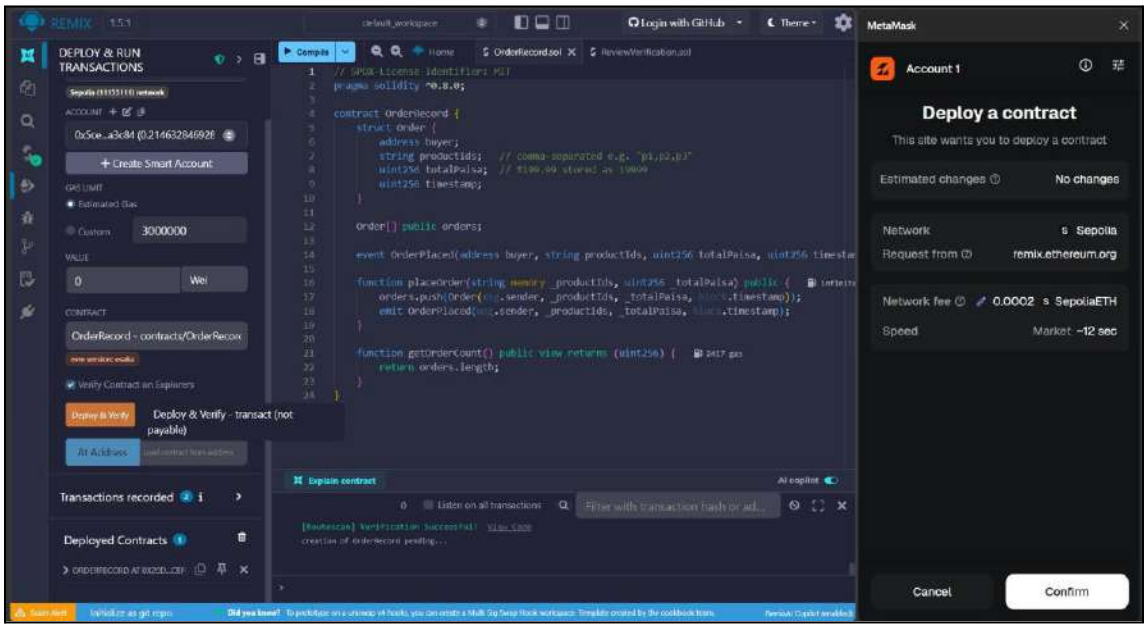


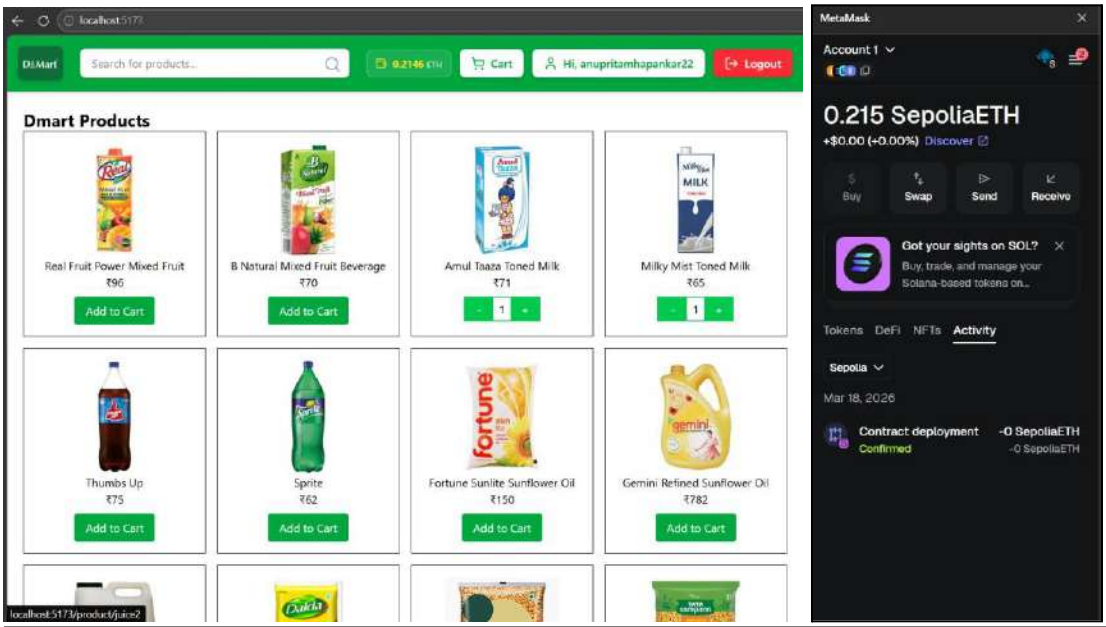


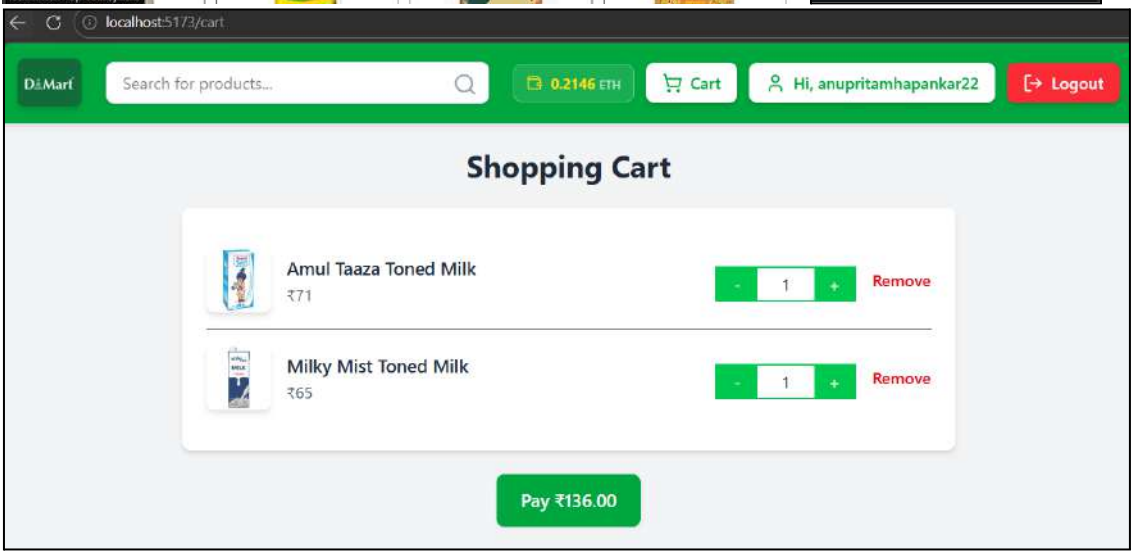
Step 7: Test the contracts

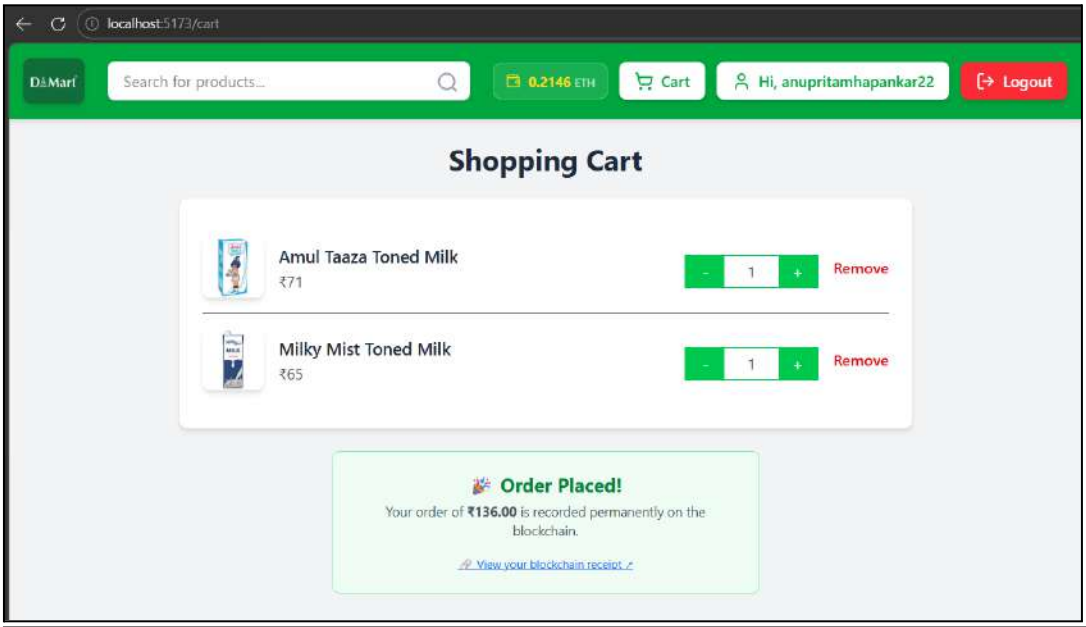
After deployment, test the contract functions such as addProduct, getProduct, and addReview. Verify that product details and reviews are stored correctly on the blockchain and can be retrieved using the contract functions.

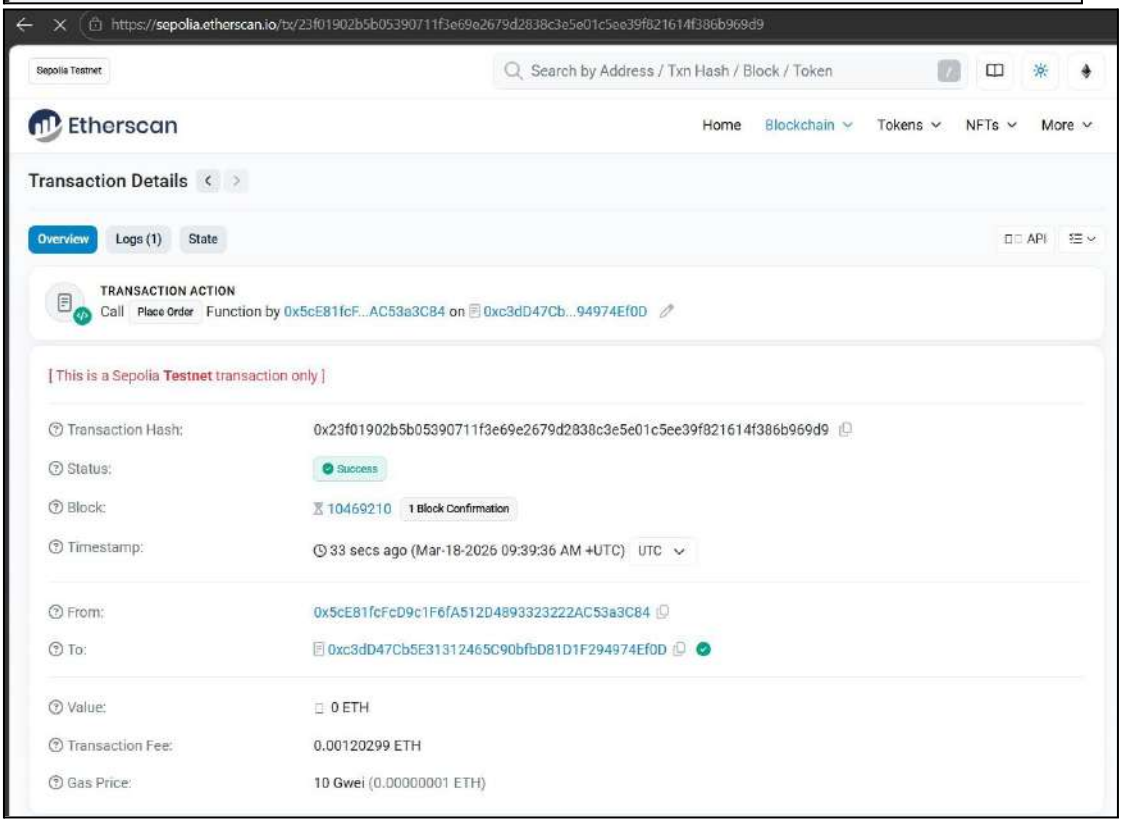
a. **Order Record Contract**



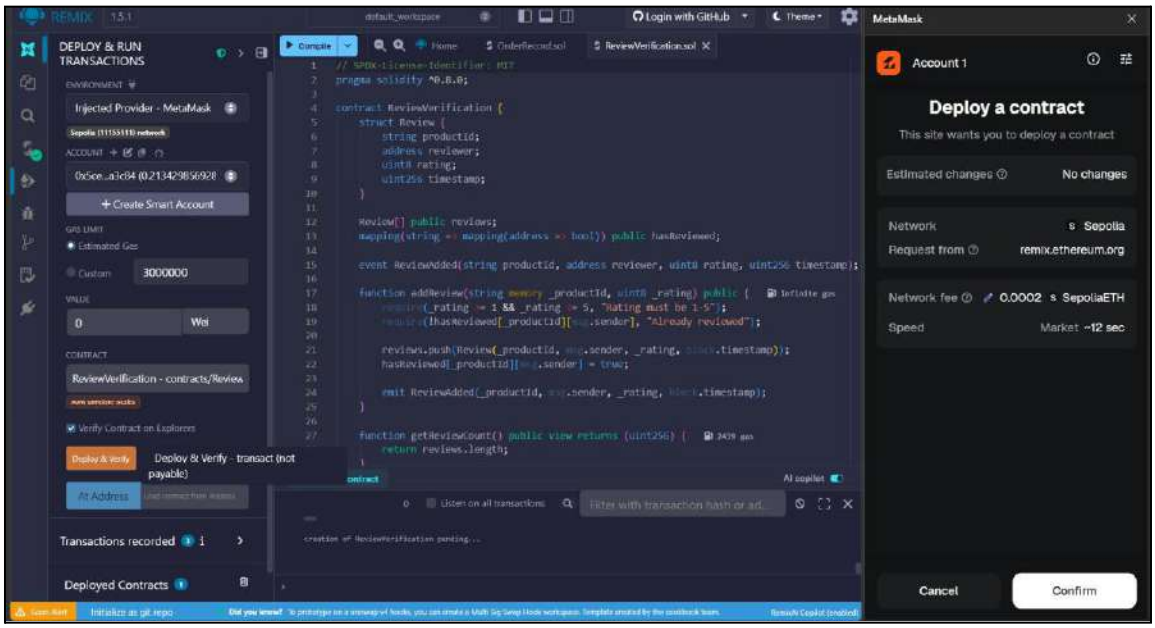

localhost:5173
DMart
Search for products...
0.2146 ETH
Cart
Hi, anupritamhapankar22
Logout
Dmart Products
Real Fruit Power Mixed Fruit
₹96
Add to Cart
B Natural Mixed Fruit Beverage
₹70
Add to Cart
Amul Taaza Toned Milk
₹71
1
Milky Mist Toned Milk
₹65
1
Thumbs Up
₹75
Add to Cart
Sprite
₹62
Add to Cart
Fortune Sunlite Sunflower Oil
₹150
Add to Cart
Gemini Refined Sunflower Oil
₹782
Add to Cart
localhost:5173/product/juice2
MetaMask
Account 1
0.215 SepoliaETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive
Got your sights on SOL?
Buy, trade, and manage your Solana-based tokens on...
Tokens
DeFi
NFTs
Activity
Sepolia
Mar 18, 2026
Contract deployment
Confirmed
-0 SepoliaETH
-0 SepoliaETH

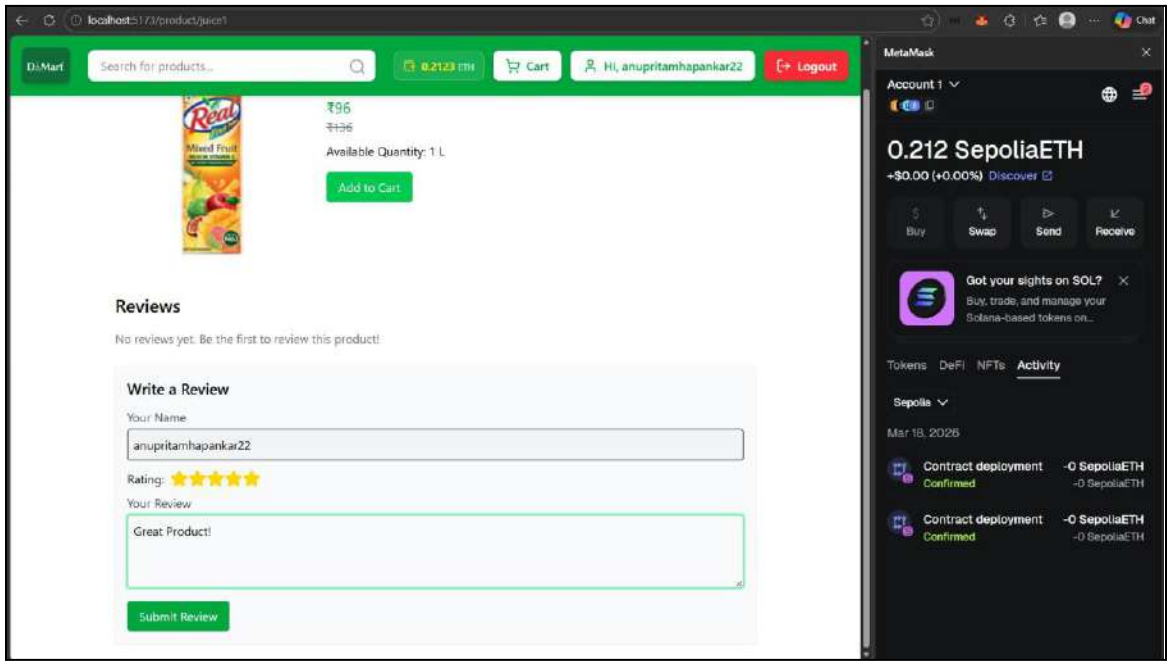

localhost:5173/cart
DMart
Search for products...
0.2146 ETH
Cart
Hi, anupritamhapankar22
Logout
Shopping Cart
Amul Taaza Toned Milk
₹71
1
Remove
Milky Mist Toned Milk
₹65
1
Remove
Pay ₹136.00

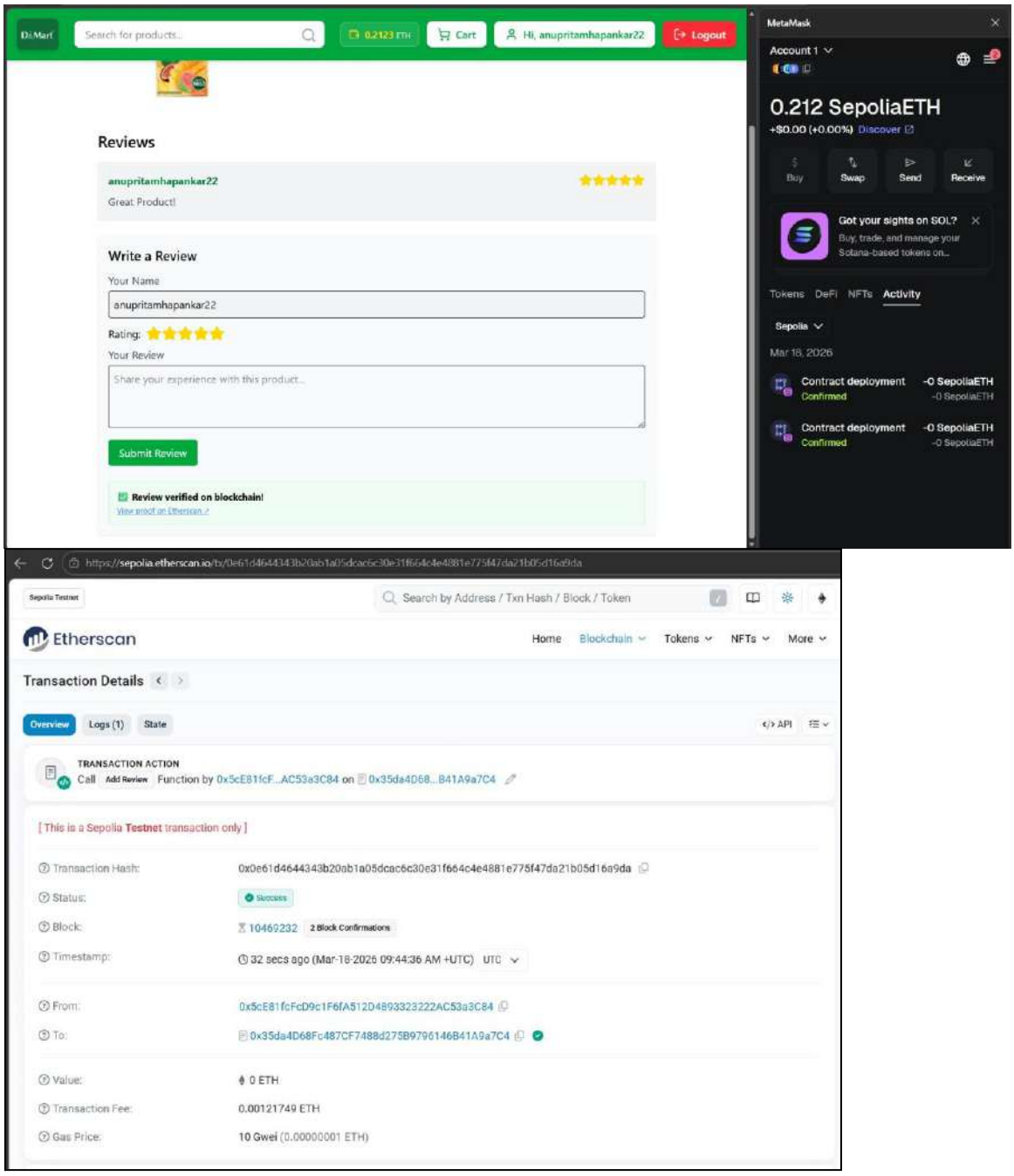

localhost:5173/cart
DMart
Search for products...
0.2146 ETH
Cart
Hi, anupritamhapankar22
Logout
Shopping Cart
Amul Taaza Toned Milk
₹71
1
Remove
Milky Mist Toned Milk
₹65
1
Remove
Order Placed!
Your order of ₹136.00 is recorded permanently on the blockchain.
View your blockchain receipt


https://sepolia.etherscan.io/tx/23f01902b5b05390711f3e69e2679d2838c3e5e01c5ee39f821614f386b969d9
Sepolia Testnet
Search by Address / Txn Hash / Block / Token
Etherscan
Home
Blockchain
Tokens
NFTs
More
Transaction Details
Overview
Logs (1)
State
API
TRANSACTION ACTION
Call Place Order Function by 0x5cE81fcF...AC53a3C84 on 0xc3dD47Cb...94974Ef0D
[This is a Sepolia Testnet transaction only]
Transaction Hash:
0x23f01902b5b05390711f3e69e2679d2838c3e5e01c5ee39f821614f386b969d9
Status:
Success
Block:
10469210
1 Block Confirmation
Timestamp:
33 secs ago (Mar-18-2026 09:39:36 AM +UTC)
UTC
From:
0x5cE81fcFcD9c1F6fA512D4893323222AC53a3C84
To:
0xc3dD47Cb5E31312465C90bfbD81D1F294974Ef0D
Value:
0 ETH
Transaction Fee:
0.00120299 ETH
Gas Price:
10 Gwei (0.00000001 ETH)

b. Review Verification Contract







CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment 8

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

• Decentralization: No single entity controls the network; authority is distributed across nodes. •

Immutability: Once data is written to the blockchain, it cannot be altered or deleted. •

Transparency: All transactions are visible to network participants.

• Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).

• Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM),which executes the contract logic on every node in the network.

Key concepts:

• Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.

• ABI (Application Binary Interface): Defines how to interact with the deployed contract — the

function signatures and data encodings.

• Address: Every deployed contract gets a unique Ethereum address on the blockchain.

• msg.sender: A global variable in Solidity that holds the address of the account that called the

current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates a full Ethereum node locally on your machine, providing:

• 10 pre-funded test accounts, each loaded with 100 ETH.

• Instant mining — transactions are confirmed immediately (no wait time).

• A local RPC server (default: http://127.0.0.1:7545) that tools like MetaMask and Remix can

connect to.

• A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test

contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

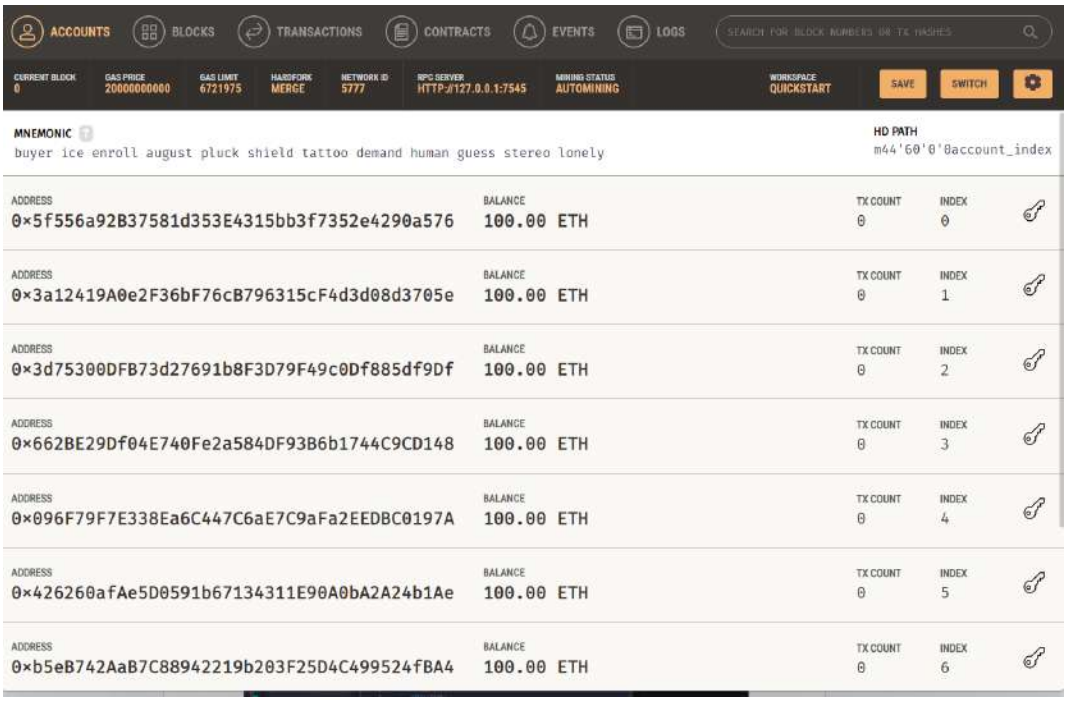
Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data

Implementation:

Step 1:Install Ganache and click on Quick Start Ethereum



Note the following details from Ganache for later use:

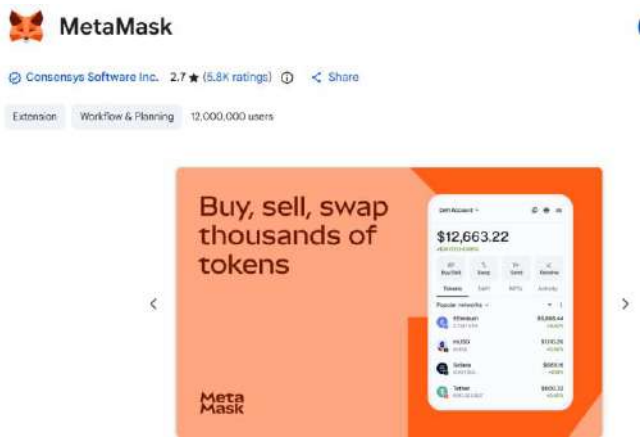
• RPC Server URL: http://127.0.0.1:7545

• Network ID: 5777

• Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications. Open Google Chrome and go to the Chrome Web Store.Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.



After installation, click 'Create a new wallet' on the MetaMask welcome screen. Set a password and save the Secret Recovery Phrase securely

Add Ganache Network to MetaMask

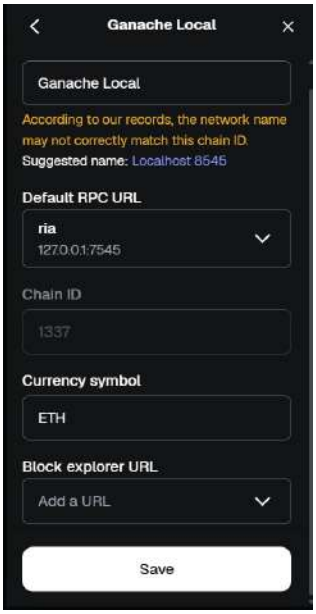
9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the RPC URL:

RPC URL: http://127.0.0.1:7545

RPC Name: Ganache Local

11. Click Save

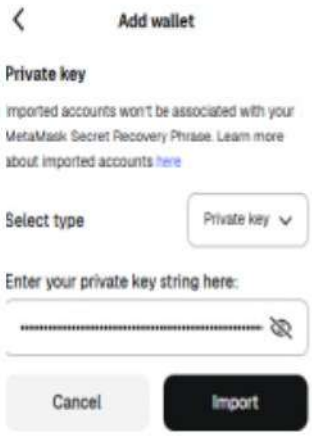


Import Ganache Accounts into MetaMask

In Ganache, click the key icon next to an account address to reveal its private key.

In MetaMask, go to Accounts > Add wallet > Import using Private Key.

Paste the private key and click Import.



Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

Open remix.ethereum.org in the browser.

In the File Explorer, create a new file named DmartRetail.sol.

Write the following smart contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract DmartRetail {
struct Order {
address buyer;
string productIds;
uint totalAmount;
uint timestamp;
}
struct Review {
string productId;
address reviewer;
uint8 rating;
}
Order[] public orders;
Review[] public reviews;
mapping(string => mapping(address => bool)) public reviewed;
// Place Order
function placeOrder(string memory _products, uint _amount) public {
orders.push(Order(msg.sender, _products, _amount, block.timestamp));
}
// Add Review
```

```
function addReview(string memory _productId, uint8 _rating) public {
require(_rating >= 1 && _rating <= 5, "Invalid rating");
require(!reviewed[_productId][msg.sender], "Already reviewed");
reviews.push(Review(_productId, msg.sender, _rating));
reviewed[_productId][msg.sender] = true;
}
// Get total orders
function getOrderCount() public view returns(uint) {
return orders.length;
}
// Get total reviews
function getReviewCount() public view returns(uint) {
return reviews.length;
}
}
```

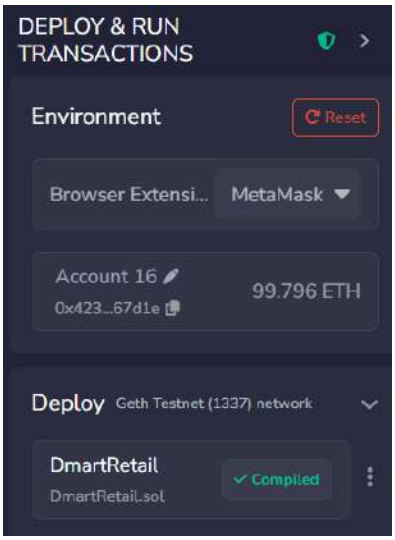
Compile the Smart Contract

In Remix, click the Solidity Compiler icon on the left sidebar.

Select compiler version 0.8.x.

Click Compile SupplyChain.sol.

A green checkmark confirms successful compilation.

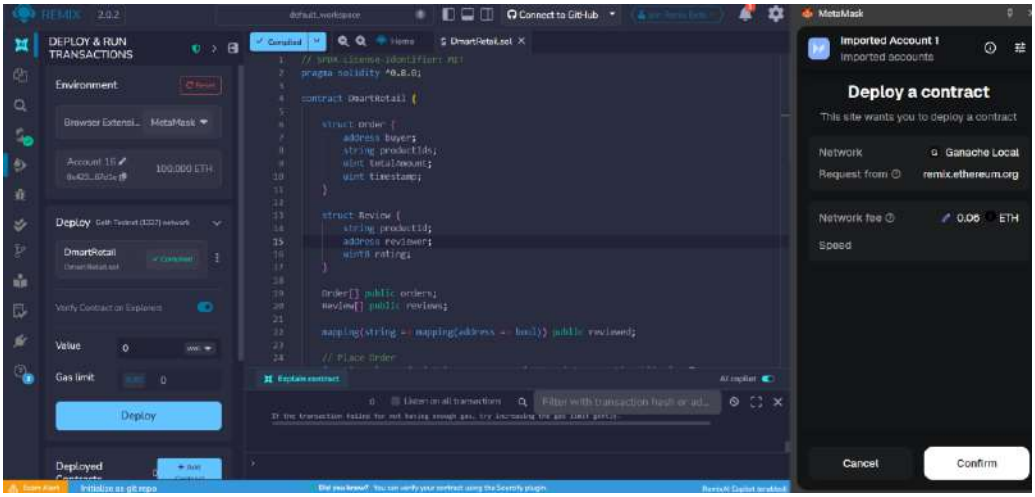


Deploy the Smart Contract

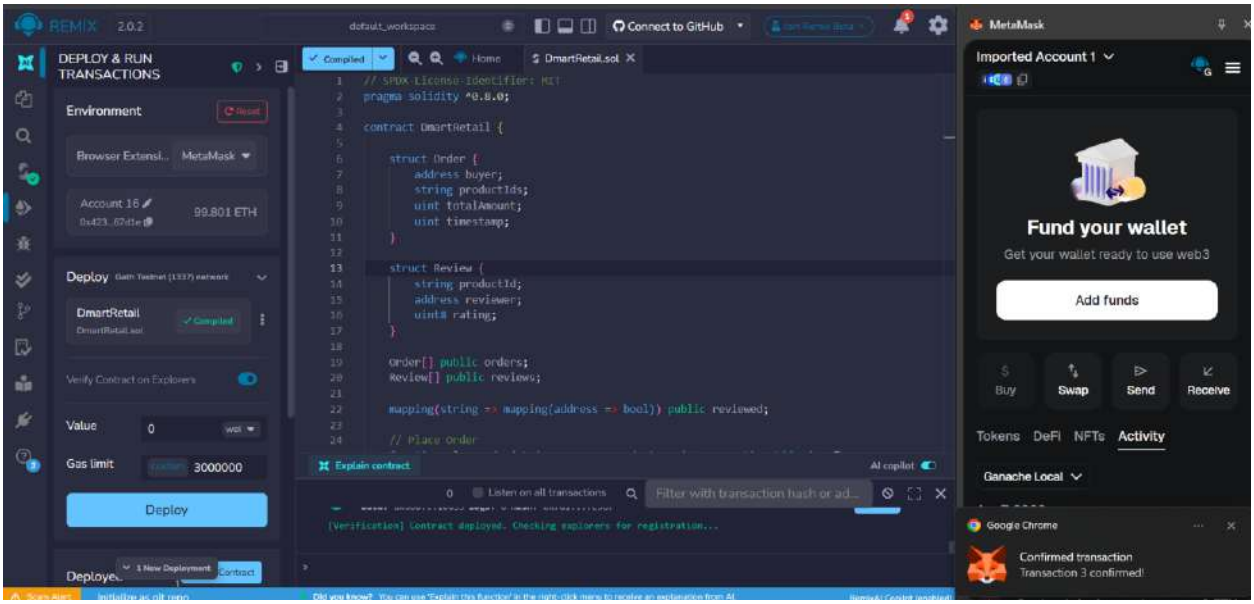
In the Deploy & Run panel, confirm DmartRetail is selected under Contract.

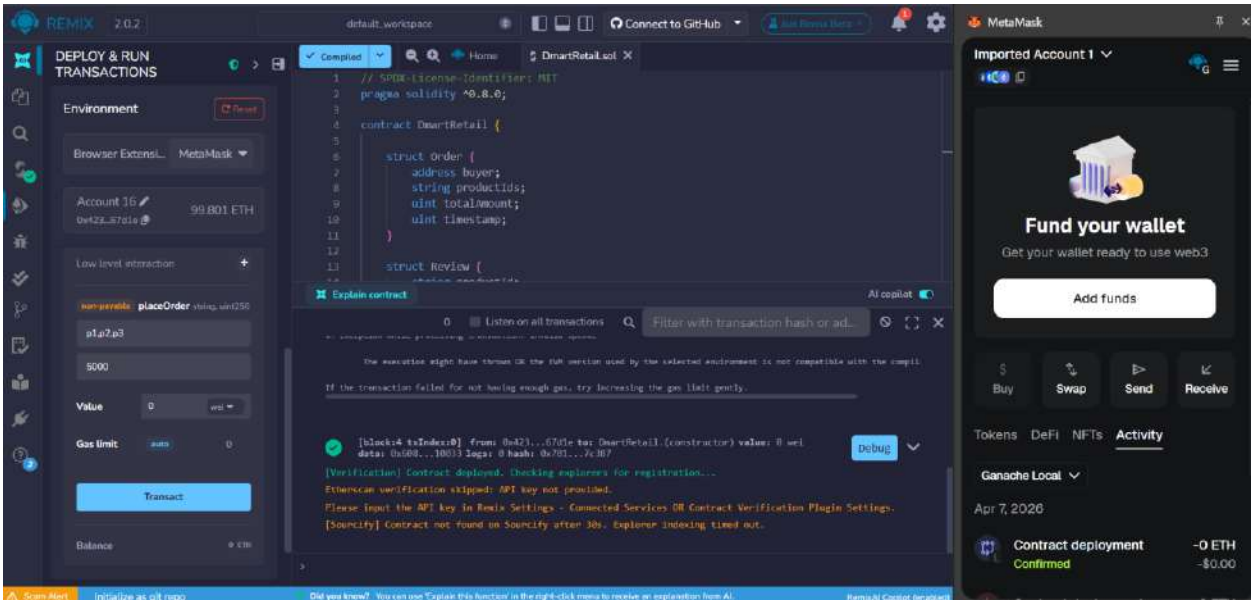
Click Deploy.

MetaMask will open a confirmation popup.



A 'Confirmed transaction' notification from Chrome confirms the deployment was successful





The contract deployment details in MetaMask show:

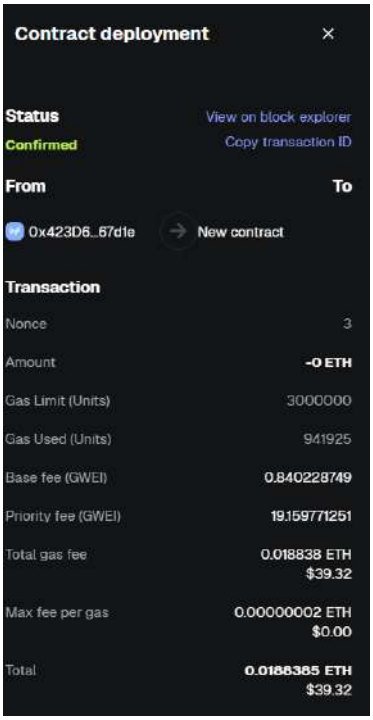
• Status: Confirmed

• From: 0x423D6…67d1e→ To: New contract

• Gas Used: 941,926 units

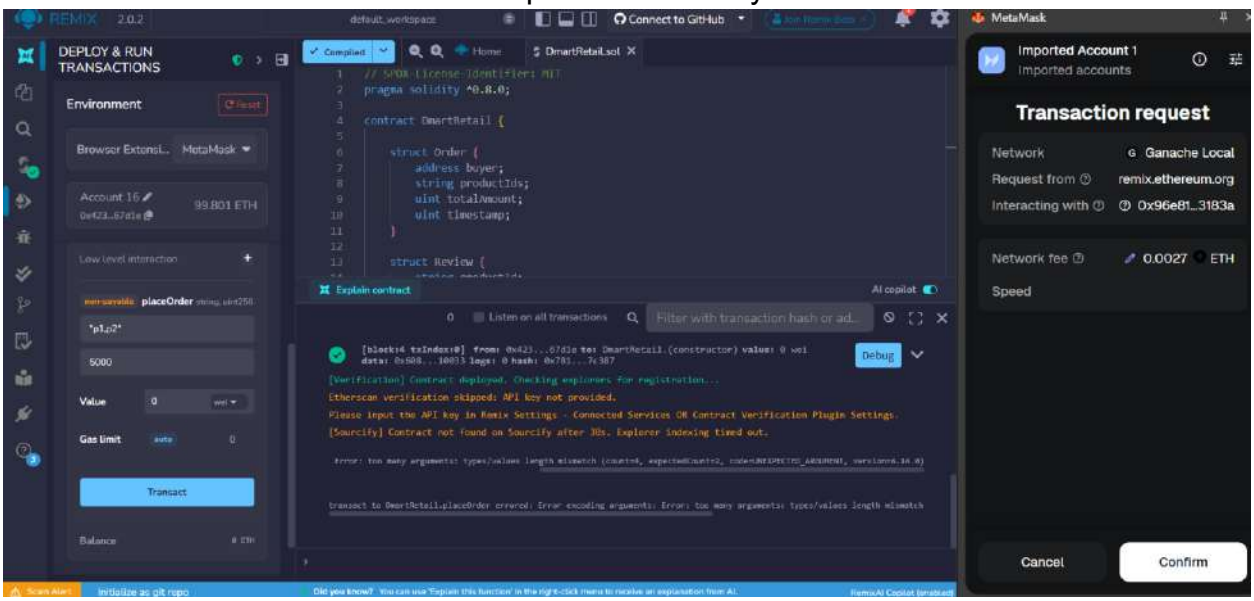
• Total gas fee: 0.018838 ETH

• Account balance after: 99.80 ETH

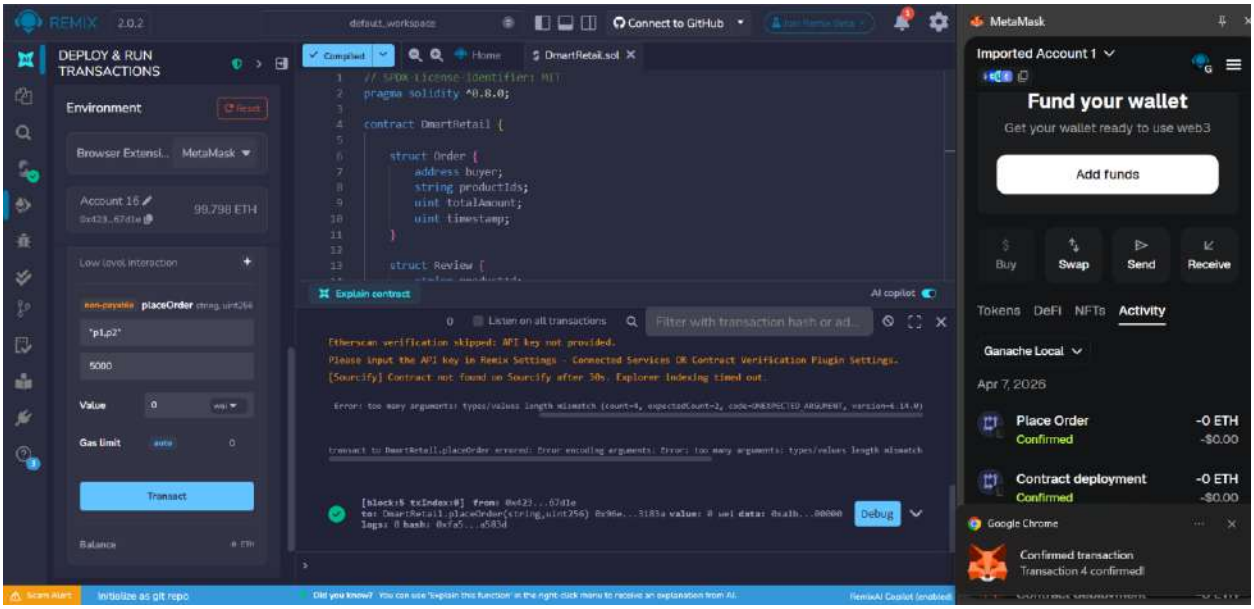


Interact with the Smart Contract After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

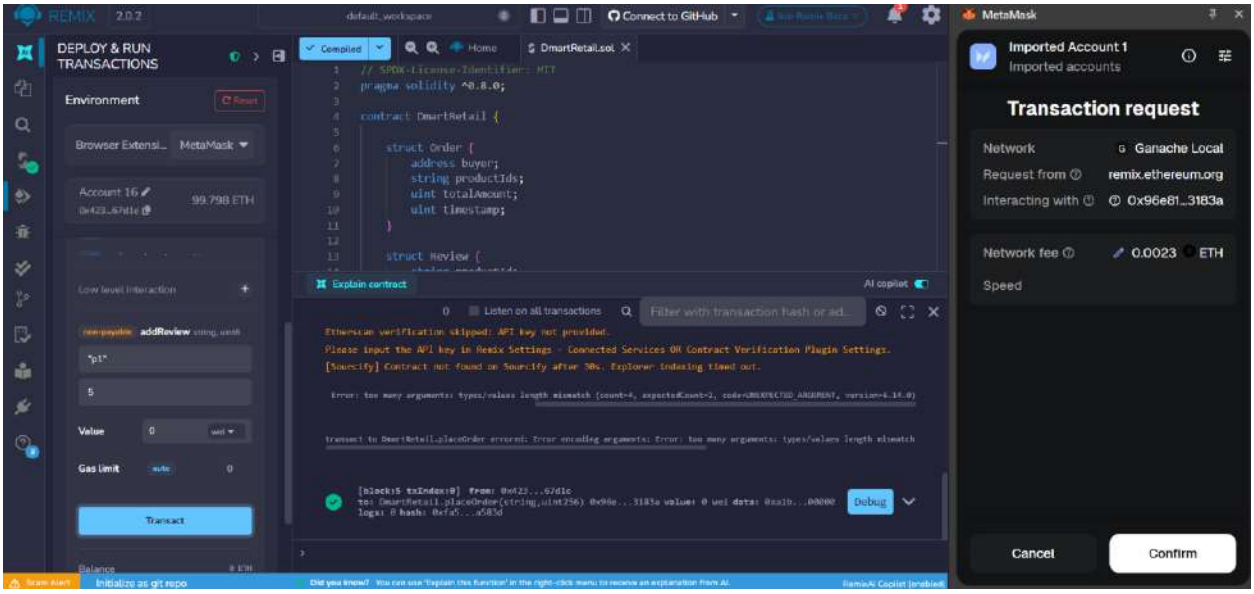
Call placeOrder(p1,p2,5000) where p1,p2 are products and 5000 is the amount.Below screenshot shows the transaction request received by MetaMask from ethereum.



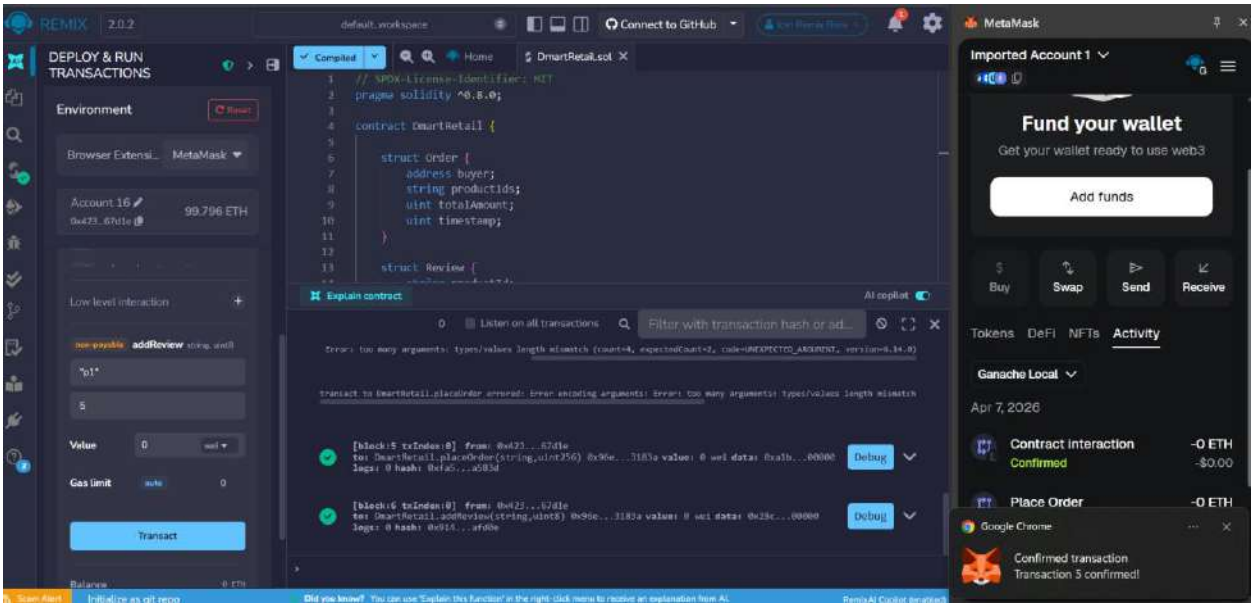
Order Placed is Confirmed.



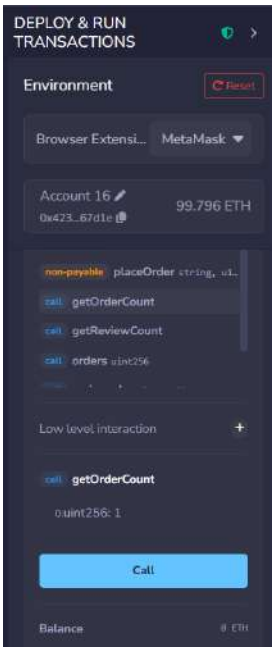
Call addReview where p1 is the product and 5 is the rating(review) given to it.



Review Confirmed.



Verify Data using getOrderCount() and getReviewCount()



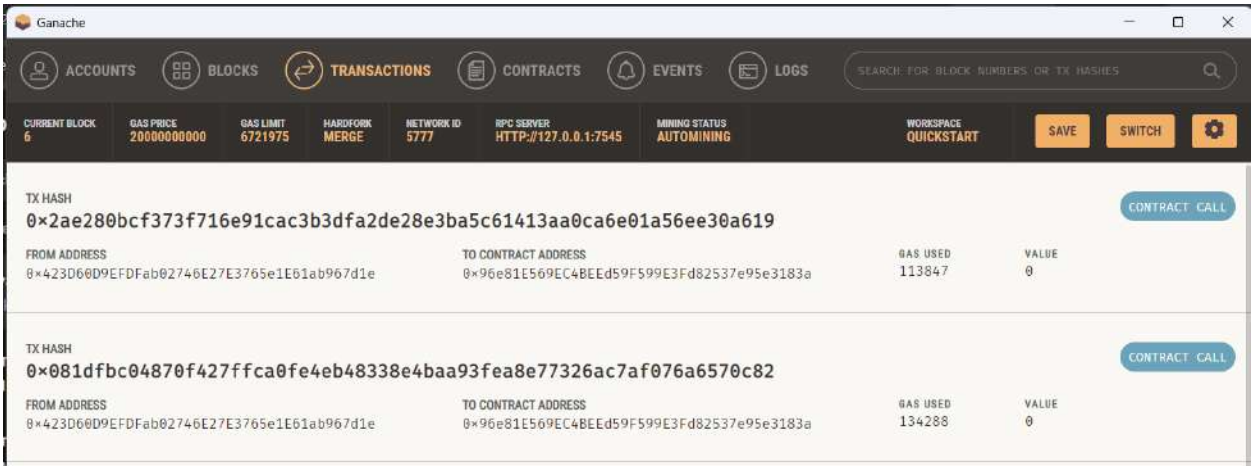
	

Expected Outcome:

0:

uint256: 1

Output shown in Ganache:



Conclusion:

The experiment was successfully completed using Ganache v2.7.1, MetaMask, and Remix IDE. A pharmaceutical supply chain smart contract (PharmaSupplyChain) was written in Solidity, compiled, and deployed to the local Ganache blockchain via

MetaMask. The contract tracked a medicine batch through all four stages of the supply chain — manufacturer, distributor, retailer, and customer — with each stage recorded on-chain. Transaction details were verified in both MetaMask and the Ganache environment, confirming correct gas deduction and block creation

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Ria Chaudhari D20A14

Blockchain Experiment 9

AIM: Implement the Private Ethereum Blockchain using Geth

THEORY:

Q1: What is Geth?

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go

programming language. It allows users to run an Ethereum node, mine Ether, and interact with

the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Q2: Significance of a Private Ethereum Network

Setting up a Private Ethereum Network involves creating a customized blockchain environment

isolated from the public Ethereum network. It enables developers to test smart contracts,

consensus mechanisms, and network behavior without incurring gas costs or depending on

public validators.

Q3: Steps for creating a Private Ethereum Network

Steps Involved:

- **Choosing a Network ID:**

Each private blockchain must have a unique network ID to differentiate it from public or

other private networks.

- **Choosing a Consensus Algorithm:**

Ethereum supports different consensus mechanisms such as Proof of Work (PoW) and

Proof of Authority (PoA). In private networks, Clique (PoA) is often used for faster block

confirmation.

● **Creating a Genesis Block:**

The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.

● **Initializing the Geth Database:**

The Geth init command initializes the blockchain using the genesis.json file, preparing the data directory for node operation.

● **Setting up Networking:**

Nodes communicate using enodes (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.

● **Running the Member Nodes:**

Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.

● **Running a Signer (in Clique):**

In the Clique consensus, designated signers validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

NOTE: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

PROCEDURE

Step 1: Install Geth

wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz

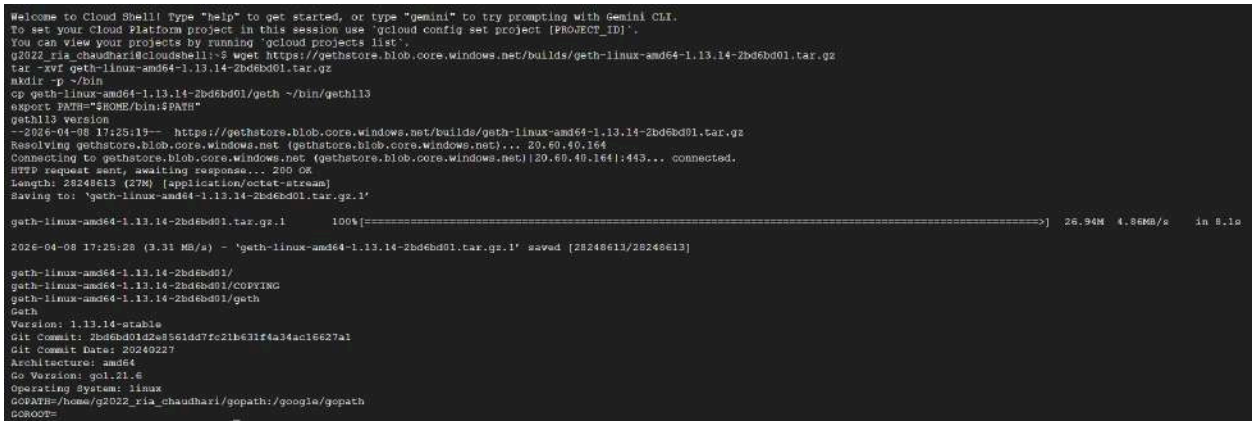
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz

mkdir -p ~/bin

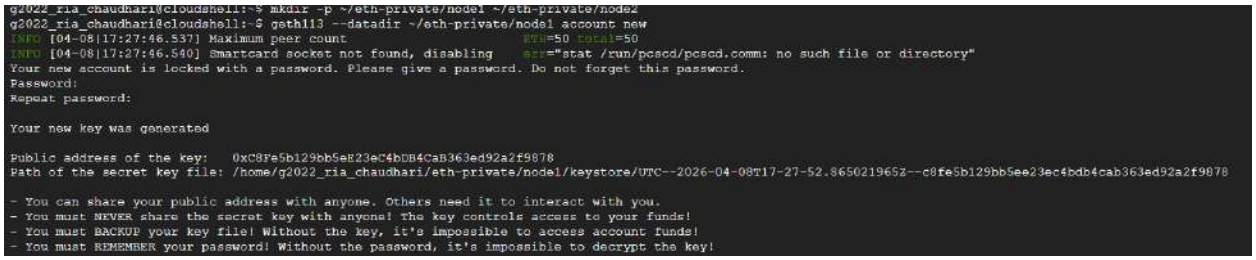
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113

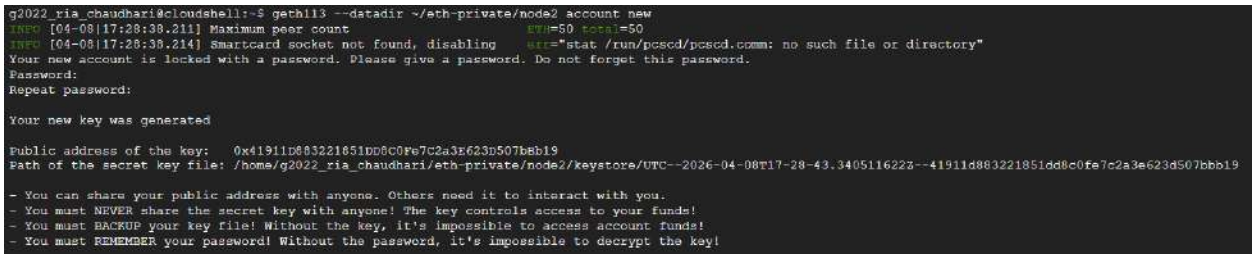
export PATH="$HOME/bin:$PATH"

geth113 version

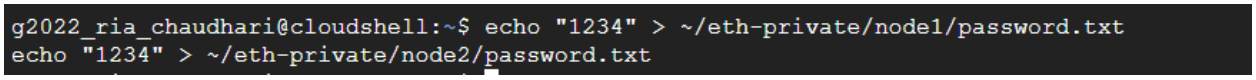


Step 2: Create folders and accounts for both nodes





Step 3: Create password files



Step 4: Create genesis.json and verify it looks correct

NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"

NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"

```
cat > ~/eth-private/genesis.json << EOF

{

"config": {

"chainId": 1234,

"homesteadBlock": 0,

"eip150Block": 0,

"eip155Block": 0,

"eip158Block": 0,

"byzantiumBlock": 0,

"constantinopleBlock": 0,

"petersburgBlock": 0,

"clique": {

"period": 5,

"epoch": 30000

}

},

"difficulty": "1",

"gasLimit": "8000000",

"extradata":

"0x0000000000000000000000000000000000000000000000000000000000000000${NODE1

}00000000000000000000000000000000000000000000000000000000000000000000000000000000

0000000000000000000000000000000000000000000000000000000000000000",
```

"alloc": {

"0x${NODE1}": { "balance": "1000000000000000000000" },

"0x${NODE2}": { "balance": "1000000000000000000000" }

}

}

EOF

```
g2022_ria_chaudhari@cloudshell:~$ NODE1="C8Fe5b129bb5eE23eC4bDB4CaB363ed92a2f9878"
NODE2="41911D883221851DD8C0Fe7C2a3E623D507bBb1"

cat > ~/eth-private/genesis.json << EOF
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
```

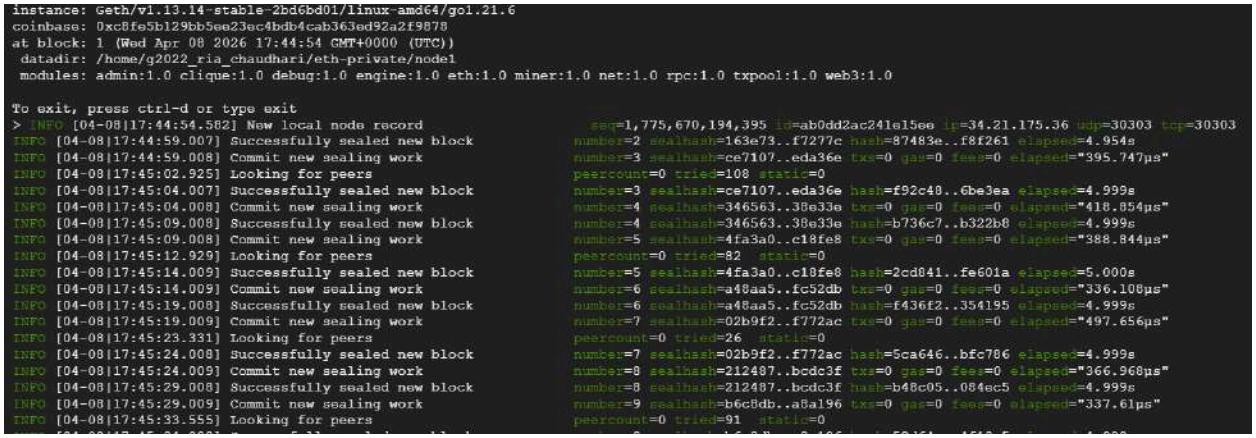
```
EOF
g2022_ria_chaudhari@cloudshell:~$ cat ~/eth-private/genesis.json
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000C8Fe5b129bb5eE23eC4bDB4CaB363ed92a2f98780000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0xC8Fe5b129bb5eE23eC4bDB4CaB363ed92a2f9878": { "balance": "1000000000000000000000" },
    "0x41911D883221851DD8C0Fe7C2a3E623D507bBb1": { "balance": "1000000000000000000000" }
  }
}
g2022_ria_chaudhari@cloudshell:~$
```

Step 5: Initialize both nodes

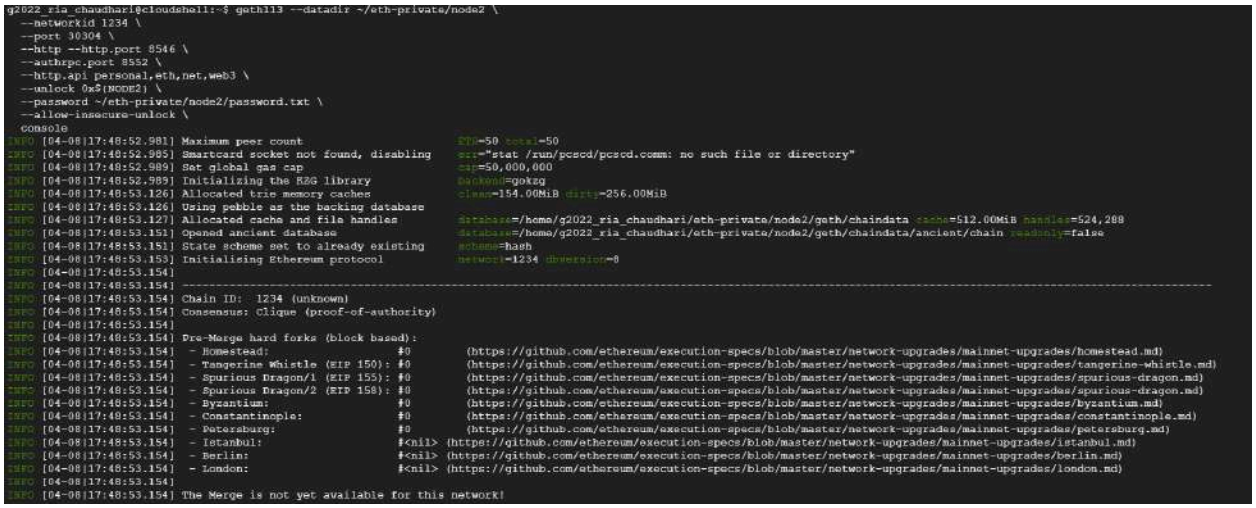
```
g2022_ria_chaudhari@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-08|17:42:33.075] Maximum peer count                       ETH=50 total=50
INFO [04-08|17:42:33.079] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-08|17:42:33.085] Set global gas cap                       cap=50,000,000
INFO [04-08|17:42:33.086] Initializing the KZG library             backend=gokzg
INFO [04-08|17:42:33.219] Defaulting to pebble as the backing database
INFO [04-08|17:42:33.219] Allocated cache and file handles         database=/home/g2022_ria_chaudhari/eth-private/node1/geth/chaindata cache=16.00MiB handles=16
INFO [04-08|17:42:33.300] Opened ancient database                  database=/home/g2022_ria_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-08|17:42:33.300] State schema set to default              scheme=hash
INFO [04-08|17:42:33.301] Writing custom genesis block
INFO [04-08|17:42:33.304] Persisted trie from memory database      nodes=4 size=478.00B time=3.369394ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [04-08|17:42:33.340] Successfully wrote genesis state         database=chaindata hash=38de1a..afc19d
INFO [04-08|17:42:33.340] Defaulting to pebble as the backing database
INFO [04-08|17:42:33.340] Allocated cache and file handles         database=/home/g2022_ria_chaudhari/eth-private/node1/geth/lightchaindata cache=16.00MiB handles=16
INFO [04-08|17:42:33.400] Opened ancient database                  database=/home/g2022_ria_chaudhari/eth-private/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [04-08|17:42:33.400] State schema set to default              scheme=hash
INFO [04-08|17:42:33.400] Writing custom genesis block
INFO [04-08|17:42:33.404] Persisted trie from memory database      nodes=4 size=478.00B time=3.273769ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [04-08|17:42:33.438] Successfully wrote genesis state         database=lightchaindata hash=38de1a..afc19d
INFO [04-08|17:42:33.515] Maximum peer count                       ETH=50 total=50
INFO [04-08|17:42:33.518] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-08|17:42:33.523] Set global gas cap                       cap=50,000,000
INFO [04-08|17:42:33.524] Initializing the KZG library             backend=gokzg
INFO [04-08|17:42:33.701] Defaulting to pebble as the backing database
INFO [04-08|17:42:33.701] Allocated cache and file handles         database=/home/g2022_ria_chaudhari/eth-private/node2/geth/chaindata cache=16.00MiB handles=16
INFO [04-08|17:42:33.758] Opened ancient database                  database=/home/g2022_ria_chaudhari/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-08|17:42:33.758] State schema set to default              scheme=hash
INFO [04-08|17:42:33.758] Writing custom genesis block
INFO [04-08|17:42:33.762] Persisted trie from memory database      nodes=4 size=478.00B time=3.381802ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [04-08|17:42:33.798] Successfully wrote genesis state         database=chaindata hash=38de1a..afc19d
INFO [04-08|17:42:33.798] Defaulting to pebble as the backing database
INFO [04-08|17:42:33.798] Allocated cache and file handles         database=/home/g2022_ria_chaudhari/eth-private/node2/geth/lightchaindata cache=16.00MiB handles=16
INFO [04-08|17:42:33.853] Opened ancient database                  database=/home/g2022_ria_chaudhari/eth-private/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [04-08|17:42:33.853] State schema set to default              scheme=hash
INFO [04-08|17:42:33.853] Writing custom genesis block
INFO [04-08|17:42:33.857] Persisted trie from memory database      nodes=4 size=478.00B time=3.539219ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [04-08|17:42:33.891] Successfully wrote genesis state         database=lightchaindata hash=38de1a..afc19d
g2022_ria_chaudhari@cloudshell:~$
```

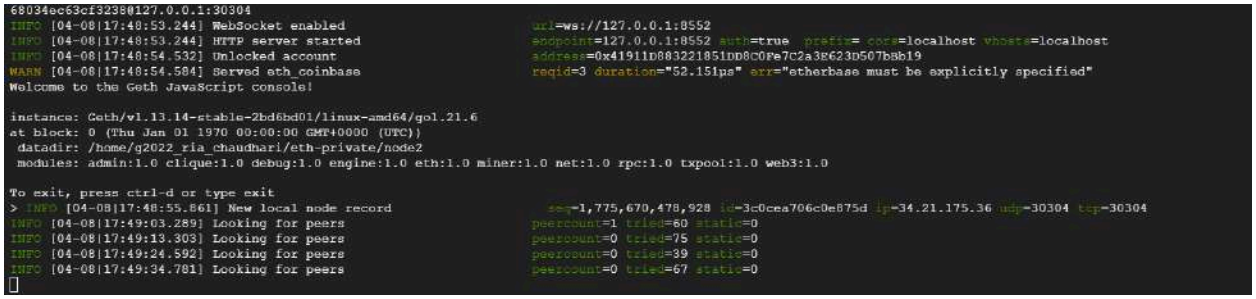
Step 6: Run Node 1 (signer)

```
g2022_ria_chaudhari@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
  --networkid 1234 \
  --port 30303 \
  --http --http.port 8545 \
  --http.api personal,eth,net,web3,miner \
  --unlock 0x$(NODE1) \
  --password ~/eth-private/node1/password.txt \
  --mine \
  --miner.etherbase 0x$(NODE1) \
  --allow-insecure-unlock \
  console
INFO [04-08|17:44:52.561] Maximum peer count                       ETH=50 total=50
INFO [04-08|17:44:52.564] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-08|17:44:52.568] Set global gas cap                       cap=50,000,000
INFO [04-08|17:44:52.569] Initializing the KZG library             backend=gokzg
INFO [04-08|17:44:52.700] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-08|17:44:52.701] Using pebble as the backing database
INFO [04-08|17:44:52.701] Allocated cache and file handles         database=/home/g2022_ria_chaudhari/eth-private/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-08|17:44:52.730] Opened ancient database                  database=/home/g2022_ria_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-08|17:44:52.730] State scheme set to already existing     scheme=hash
INFO [04-08|17:44:52.732] Initialising Ethereum protocol           network=1234 dbversion=8
INFO [04-08|17:44:52.732]
INFO [04-08|17:44:52.733] ---------------------------------------------------------------------------------------------------------------------------------------------------------
INFO [04-08|17:44:52.733] Chain ID:  1234 (unknown)
INFO [04-08|17:44:52.733] Consensus: Clique (proof-of-authority)
INFO [04-08|17:44:52.733]
INFO [04-08|17:44:52.733] Pre-Merge hard forks (block based):
INFO [04-08|17:44:52.733]  - Homestead:                   #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-08|17:44:52.733]  - Tangerine Whistle (EIP 150): #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-08|17:44:52.733]  - Spurious Dragon/1 (EIP 155): #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-08|17:44:52.733]  - Spurious Dragon/2 (EIP 158): #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-08|17:44:52.733]  - Byzantium:                   #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/byzantium.md)
INFO [04-08|17:44:52.733]  - Constantinople:              #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/constantinople.md)
INFO [04-08|17:44:52.733]  - Petersburg:                  #0       (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/petersburg.md)
INFO [04-08|17:44:52.733]  - Istanbul:                    #<nil>   (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/istanbul.md)
INFO [04-08|17:44:52.733]  - Berlin:                      #<nil>   (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/berlin.md)
INFO [04-08|17:44:52.733]  - London:                      #<nil>   (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/london.md)
```



Step 7: Open new terminal tab and run Node 2 (member)





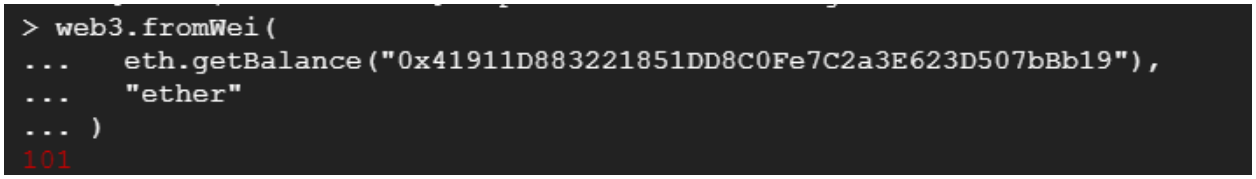
Step 8: Connect the two nodes In Node 1 console, get its enode

Now, switch to Node 2 console and paste the Node 1 enode

In Node 2 console, check they are connected

Step 9: Send a transaction In Node 1 console, send transaction to Node 2

and check the balance of Node 2



CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Ria Chaudhari **D20A** **14**

Blockchain Experiment 10

AIM: Explore the Cryptocurrency Landscape

THEORY:

Cryptocurrencies are decentralized digital assets built on blockchain technology, enabling secure, transparent, and immutable transactions without the need for intermediaries. Among various types of cryptocurrencies, **stablecoins** are designed to maintain a stable value by being pegged to fiat currency such as the US Dollar.

Stablecoins are a type of cryptocurrency designed to maintain a stable value by being pegged to an underlying asset such as a fiat currency (e.g., US Dollar), cryptocurrency, or algorithmic mechanism. Unlike volatile cryptocurrencies like Bitcoin, stablecoins aim to reduce price fluctuations, making them suitable for trading, payments, and decentralized finance (DeFi) applications.

Fiat-backed stablecoins are supported by real-world currencies like USD, EUR, etc. For every coin issued, an equivalent amount of money is stored in reserves (like banks).
Example: USDT (Tether), USDC, TUSD

Crypto-backed stablecoins are backed by other cryptocurrencies (like ETH). Since crypto prices fluctuate, extra collateral is required to maintain stability.

Example: DAI

Hybrid stablecoins use a combination of collateral (like USDC) and algorithmic mechanisms to maintain price stability.

Example: FRAX

This experiment focuses on analyzing stablecoins including USDT, USDC, DAI, FRAX, and TUSD using real-time data collected from multiple sources:

- **CoinGecko API:** Provided historical price, trading volume, and market capitalization data
- **CoinMarketCap / Crypto platforms:** Used for validation and market insights
- **Online resources:** Used for understanding adoption and use cases

Cryptocurrency Landscape:

- **Stablecoins** – Maintain stable value (USDT, USDC, DAI)
- **Store of Value** – Bitcoin
- **Utility Tokens** – Ethereum
- **Governance Tokens** – UNI, AAVE

Key Characteristics of Stablecoins:

- Low volatility compared to other cryptocurrencies
- Used for trading, payments, and DeFi applications
- Backed by fiat reserves, crypto assets, or hybrid mechanisms

TECHNOLOGICAL ANALYSIS:

Recent advancements in blockchain technology include:

- Consensus Mechanisms: PoW, PoS for secure validation
- Scalability: Layer-2 solutions like rollups
- Interoperability: Cross-chain communication
- Privacy: Zero-knowledge proofs

Stablecoins leverage these technologies to provide efficient and scalable financial solutions in decentralized ecosystems.

MARKET TRENDS & ADOPTION:

- Increasing use of stablecoins in DeFi platforms
- Growing merchant acceptance for payments
- Rising number of crypto wallets and users
- Integration with traditional financial systems
- Regulatory developments across countries

TASK PERFORMED:

1. Collected historical price and volume data using CoinGecko API
2. Processed data using Python libraries (Pandas, Requests)
3. Visualized data using Matplotlib and Seaborn
4. Compared price stability, volatility, and market dominance
5. Classified stablecoins based on type and use case

CODE:

```
!pip install requests pandas matplotlib seaborn -q

import requests
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import time
STABLECOINS = {
    'tether':          'USDT',
    'usd-coin':        'USDC',
```

```
    'dai':              'DAI',
    'frax':             'FRAX',
    'true-usd':         'TUSD'
}
DAYS   = 90         # last 90 days
COLOR = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
def fetch_market_chart(coin_id, days=DAYS, retries=3):
    url = f"https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart"
    params = {"vs_currency": "usd", "days": days, "interval": "daily"}

    for attempt in range(retries):
        try:
            r = requests.get(url, params=params, timeout=20)
            if r.status_code == 200:
                return r.json()
            elif r.status_code == 429:
                wait = 15 * (attempt + 1)    # 15s → 30s → 45s
                print(f"  ⏳ Rate limited. Waiting {wait}s before retry
{attempt+1}/{retries}...")
                time.sleep(wait)
            else:
                print(f"  ⚠ Could not fetch {coin_id}: HTTP
{r.status_code}")
                return None
        except Exception as e:
            print(f"  Error fetching {coin_id}: {e}")
            return None

    print(f"  Failed after {retries} retries: {coin_id}")
    return None

# --- Fetch historical data for each stablecoin ---
price_frames = {}
volume_frames = {}

print("Fetching data from CoinGecko API (with rate-limit handling)...\n")
for coin_id, ticker in STABLECOINS.items():
    print(f"  → Fetching {ticker} ({coin_id})...")
    data = fetch_market_chart(coin_id)
    if data:
```

```
            pdf = pd.DataFrame(data['prices'], columns=['ts', 'price'])
            vdf = pd.DataFrame(data['total_volumes'], columns=['ts',
'volume'])
            pdf['date'] = pd.to_datetime(pdf['ts'], unit='ms').dt.normalize()
            vdf['date'] = pd.to_datetime(vdf['ts'], unit='ms').dt.normalize()
            pdf = pdf.drop_duplicates('date').set_index('date')
            vdf = vdf.drop_duplicates('date').set_index('date')
            price_frames[ticker] = pdf['price']
            volume_frames[ticker] = vdf['volume']
            print(f" {ticker} fetched successfully!")

        time.sleep(8)    # ← increased from 1.2s to 8s between each call

prices_df  = pd.DataFrame(price_frames)
volumes_df = pd.DataFrame(volume_frames)
print("\n All data fetched!")
def fetch_current_data():
    """Fetch current market cap, volume, price for all stablecoins."""
    ids = ",".join(STABLECOINS.keys())
    url = "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd"
    params = {
        "vs_currency": "usd",
        "ids": ids,
        "order": "market_cap_desc",
        "per_page": 10,
        "page": 1,
        "sparkline": False
    }
    r = requests.get(url, params=params, timeout=15)
    return pd.DataFrame(r.json())
# Current snapshot
current_df = fetch_current_data()
print("\n Data collection complete!\n")
print(current_df[['name','current_price','market_cap','total_volume']].to_
string(index=False))
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Define colors
```

```
COLOR = ['blue', 'orange', 'green', 'red', 'purple']
sns.set_theme(style="darkgrid")

fig = plt.figure(figsize=(12, 6))

tickers = list(STABLECOINS.values())
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        plt.plot(prices_df.index, prices_df[ticker], label=ticker)

plt.axhline(1.0, linestyle='--', label='$1 Peg')

plt.title('Price Stability vs $1 Peg')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        deviation = (prices_df[ticker] - 1.0) * 100
        plt.plot(prices_df.index, deviation, label=ticker)

plt.axhline(0, linestyle='--')

plt.title('Deviation from $1 Peg (%)')
plt.xlabel('Date')
plt.ylabel('Deviation (%)')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(8,5))

if not current_df.empty:
```

```
    names = current_df['symbol'].str.upper()
    mcaps = current_df['market_cap'] / 1e9

    plt.bar(names, mcaps)

    for i, val in enumerate(mcaps):
        plt.text(i, val, f'{val:.1f}B', ha='center')

plt.title('Market Capitalization (Billion USD)')
plt.xlabel('Stablecoin')
plt.ylabel('Market Cap (Billion USD)')
plt.grid()

plt.show()
plt.figure(figsize=(6,6))

if not current_df.empty:
    labels = current_df['symbol'].str.upper()
    sizes = current_df['market_cap']

    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)

plt.title('Market Cap Dominance')
plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in volumes_df.columns:
        plt.plot(volumes_df.index, volumes_df[ticker] / 1e9, label=ticker)

plt.title('Trading Volume (Billion USD)')
plt.xlabel('Date')
plt.ylabel('Volume')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
```

```
    if ticker in prices_df.columns:
        returns = prices_df[ticker].pct_change()
        volatility = returns.rolling(7).std() * 100
        plt.plot(prices_df.index, volatility, label=ticker)

plt.title('7-Day Volatility (%)')
plt.xlabel('Date')
plt.ylabel('Volatility')
plt.legend()
plt.grid()

plt.show()
classification = {
    'Coin': ['USDT', 'USDC', 'DAI', 'FRAX', 'TUSD'],
    'Type': ['Fiat-backed', 'Fiat-backed', 'Crypto-backed',
             'Hybrid', 'Fiat-backed'],
    'Use Case': ['Trading & Liquidity', 'Payments & DeFi', 'DeFi Lending',
                 'Stable DeFi Ecosystem', 'Cross-border Payments']
}
cls_df = pd.DataFrame(classification)
print(cls_df.to_string(index=False))
```

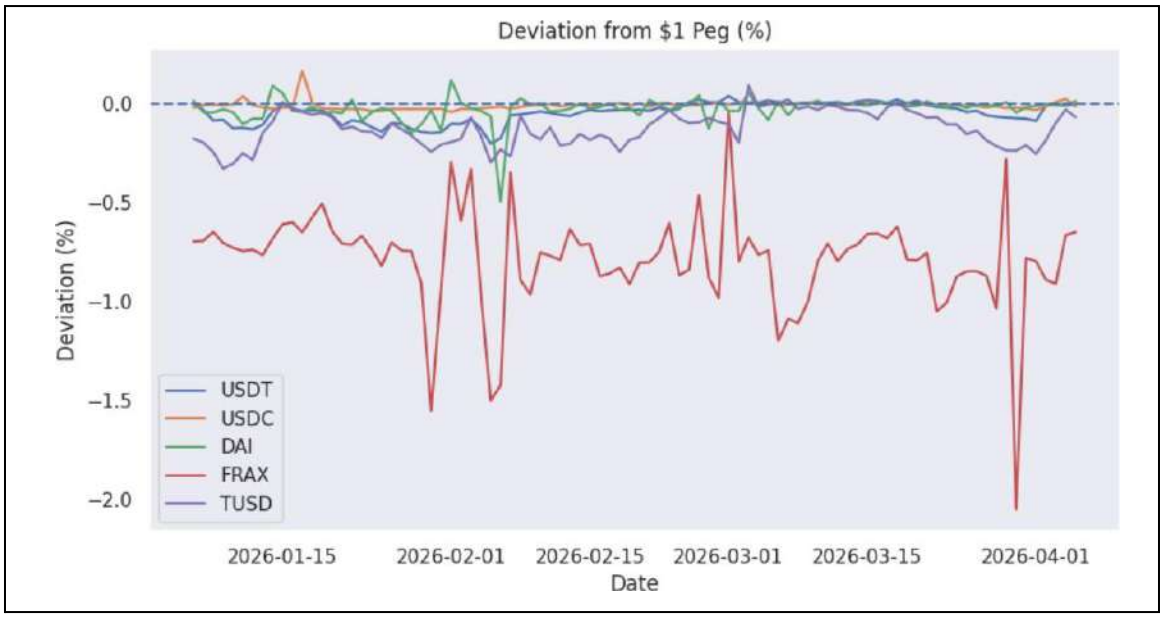
OUTPUT:

1. **Price Stability**

   This graph shows the price movement of stablecoins over time. All selected stablecoins remain close to the $1 value, demonstrating their ability to maintain price stability through pegging mechanisms.

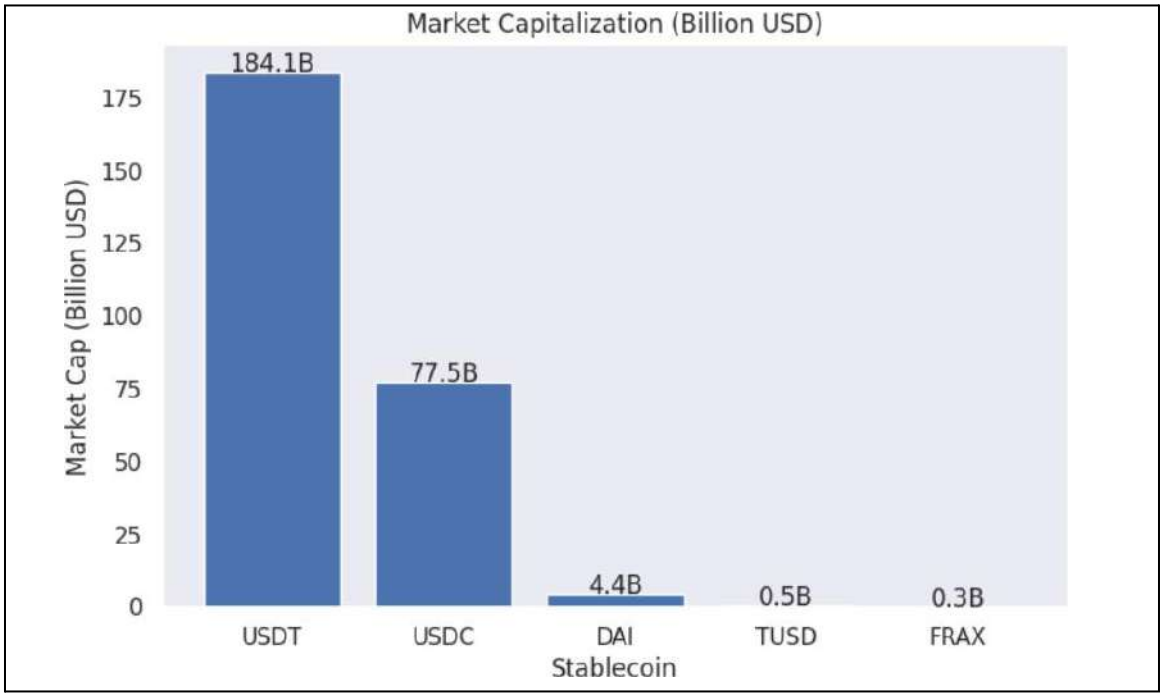
2. **Deviation from $1 Peg**

This graph represents the percentage deviation of stablecoin prices from the $1 peg. The minimal fluctuations indicate low de-pegging risk and high reliability of stablecoins in maintaining consistent value.



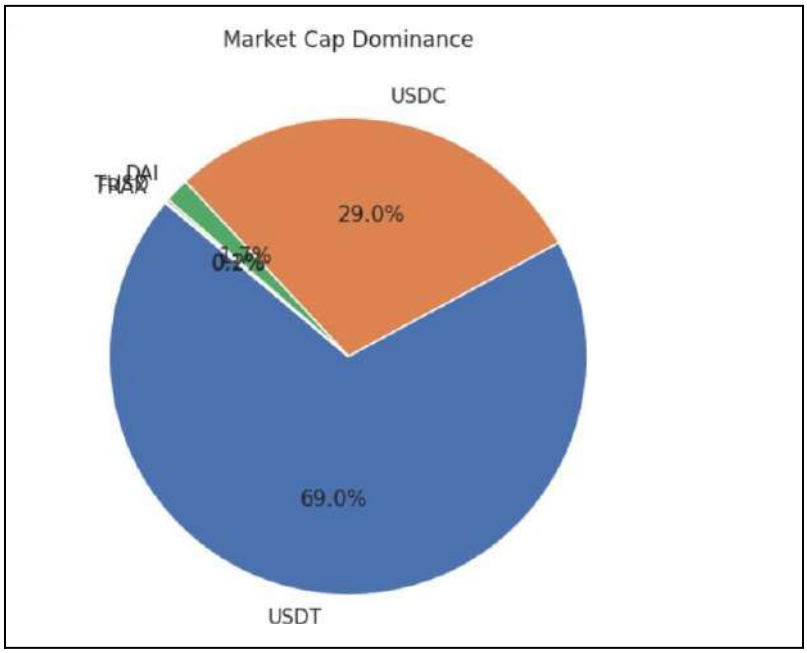
3. **Market Capitalization**

This graph compares the market capitalization of different stablecoins. It shows that USDT and USDC dominate the market, indicating higher adoption, liquidity, and trust among users.



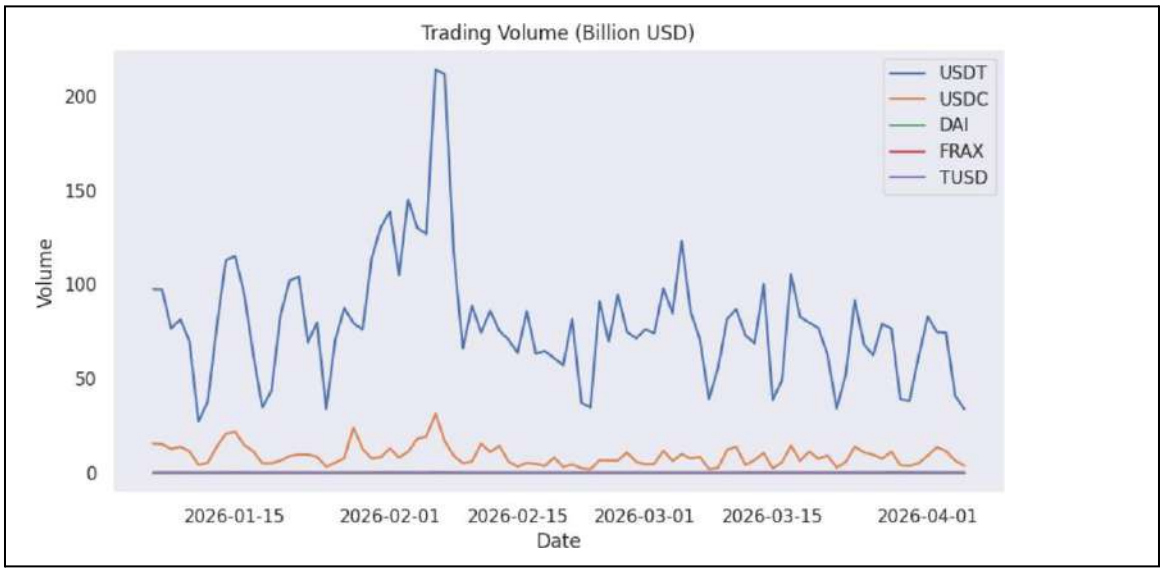
4. **Market Dominance**

This pie chart illustrates the proportion of total market share held by each stablecoin. It highlights that a few major stablecoins control a large portion of the market.

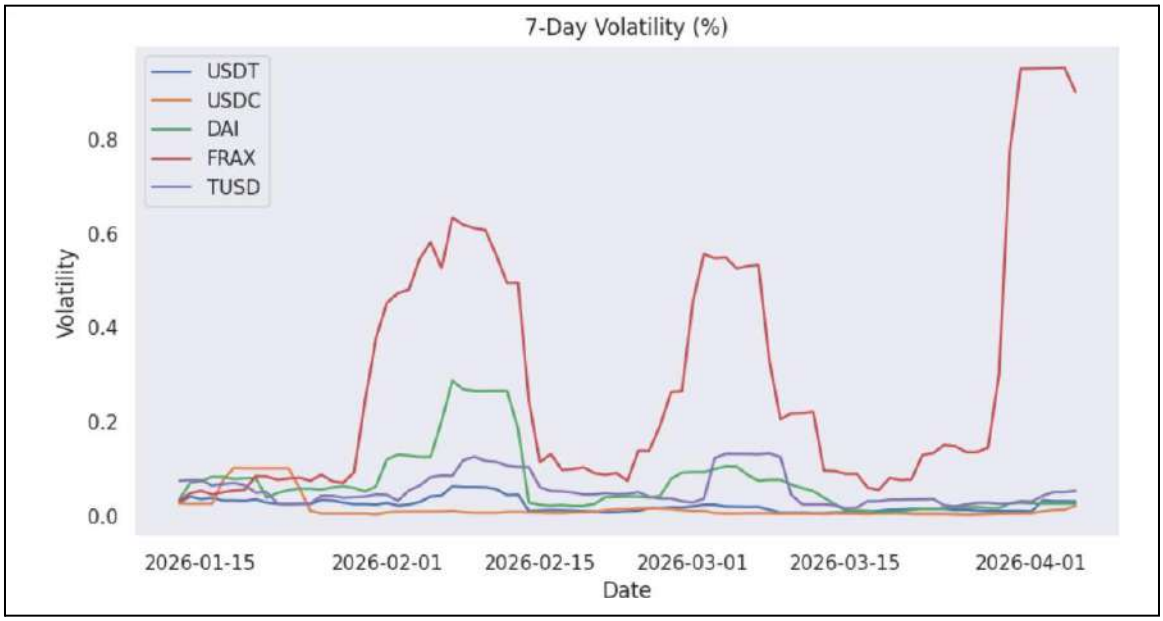


5. **Trading Volume**

This graph shows the daily trading volume of stablecoins over time. Higher trading volume reflects greater usage in transactions, exchanges, and DeFi applications.



6. **7-Day Volatility**
   This graph represents the rolling volatility of stablecoin prices. The low volatility confirms that stablecoins experience minimal price fluctuations compared to other cryptocurrencies.



The selected cryptocurrencies are classified based on their backing mechanism and use cases.

Coin	Type	Use Case
USDT	Fiat-backed	Trading & Liquidity
USDC	Fiat-backed	Payments & DeFi
DAI	Crypto-backed	DeFi Lending
FRAX	Hybrid	Stable DeFi Ecosystem
TUSD	Fiat-backed	Cross-border Payments

CONCLUSION:

The experiment successfully explored the cryptocurrency landscape by analyzing stablecoins using real-time data. The results demonstrated that stablecoins maintain price stability, exhibit low volatility, and play a crucial role in trading, payments, and decentralized finance. This study highlights the importance of blockchain technology in real-world financial applications.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Ria Chaudhari **D20A** **14**

Blockchain Experiment 11

AIM: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

PROCEDURE

Step 1: Update the system

Go to https://shell.cloud.google.com and run the following commands:

```
sudo apt update
sudo apt upgrade -y
```

```
g2022_anuprita_mhapankar@cloudshell:~$ sudo apt update
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Hit:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease
Hit:4 https://packages.cloud.google.com/apt gcsfuse-noble InRelease
Hit:5 https://packages.cloud.google.com/apt cloud-sdk InRelease
Hit:6 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease
Hit:7 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:8 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:9 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:10 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Hit:11 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 3,917 B in 6s (639 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
55 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
g2022_anuprita_mhapankar@cloudshell:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  libpam-systemd libsystemd-shared libsystemd0 libudev1 qemu-user qemu-user-static systemd systemd-dev systemd-sysv udev
The following packages will be upgraded:
  google-cloud-cli-app-engine-go google-cloud-cli-app-engine-java google-cloud-cli-app-engine-python google-cloud-cli-app-engine-python-extras
  google-cloud-cli-bigtable-emulator google-cloud-cli-cbt google-cloud-cli-cloud-run-proxy google-cloud-cli-datastore-emulator google-cloud-cli-gke-gcloud-auth-plugin
  google-cloud-cli-istioctl google-cloud-cli-kpt google-cloud-cli-local-extract google-cloud-cli-minikube google-cloud-cli-nomos google-cloud-cli-package-go-module
  google-cloud-cli-pubsub-emulator google-cloud-cli-skaffold google-cloud-cli-spanner-cli kubectl libarchive13t64 libgdk-pixbuf-2.0-0 libgdk-pixbuf2.0-common libpq-dev
  libpq5 libruby3.2 linux-libc-dev php8.3-bcmath php8.3-cgi php8.3-cli php8.3-common php8.3-dev php8.3-mbstring php8.3-mysql php8.3-opcache php8.3-readline php8.3-xml
  ruby3.2 ruby3.2-dev
38 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
10 not fully installed or removed.
Need to get 603 MB of archives.
After this operation, 3,519 kB disk space will be freed.
Get:1 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 libpq-dev amd64 18.3-1.pgdg24.04+1 [167 kB]
Get:2 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 libpq5 amd64 18.3-1.pgdg24.04+1 [255 kB]
Get:3 https://packages.cloud.google.com/apt cloud-sdk/main all google-cloud-cli-app-engine-python all 564.0.0-0 [4,032 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libarchive13t64 amd64 3.7.2-2ubuntu0.6 [382 kB]
Get:5 https://packages.cloud.google.com/apt cloud-sdk/main amd64 google-cloud-cli-app-engine-go amd64 564.0.0-0 [3,007 kB]
Get:6 https://packages.cloud.google.com/apt cloud-sdk/main all google-cloud-cli-app-engine-java all 564.0.0-0 [160 MB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgdk-pixbuf2.0-common all 2.42.10+dfsg-3ubuntu3.3 [8,302 B]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgdk-pixbuf-2.0-0 amd64 2.42.10+dfsg-3ubuntu3.3 [147 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 ruby3.2-dev amd64 3.2.3-1ubuntu0.24.04.7 [399 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libruby3.2 amd64 3.2.3-1ubuntu0.24.04.7 [5,341 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-libc-dev amd64 6.8.0-107.107 [2,096 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 php8.3-xml amd64 8.3.6-0ubuntu0.24.04.8 [126 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 php8.3-readline amd64 8.3.6-0ubuntu0.24.04.8 [13.5 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 php8.3-opcache amd64 8.3.6-0ubuntu0.24.04.8 [371 kB]
```

Step 2: Install Required Dependencies

```
g2022_anuprita_mhapankar@cloudshell:~$ sudo apt install -y jq curl git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
g2022_anuprita_mhapankar@cloudshell:~$
```

Then verify everything is present:

```
node -v
npm -v
git --version
docker --version
docker ps
```

```
g2022_anuprita_mhapankar@cloudshell:~$ node -v
v24.14.1
g2022_anuprita_mhapankar@cloudshell:~$ npm -v
11.12.1
g2022_anuprita_mhapankar@cloudshell:~$ git --version
git version 2.43.0
g2022_anuprita_mhapankar@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
g2022_anuprita_mhapankar@cloudshell:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
g2022_anuprita_mhapankar@cloudshell:~$
```

Step 3: Fix Docker Compose

```
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL
https://github.com/docker/compose/releases/download/v2.24.5/docker-compose
-linux-x86_64 \
  -o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null ||
true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
docker compose version
```

```
docker-compose version
```

```
g2022_anuprita_mhapankar@cloudshell:~$ DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins

curl -SL https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
  -o $DOCKER_CONFIG/cli-plugins/docker-compose

chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose

docker compose version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 58.5M  100 58.5M    0     0  79.4M      0 --:--:-- --:--:-- --:--:-- 79.4M
Docker Compose version v2.24.5
g2022_anuprita_mhapankar@cloudshell:~$ sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null || true
g2022_anuprita_mhapankar@cloudshell:~$ sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
g2022_anuprita_mhapankar@cloudshell:~$ docker compose version
docker-compose version
Docker Compose version v2.24.5
Docker Compose version v2.24.5
g2022_anuprita_mhapankar@cloudshell:~$
```

Step 4: Clone Fabric Samples

```
cd $HOME
```

```
git clone https://github.com/hyperledger/fabric-samples
```

```
cd fabric-samples
```

```
g2022_anuprita_mhapankar@cloudshell:~$ cd $HOME
g2022_anuprita_mhapankar@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.13 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
g2022_anuprita_mhapankar@cloudshell:~$ cd fabric-samples
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 5: Install Fabric Binaries and Docker Images

```
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-
fabric.sh
```

```
chmod +x install-fabric.sh
```

```
./install-fabric.sh docker samples binary
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /home/g2022_anuprita_mhapankar/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  22.5M      0  0:00:05  0:00:05 --:--:-- 24.4M
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
===> Will unpack to: /home/g2022_anuprita_mhapankar/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 31.6M  100 31.6M    0     0  15.9M      0  0:00:01  0:00:01 --:--:-- 31.6M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
===>  ghcr.io/hyperledger/fabric-peer:2.5.15
```

```
===>  ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Pull complete
6f4ebca3e823: Pull complete
b9649d3f054a: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
===> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 6: Verify Docker images

```
docker images | grep fabric
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 7: Set Environment Variables

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

g2022_anuprita_mhapankar@cloudshell:~/fabric-samples$
```

Step 8: Start the Network and Create Channel

```
cd $HOME/fabric-samples/test-network
./network.sh up createChannel
```

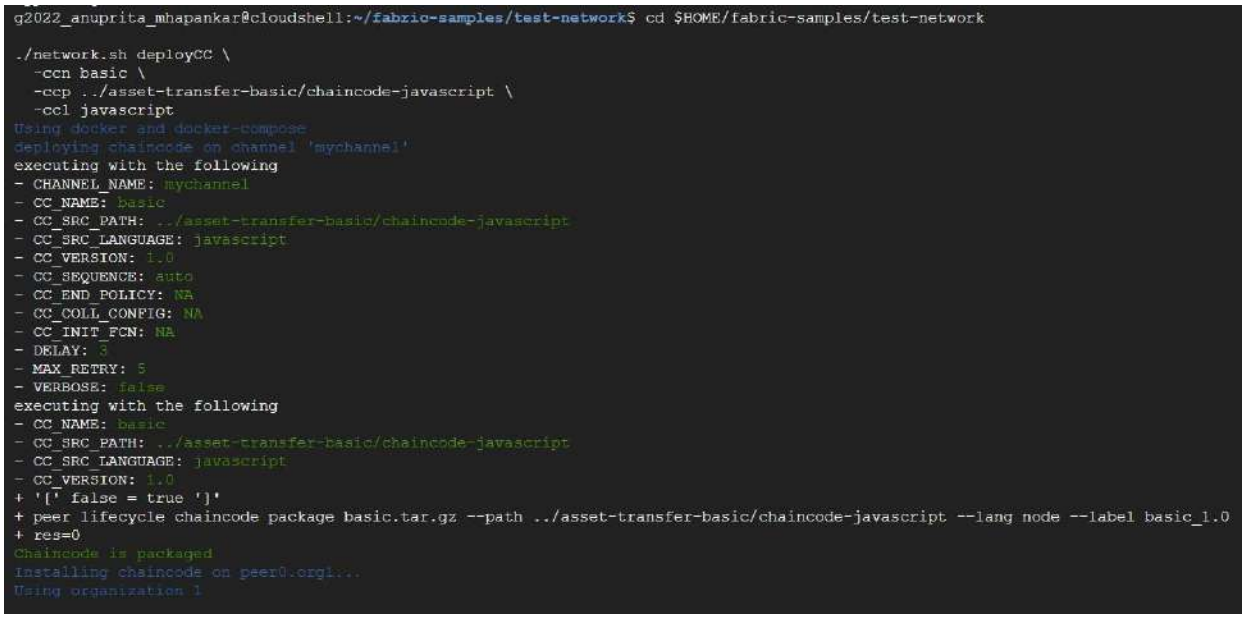
```
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb' with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/g2022_anuprita_mhapankar/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
 ! Network fabric_test                       Created
 ! Volume "compose_peer0.org1.example.com"   Created
 ! Volume "compose_peer0.org2.example.com"   Created
 ! Volume "compose_orderer.example.com"      Created
 ✓ Container orderer.example.com             Started
 ✓ Container peer0.org1.example.com          Started
 ✓ Container peer0.org2.example.com          Started
CONTAINER ID   IMAGE                                  COMMAND             CREATED        STATUS                  PORTS
          NAMES
```

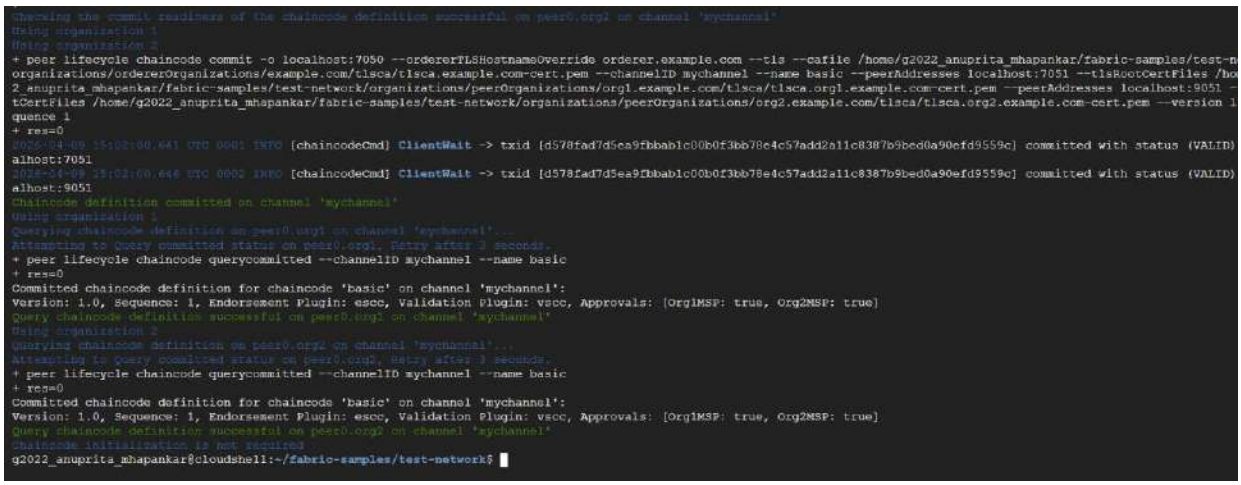
```
/home/g2022_anuprita_mhapankar/fabric-samples/test-network/../bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-04-09 14:56:55.392 UTC 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-04-09 14:56:55.402 UTC 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-04-09 14:56:55.402 UTC 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.Options unset, setti
n_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-09 14:56:55.402 UTC 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/g2022_anuprita_mhapankar/fa
configtx.yaml
2026-04-09 14:56:55.405 UTC 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-04-09 14:56:55.405 UTC 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-04-09 14:56:55.410 UTC 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
        "name": "mychannel",
        "url": "/participation/v1/channels/mychannel",
        "consensusRelation": "consenter",
        "status": "active",
        "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
```

```
Generating anchor peer update transaction for Org2 on channel mychannel
++ jq '.channel_group.groups.Application.groups.Org2MSP.values += {"AnchorPeers":{"mod_policy": "Admins","value":{"anchor_peers": [{"host": "peer0.org2.example.com","port": 90
51}]},"version": "0"}}' /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json
++ res=0
++ configtxlator proto_encode --input /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json --type common.Config --output /home/g2022
_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/original_config.pb
++ configtxlator proto_encode --input /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/Org2MSPmodified_config.json --type common.Config --output /h
ome/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/modified_config.pb
++ configtxlator compute_update --channel_id mychannel --original /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/original_config.pb --updated /ho
me/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/modified_config.pb --output /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifa
cts/config_update.pb
++ configtxlator proto_decode --input /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/config_update.pb --type common.ConfigUpdate --output /home/g
2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/config_update.json
++ jq .
+++ cat /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}},"data":{"config_update":{' '"channel_id":' '"mychannel",' '"isolated_data":' '{},' '"read
_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"
",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"poli
cy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}},' '"values":' '{' '"MSP":' '{' '"mod_policy":' '"",' '"v
alue":' null, '"version":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '
"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '},' '"write_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '
"mod_policy":' '"Admins",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"polic
y":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"
version":' '"0"' '}' '},' '"values":' '{' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '[' '{' '"host":' '"peer0.org2.example.com",' '"por
t":' 9051 '}' ']' '},' '"version":' '"0"' '},' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",
' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '}}}'
++ configtxlator proto_encode --input /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --outp
ut /home/g2022_anuprita_mhapankar/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-04-09 14:57:06.553 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-09 14:57:06.595 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
g2022_anuprita_mhapankar@cloudshell:~/fabric-samples/test-network$
```

Step 9: Deploy Chaincode

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

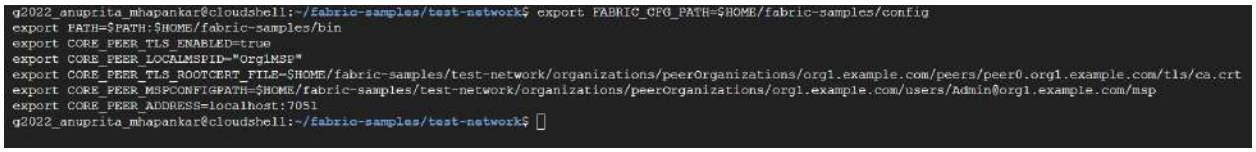




Step 10: Set Peer Environment Variables

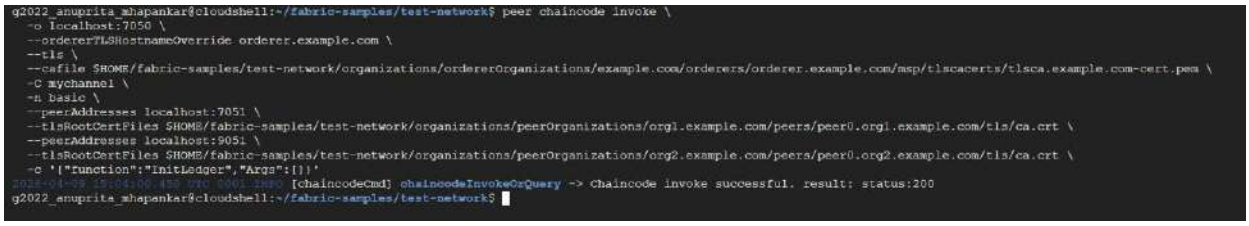
```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export PATH=$PATH:$HOME/fabric-samples/bin

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organization
s/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.c
rt
```

```
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```



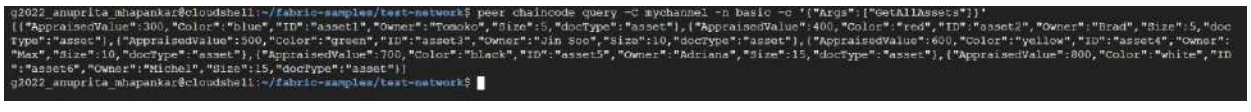
Step 11: Initialize Ledger

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"InitLedger","Args":[]}'
```



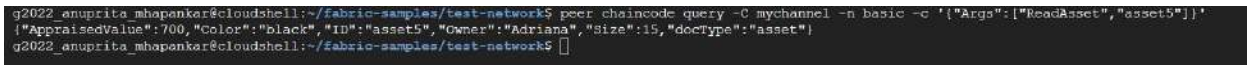
Step 12: Query All Assets

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```



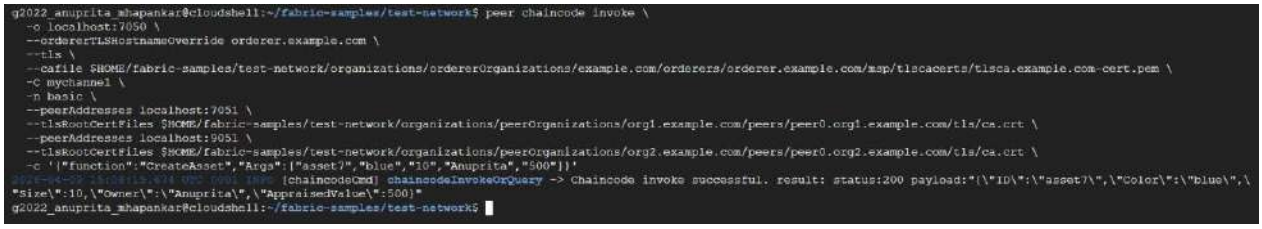
Step 13: Read a Specific Asset

```
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset5"]}'
```



Step 14: Create a New Asset

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/examp
le.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.
pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.exa
mple.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.exa
mple.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c
'{"function":"CreateAsset","Args":["asset7","blue","10","Anuprita","500"]}
'
```



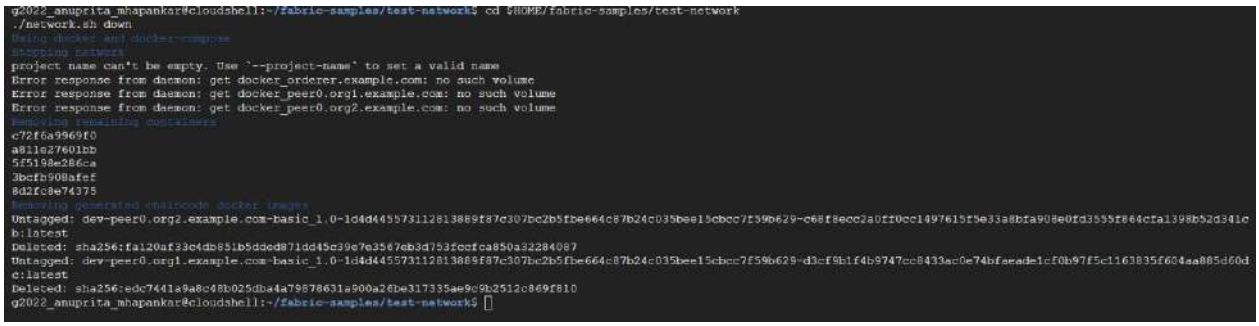
Step 15: Transfer Asset

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/examp
le.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.
pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.exa
mple.com/peers/peer0.org1.example.com/tls/ca.crt \
```

```
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.exa
mple.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"TransferAsset","Args":["asset7","Sneha"]}'
```

Step 16: Stop the Network

```
cd $HOME/fabric-samples/test-network
./network.sh down
```



CONCLUSION:

The Hyperledger Fabric network was successfully implemented and chaincode was deployed. Asset operations such as creation, querying, and transfer were performed successfully. This experiment demonstrated secure and efficient asset management using a permissioned blockchain system.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 12

Aim: Mini Project

Roll No.	14
Name	Ria Chaudhari
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

BLOCKCHAIN-BASED DMART RETAIL SYSTEM

Group Members:
14 - Ria Chaudhari
15 - Eesha Chavan
34 - Anuprita Mhapankar
43 - Sneha Patra

PROBLEM STATEMENT

Lack of transparency in order records and supply chain tracking makes it difficult for customers to verify product authenticity. Fake or duplicate product reviews mislead consumers and damage retailer credibility. Data tampering risks in centralized systems compromise the integrity of transaction records.

Dependency on centralized databases creates single points of failure and security vulnerabilities. Key stakeholders affected include customers seeking trust, retailers needing credibility, and administrators requiring secure data management. These challenges demand a decentralized, tamper-proof solution.

PROBLEM STATEMENT

Lack of transparency in order records makes it difficult for customers to verify transaction history and track purchases accurately.

Fake or duplicate product reviews undermine consumer trust and distort purchasing decisions, damaging retailer credibility.

Data tampering risks and dependency on centralized systems create vulnerabilities to manipulation and single points of failure.

EXISTING SYSTEM (TRADITIONAL APPROACH)

Current System Issues

- Centralized databases store orders
- Data can be modified or deleted
- Fake reviews are possible
- No tamper-proof records
- Customer trust is compromised

Root Cause Analysis

- Weak security protocols
- No verification mechanisms
- Lack of trust between parties
- Single point of failure
- No audit trail capability

System Limitations

- Limited transparency for customers
- No immutable record keeping
- Vulnerable to insider threats
- Dependency on central authority
- Poor accountability measures

WHY BLOCKCHAIN?

Immutable Data Storage: Once data is recorded on the blockchain, it cannot be altered or deleted. This ensures permanent, tamper-proof records of all transactions and reviews.

Transparency & Security: All transactions are visible to authorized participants while being cryptographically secured. This builds trust between customers, retailers, and administrators.

Decentralization: No single point of failure or control. Eliminates the need for intermediaries, reducing costs and removing dependency on centralized systems.

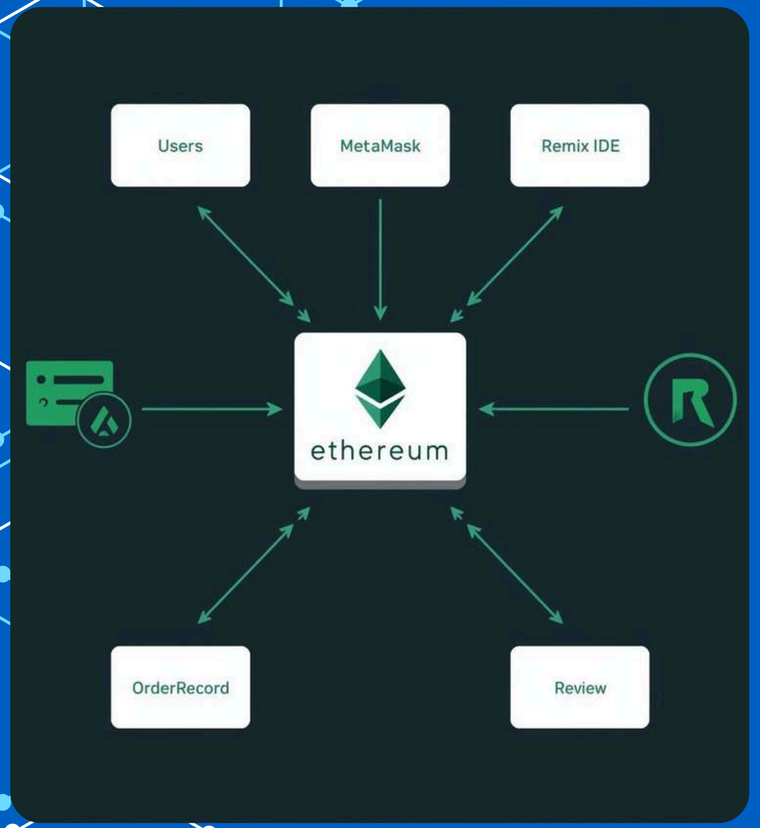
TYPE OF BLOCKCHAIN USED

Public Blockchain: Ethereum Test Network (Sepolia)
Our D-Mart Smart Retail System utilizes the Ethereum Sepolia Test Network, a public blockchain infrastructure that provides the ideal foundation for development and testing.

- **Open Access:** Anyone can participate, verify transactions, and interact with smart contracts without permission barriers
- **Full Transparency:** All order records and review verifications are publicly visible and auditable on the blockchain explorer
- **Cost-Effective Testing:** Sepolia provides free test ETH through faucets, enabling comprehensive testing without financial investment
- **Production-Ready Architecture:** Code developed on Sepolia can be seamlessly deployed to Ethereum mainnet when ready for real-world implementation
- **Decentralized Validation:** Transactions are verified through consensus mechanisms, eliminating single points of failure

DATA & INFORMATION PROVIDED

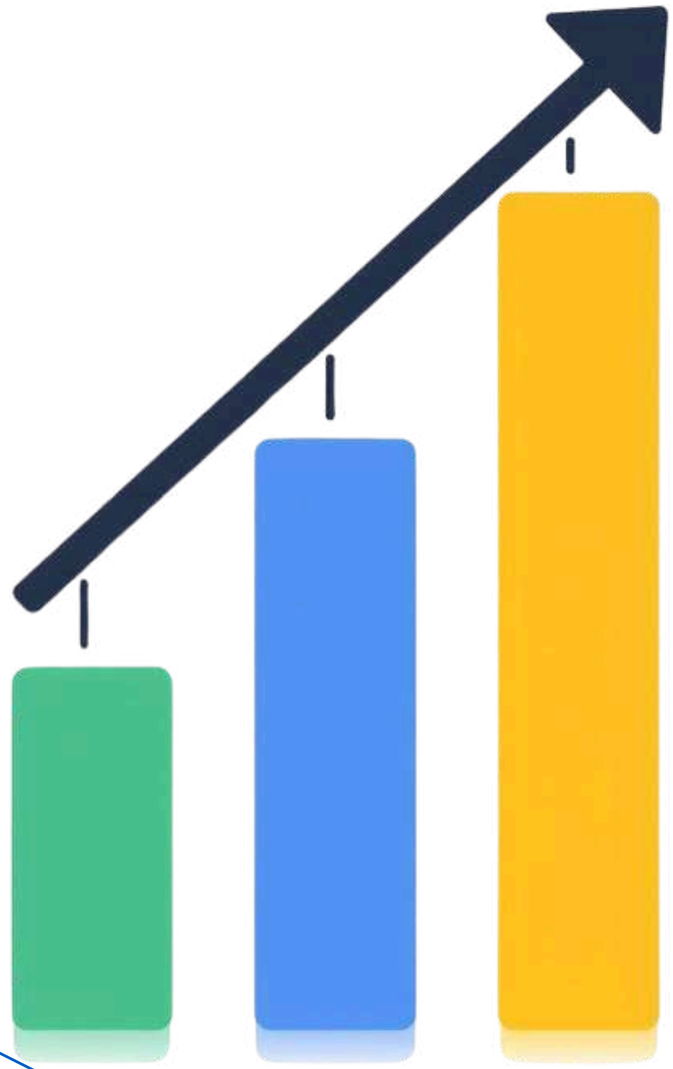
- Order Details: Buyer wallet address, product IDs, total amount, and timestamp are recorded on the blockchain, ensuring complete traceability of every transaction.
- Review Details: Product ID, reviewer wallet address, rating score, and timestamp are stored immutably, preventing fake or duplicate reviews from being submitted.
- Smart Contract Storage: All data is permanently stored via smart contracts on Ethereum, ensuring records cannot be modified, deleted, or tampered with after creation.



SYSTEM ARCHITECTURE

- Users (Customers): Interact with system via wallet
- Ethereum Blockchain: Sepolia test network
- Smart Contracts: OrderRecord & ReviewVerification
- MetaMask Wallet: User authentication & transactions
- Remix IDE: Smart contract development & deployment
- Data flows securely from users through blockchain validation

WORKING MECHANISM



1. User places order through the retail platform
2. Transaction is created with order details
3. Transaction sent to Ethereum blockchain network

1. Validated via consensus mechanism
2. Data stored immutably in blockchain block
3. Output visible and permanently unalterable

KEY TECHNOLOGIES USED

Ethereum Blockchain & Solidity: Ethereum provides the decentralized platform while Solidity enables smart contract development for secure, automated transactions.

Remix IDE & MetaMask Wallet: Remix IDE offers browser-based smart contract development, while MetaMask enables secure wallet connection for blockchain interactions.

Sepolia Test Network: Ethereum's test network allows safe deployment and testing of smart contracts using test ETH before mainnet launch.

SWOT ANALYSIS

Strengths
- Highly secure data storage
- Tamper-proof records
- Full transparency
- Decentralized architecture
- Builds customer trust

Opportunities & Threats
- Real-world retail adoption potential
- Process automation capabilities
- Regulatory uncertainty
- Scalability challenges
- Market resistance to change

Weaknesses
- Blockchain complexity for users
- Transaction (gas) fees
- Learning curve for retailers
- Integration challenges
- Limited developer expertise

PROPOSED ENHANCEMENTS

Mobile App Integration: Develop a user-friendly mobile application for customers to place orders, track deliveries, and submit verified reviews directly through their smartphones with blockchain wallet connectivity.

AI-Based Recommendation System: Implement machine learning algorithms to analyze purchase history and verified reviews, providing personalized product suggestions while maintaining data privacy on the blockchain.

Real-Time Analytics Dashboard: Create an interactive dashboard for retailers and administrators to monitor transactions, review authenticity metrics, and supply chain data with live blockchain verification.

RECOMMENDED SOLUTION

Blockchain-Based Order & Review System
- Immutable order records stored on Ethereum blockchain
- Verified product reviews preventing fake or duplicate entries
- Smart contracts automate transaction validation

Key Justifications:
- High Security: Cryptographic protection against data tampering
- Full Transparency: All transactions publicly verifiable
- Trust Building: Decentralized system eliminates single point of failure
- Cost Efficiency: Reduced intermediary and audit costs

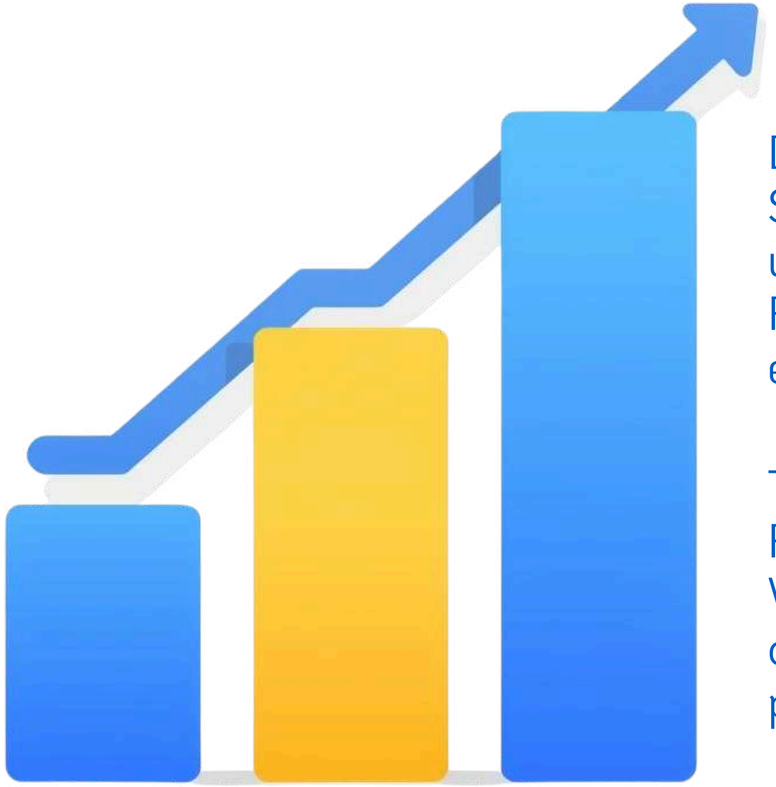
FEASIBILITY ANALYSIS

Technical Feasibility
Fully implementable using Ethereum blockchain and Solidity smart contracts. Proven technology stack with extensive documentation. Compatible with existing web infrastructure and MetaMask wallet integration.

Economic Feasibility
Cost-effective implementation using Sepolia test network with free test ETH. Minimal infrastructure investment required. No licensing fees for open-source tools like Remix IDE and MetaMask.

Scalability Potential
Cloud integration capability for handling increased transaction volumes. Modular smart contract design allows easy feature expansion. Ready for migration to mainnet when business scales.

IMPLEMENTATION PLAN



Design & Development Phase
System architecture planning and smart contract coding using Solidity. Define data structures for OrderRecord and ReviewVerification contracts. Set up development environment with Remix IDE and MetaMask wallet integration.

Testing & Deployment
Rigorous testing on Sepolia test network using test ETH. Validate all transaction flows, consensus mechanisms, and data immutability. Upon successful testing, proceed to production deployment for live retail environment.

IMPACT ANALYSIS

Societal Impact: The blockchain-based system builds stronger customer trust through verified reviews and transparent order records. Customers can confidently make purchasing decisions knowing data cannot be manipulated.

Environmental Impact: Digital blockchain records eliminate the need for paper-based documentation. Paperless transactions reduce waste and support sustainable retail operations with lower carbon footprint.

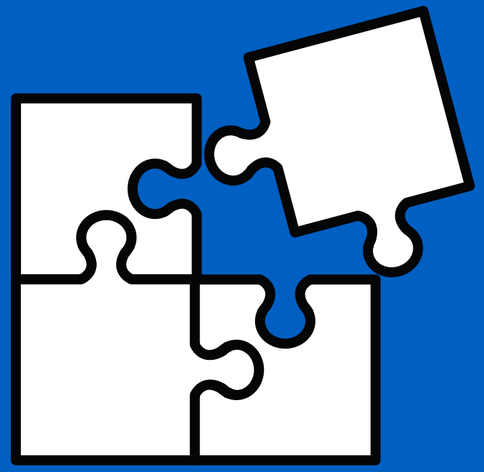
Ethical Impact: Data transparency ensures fair practices for all stakeholders. Privacy protection through cryptographic security safeguards customer information while maintaining accountability.

CHALLENGES & LIMITATIONS

- Blockchain Complexity: The technology requires specialized knowledge for development and maintenance. Smart contract programming demands expertise in Solidity, and debugging decentralized applications is more challenging than traditional systems.

- Transaction (Gas) Fees: Every blockchain operation incurs costs. During network congestion, gas fees can spike significantly, making frequent transactions economically challenging for retail operations with high volume.

- User Adoption & Regulatory Issues: Customers need crypto wallets and blockchain understanding. Additionally, evolving regulations around cryptocurrency and data storage create compliance uncertainties for retail implementations.



THANK YOU!

Questions & Discussion Welcome